

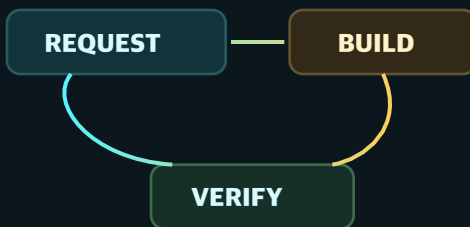
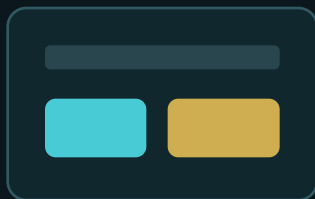
AI時代のエージェントックコーディング

Agentic Coding Tauri 2

依頼・実装・検証で完成させる

AIエージェント時代のデスクトップアプリ

Rust · SvelteKit · Tauri · 実践デプロイ



Ted Park

依頼 | 実装 | 検証 | 反復

Agentic Coding Tauri 2

リクエスト・ビルド・検証—AIエージェントと6つのアプリを完成させる

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Hiragino Kaku Gothic ProN | Menlo.

目次

まえがき	1
教育用資料と提供範囲	1
この章の流れ	2
学習の目標	2
本書を書きながら	2
検証の基準を移す	3
本書が扱うこと	4
本書が扱わないこと	4
本書の技術スタックがなじみにくい理由	4
6つのプロジェクトを一目で	5
本書の読み方	6
全体の構成	6
前提知識	7
技術スタックを一目で	8
開発環境	9
著者からのメモ	10
この章で必ず覚えておくこと	10
第1章. スピードよりも大切なのは検証だ—主導権を握るべき理由	12
この章の流れ	13
学習目標	14
1.1 計画なしのAIコーディングはなぜ長く持たないのか	14
1.2 なぜ今なのか	15
1.3 開発、それ以前は: 自分でコードを書いていた	15

1.4 今は: AIが書いて、自分が決める	16
1.5 「読むだけでよい」という言葉が半分しか正しくない理由	17
1.6 役割の転換: コードレビューから要件整理へ	18
1.7 ワンシーン—readbooks.ai のPDFパーサーのエピソード	19
1.8 本書で繰り返し登場する六つの事例	20
1.9 本書を読み終えたあとに残るもの	20
この章で必ず覚えておくこと	21
第2章. Request, Build, Verify —ループの形	22
この章の流れ	23
学習目標	23
この章で使う技術	23
2.1 ひとつのループ、サイズが変わるだけ	24
2.2 ステップ1 —要求	25
短すぎる要求	25
長すぎる要求	26
2.3 ステップ2 —実装	26
2.4 ステップ3 —検証	27
2.5 ステップ4 —反復	27
2.6 「完璧なプロンプトひとつでアプリ完成」が幻想である理由	28
2.7 骨組み優先戦略—トップダウンAI開発	29
ステップ1: 骨組み—アーキテクチャプロンプト	29
ステップ2: 詳細実装—一度にひとつのモジュール	30
ステップ3: バグ修正—コンパイル、実行、修正	30
ステップ4: 機能QA —各機能を最初から最後まで検証	31
ステップ5: 仕上げ—エッジケース、性能、UX	31
この順序が重要な理由	32
2.8 実例—MandaiのDBスキーマループ	32
サイクル1 —要求	32
サイクル1 —実装	33
サイクル1 —検証	33
2.9 基本プロンプトテンプレート: 何を / なぜ / 制約 / 形式	34

2.10 ループを回すときの注意点	35
この章で必ず覚えておくこと	36
第2サイクルの修正版SQL	36
第3章 AIコーディングツール—プロンプトだけで回す開発ループ	37
本章の流れ	38
学習目標	38
3.1 本章の範囲	38
3.2 プロンプトだけのループコードを直接打たずに開発する	39
ステップ1: 自然言語プロンプトを書く	39
ステップ2: AIがコードを生成する	39
ステップ3: 結果をレビューし、動作を検証する	40
ステップ4: 受け入れ、却下、または修正	40
ステップ5: 検証	40
ステップ6: 証拠をもとに改善する	41
3.3 プロンプトパターン—実戦の工具箱	41
パターン1: 骨組みプロンプト—コードよりアーキテクチャが先	41
パターン2: 範囲を絞った修正	42
パターン3: スペック先行プロンプト—実装よりシグネチャが先	42
パターン4: 否定の制約—してはいけないことを書く	43
パターン5: ファイル参照で文脈を固定する	44
パターン6: フェーズ認識プロンプト—現在のフェーズを明示する	44
パターンの組み合わせ	45
3.4 通しセッション—ポモドーロタイマーCommandを組み立てる	45
ラウンド1 —最初のリクエスト	45
ラウンド2 —欠けている型を解決する	47
ラウンド3 —スコープ問題 + キャンセル機能	47
セッションのまとめ	48
3.5 まとめ	48
最小検証コマンド	48
第4章. ツールと環境—Claude Code、Cursor、Codex、GitHub Copilot	50
この章の流れ	51
学習目標	51

4.1 4つのツール、4つのアプローチ	52
4.2 いつ何を使うか	53
4.3 Tauri 2 + SvelteKit + Rustの最小環境	54
バージョン固定の方針	54
必須インストール手順	54
AIツールの設定	55
SvelteKitを作ってからTauriを追加する	56
インストールが失敗したとき—よくある3つの原因	57
4.4 最初の30分—スキヤフォールドから最初のコミットまで	58
すぐに確認 (5分)	58
プロジェクト設定 (10分)	58
最初の生産的な変更 (15分)	59
確認チェックリスト	60
4.5 .cursorrules / CLAUDE.md —規約ファイルの活用	61
CLAUDE.md (Claude Code用) —実例	61
.cursorrules —同じ概念、別のファイル名	62
規約ファイルの効果	62
4.6 AIに与える制約—スタックとバージョンの明示	63
4.7 もう少し知っておく: MCP、Agent、スラッシュコマンド	63
4.8 ツール選びで守る5つの原則	64
この章で必ず覚えておくこと	64
第5章. Tauri 2の基礎—中核パターンとAIのミス防止	66
この章の流れ	67
学習目標	68
5.1 Tauri 2アーキテクチャの概観—明確な境界が重要な理由	68
ネイティブUIではなくWebViewを使う理由	69
5.2 命令—#[tauri::command] パターン	69
命令の名前を付ける原則	69
最小の例	69
失敗しうる命令	70
複数フィールドのデータ	71
5.3 状態管理—tauri::State<T>	72

基本パターン: RwLock	72
ロックは型名ではなく臨界区間で選ぶ	73
初期化コストの大きいリソース	74
5.4 イベント—emitとlisten	75
バックエンドからフロントエンドへ送る	75
ストリーミングのパターン	76
5.5 プラグイン—tauri-plugin-*エコシステム	77
AIアプリでよく使うプラグイン	77
5.6 権限とcapability —Tauri 2の境界	78
plugin権限と独自命令を区別する	78
5.7 フロントエンド—SvelteKit 2 + Svelte 5 + Tauri	79
Svelte 5 runeとTauriイベントの相性	79
APIラッパーで命令呼び出しをまとめる	80
5.8 プロジェクト構造—リファレンスレイアウト	81
5.9 AI開発のためのTauri 2メンタルモデル	82
プロンプティング戦略	82
5.10 検証チェックリスト	82
第6章 Mandai の設計—AI にどれくらい大きな単位で任せられるか	84
この章の流れ	85
学習目標	86
6.1 Mandai は最初どんなアプリだったか	86
6.2 最初の失敗—「全部を一度に作って」	87
問題 1. セル ID に UUID を使った	87
問題 2. Pomodoro セッションがタスクと繋がっていなかった	88
問題 3. フォルダ構造は立派だが中身が空だった	88
6.3 サイズ調整—AI が一度に処理できる単位	88
セル ID 体系—二度の問い返しで確定	89
Pomodoro セッション—分析シナリオまで追いかける	90
AI チャット—選択肢を提示してもらう	90
6.4 AI の問い返しを設計ツールとして使う	91
6.5 三層に分ける: 画面・データ・コマンド	93
層 1: 画面の定義	93

層 2: データ要件	94
層 3: コマンドの列挙	94
6.6 AI チャンクサイズの法則	96
6.7 ビルドの途中で計画が変わるとき	97
ピボット 1: セル ID 衝突問題	97
ピボット 2: リアルタイムタイマー問題	98
ピボットが多すぎるとき	99
6.8 まとめ	99
第7章 DBスキーマ—最初のRequest・Build・Verifyループ	101
なぜこの章が「ループの話」なのか	102
どこまで自分で手を動かして写すか	102
この章の流れ	102
学習目標	103
7.1 DuckDBを選んだ理由	103
まず見るべきもの—7テーブルの関係	103
なぜDuckDBか	104
DuckDB vs SQLite —AIがよく間違える部分	104
7.2 1回目のループ—リクエスト草案を書く	105
接続は一度開いて共有する	105
スキーマのリクエストを送る	106
草案レビュー—何が抜けていたか	107
7.3 2回目のループ—穴を埋める	108
修正された核のテーブル	108
インデックス—よく実行するクエリから先に決める	109
7.4 3回目のループ—マイグレーションシステム	110
7.5 スキーマのテスト—AIで品質検証する	112
7.6 一発で完成したコマンド—db_query_json	112
7.7 Mandaiのための DuckDB クエリパターン	113
ソフト削除とJOINの落とし穴	114
7.8 TypeScript型レイヤー—AIが確かに役立った場所	114
7.9 この章のループまとめ	116
DB作業のパターン分類	116

7.10 Rust側のコマンドレジストリ	116
7.11 次の章	117
この章で必ず覚えておくこと	117
第8章. AI API統合—AIを使ってAIコードを書く	119
この章の流れ	120
学習目標	121
一読目に見るべきこと	121
8.1 要件を先に文章でまとめる	122
8.2 複数プロバイダの抽象化—Rustのtrait	122
8.3 イベント構造	124
ストリームのキャンセル	125
8.4 SSEパースで詰まったところ	126
8.5 エラー処理—プロバイダごとに違う失敗をするとき	127
8.6 リクエスト制限とコスト管理	128
8.7 レスポンスの正規化—フロントエンドに統一された形式を渡す	128
8.8 この章のループまとめ	129
8.9 AIがコードを書くときの特徴	129
8.10 この章で必ず覚えておくこと	129
第9章 Mandai 開発で犯した失敗 — AI が間違えた箇所	131
本章の流れ	132
学習目標	133
9.1 ポモドーロタイマー — 状態が二箇所に存在する	133
本当の問題	133
最初の試行 (失敗)	134
落とし穴1 — CHECK 制約違反	135
落とし穴2 — 過剰なクリーンアップと時間ベースの復旧	135
落とし穴3 — バックグラウンドのスロットリングとタスクの紐付け	135
修正後のプロンプトテンプレート	136
核心的な教訓	137
9.2 Mandal-Art Cell ID — 一般的なパターンがドメインルールを上書きする瞬間	137
どこでズレたか	138

なぜこんなことが起きるのか	138
修正の方法	138
9.3 生産性スコアの計算 — エッジケースを明示すると AI が改善する	139
ループのたびに現れたエッジケース	140
この経験から得たもの	140
9.4 バージョンの混同 — Tauri 1.x と 2.x の API を混ぜてしまうとき	141
典型例: ウィンドウ管理 API	141
バージョン明示プロンプトで解決	142
v1 -> v2 で混ざりやすいパターン	142
9.5 防ぐ方法: プロンプト・レビュー・テストの3点セット	143
偽の生産性	143
道具1 — プロンプトチェックリスト	144
道具2 — レビュー用プロンプト	145
道具3 — テストプロトコル	146
3つの道具の関係	147
9.6 まとめと次章	147
次章	148
第10章. readbooks.ai —AI出力品質を設計するプロンプトエンジニアリング	149
この章の流れ	150
学習目標	151
10.1 背景	151
クラウドとオンデバイスを同じprovider境界へ置く	151
on-device・on-demand・offlineを区別する	151
小型モデルは用途別に検証する	153
10.2 最初のプロンプト—すべてを取りこぼした初稿	154
なぜAIはこう動くのか	155
実際の失敗例—ブロックに5つの誤り	155
1回目の修正—保持すべき範囲を先に固定する	156
2回目の修正—「記号」と「内容」を分離する	156
10.3 言語別の分岐—「36言語のための一つのプロンプト」という幻想	157
解決—基本プロンプト + 言語別プロファイル	158
実際のプロファイルはどんな形か	158

言語プロファイルの構築: AIをリサーチアシスタントとして使う	159
10.4 AIと一緒にプロンプトを設計する—メタループ	159
メタプロンプトとは	160
同じAIをレビュアーとして再利用する	160
デュアルエージェント—翻訳者とレビュアーの分離	161
失敗したメタプロンプトたち	161
プロンプトのバージョン管理	161
10.5 翻訳品質基準—AIに伝える方法	162
各基準がどの失敗から生まれたか	162
ドメイン別の追加基準	163
検証パイプライン—自動検査 + 人間レビュー10%	163
10.6 この章のループの整理	164
繰り返し現れた3つのパターン	165
10.7 次の章	165
まとめ	165
第11章 翻訳の検証—AIの出力をどう信頼するか	167
この章の流れ	168
学習目標	168
11.1 問題—「それらしいのに間違っている」翻訳	169
11.2 逆翻訳—手作業で一度やってみる	169
11.3 検証 command を作る—いちばん単純な骨格	170
11.4 単一スコアの限界—BLEU と cosine 類似度	171
11.5 三つの軸に分ける	171
結果型をフロントエンドと揃える	172
でっ上げ検知を追加する	172
三段階バッジに整理	173
11.6 運用しながら磨いていく話	173
11.7 ユーザーフィードバックのループ	174
機能	174
実データでプロンプトを更新	174
フィードバックを単純な三段階に並べる	175
11.8 「AIの出力を別のAIで検証する」の図	175

11.9 自分のアプリに応用するコツ	176
コツ1. 単一スコアは禁止。軸を分けよ。	176
コツ2. 生成モデルと検証モデルを別にする。	176
コツ3. しきい値は三段階で。	176
コツ4. ユーザーフィードバックをプロンプトに反映する。	176
11.10 この章のまとめ	177
11.11 Part II と Part III を振り返って	177
11.12 次章予告	178

まえがき

本書は、説明、計算例、状態の流れを通じて、設計と検証の原理を理解できるように構成している。コードの一部は著者のアプリの構成を示す抜粋であり、製品全体のソースではない。独立して実行できる例には、実行条件と期待する結果を示す。

教育用資料と提供範囲

任意の補助資料の名前は `tauri2-app-examples.zip` だ。対象は、`tauri/ch05-local-notes/` の最小Tauriアプリと、`flet/ear-loop/` のFlet演習である。インストール手順、依存関係のロックファイル、検証範囲は各READMEで確認する。商用アプリ全体や自動売買の運用リポジトリへのアクセス権は含まれない。

添付ファイルやダウンロードリンクの有無は、販売者の配布ページで確認する。本文に資料名があるだけでは、購入内容に添付済みとは限らない。リンクがない場合は、書籍紹介ページからたどれる著者の公開プロフィールで問い合わせ先を確認する。PDF中の説明用コードと、別途配布される実行用の教材は区別しよう。

この章の流れ

このまえがきでは、本文に入る前に次の点を整理しておく。

- ・ 著者がAIとともにアプリを開発し、検証する方法
- ・ 本書が扱うことと扱わないこと
- ・ 本書を読むために知っておくべきこと
- ・ 本書で作る6つのプロジェクトと使う技術
- ・ どんな順番で読むのがよいか

最初から全部を理解する必要はない。本文を読みながら迷ったときに、ここに戻ってくればいい。

学習の目標

このまえがきを読み終えたら、次の3点だけ押さえておけばよい。

- ・ 本書はAIツールの使い方そのものより、**動作を検証しながらアプリを完成させるやり方**を扱う、ということ
- ・ Tauri 2、SvelteKit、Rust、DuckDBといった技術をすべて最初から覚える必要はない、ということ
- ・ 本文をどんな順番で読み、詰まったときにどこへ戻ればよいか

本書を書きながら

2026年、AIツールは大きく変わった。だが私の作業のやり方が変わったのは、単にツールが新しくなったからではない。**自分が引き受ける仕事の範囲そのもの**が変わったからだ。

1月の私は、これまでと同じくコードを自分で書く開発者だった。チケットを開き、エディタを開き、タイピングする。機能ひとつに2日、複雑なら3日。それが慣れた速度だった。

本書では、デスクトップアプリ4つとターミナルツール2つ、計6つのプロジェクトを例にする。生産性アプリや、トレーディングシステムを確認するTUI（ターミナル画面で動作するアプリ）などを扱う。

腕が急に上がったわけではない。**自分がやる仕事とAIがやる仕事の役割**が、根本的に変わったのだ。

検証の基準を移す

年の初めは、みんなと同じくAIをもっと速い自動補完のように使っていた。やりたいことを説明すればAIがコードを書き、私はそれを受け取って作業を続ける。

AIは速かったが、私は遅かった。AIが3分で作ったコードを、私は1時間半かけてレビューしていた。丁寧なレビューは良い習慣だと思っていたが、繰り返すほどに奇妙なボトルネックができていた。**読む必要すらないコードを読みながら、私が全体の作業を止めていたのだ。**

そのとき自分に問いかけた。

自分がやるべきことは、コードが正しいかを確認することなのか。それとも、機能が正しく動くかを確認することなのか。

前者はコードを読んで確かめる。後者は実際に動かして確かめる。本当に大事なのは後者なのに、私は前者に時間を使っていた。

それからは作業のやり方をこう変えた。

- ・ AIに対して正確なリクエストを書く。
- ・ 機能を自分で実行して確認する。
- ・ よく起きるエラーをチェックリストで点検する。
- ・ 通ればコミットする。失敗すれば、その内容をそのまま次のリクエストに入れて繰り返す。

これはコードを読まないという意味ではない。**検証の基準を、コードの見た目から実際の動作へと移す**という意味だ。

その結果が6つのプロジェクトと本書である。時間が増えたわけではない。重要でない場所に時間を使うのをやめただけだ。

AIがコードを書き、判断は読者がする。無計画なAIコーディングは早く始められるが、何が間違っているか分からなければAIに説明することもできず、直すこともできない。

なぜ詰まるのか、そしてどうやって検証の主導権を人が握るのかは、第1章で詳しく扱う。

本書が扱うこと

本書全体を1つのサイクルが貫いている。

Request(リクエスト) -> Build(構築) -> Verify(検証)

アプリを作るとき、この3ステップを繰り返す。それぞれが具体的に何を意味し、どこでよく間違えるかは第2章で説明する。このサイクルがTauri 2アプリで実際にどう回るかを、6つのプロジェクトで見せていく。

本書が扱わないこと

本書が扱わないことを先に知っておくと、別の資料を併読するときに迷わなくて済む。

- ・ **Rust文法のチュートリアル**。Rustの基礎文法は説明しない。
- ・ **Svelte 5文法のチュートリアル**。Svelte 5の新しい文法はコード中で使うが、ゼロから教えることはしない。
- ・ **公式Tauri 2ドキュメントの焼き直し**。本書は公式ドキュメントの代替や繰り返しではない。
- ・ **プロンプトエンジニアリングの一般論**。文章執筆や画像生成など、コーディング以外の領域は扱わない。
- ・ **AIモデルの内部構造**。ClaudeとGPTが内部でテキストをどう処理しているかは比較しない。

本書の技術スタックがなじみにくい理由

本書が扱う技術が見慣れないものに感じるのは、自然なことだ。

Tauri 2はRustとJavaScript/TypeScriptと一緒に使うフレームワークだ。2つの言語は違う環境で動くので、最初は両者がどう繋がるのか掴みにくい。最初からすべての構造を理解しなくていい。

本書は各技術を個別チュートリアルのように一つずつ説明しない。実際のプロジェクトを進めて問題が起きたらAIに尋ね、AIが提案した解決策を自分で実行して確かめる。学ぶべきは、すべての技術の細部の文法ではない。**知らない技術を前にしても、問題を分解して確認するやり方だ。**

たとえばMandaiアプリ(マンガラート式のタスク・目標管理アプリ)を作りながら、DuckDB(軽量な分析用組み込みデータベース)との連携を学ぶ。トレーディングダッシュボードでは、WebSocket(サーバーと途切れずに双方向でデータをやり取りする通信方式)でリアルタイムデータを受け取る。詰まる部分はAIに任せればよい。最初から全部を理解して始める必要はない。

実装の多くはAIが担当する。ある方法でうまくいかなければ、別の方法を探すよう指示できる。ただし、すべての判断をAIに委ねるわけではない。AIは選択肢を作り、実装を素早く試す。**最終的な判断は人が下す。**

AIがRustコードを代わりに書いてくれるが、生成されたコードから問題を見つけるには、最低限の文法を読めるくらいの力が必要だ。自分で書けるようになる必要はない。

6つのプロジェクトを一目で

本書で扱う6つのプロジェクトは、説明のために作った例題ではない。著者が実際に使っているアプリだ。

- ・ **Mandai** —AI コーチを内蔵した生産性アプリ
- ・ **readbooks.ai** —2025年にNext.jsで作ったPDF翻訳アプリ。本書ではTauri 2で作り直す
- ・ **Trading Dashboard** —リアルタイムのトレーディングダッシュボード
- ・ **RL & Backtest Lab** —強化学習の訓練制御
- ・ **TUI Trading Monitor** —ターミナルで動くトレーディングモニター
- ・ **TUI Log Viewer** —ターミナル上のログビューア

それぞれが具体的に何をするのか、本書全体でどんな役割を担うのかは1.8節で詳しく扱う。

本書の読み方

Part I(第1~5章) は順番に読む。本書全体の前提、ループの定義、AIコーディングツールのワークフロー、ツール設定、Tauri 2の基礎を扱う。

Part IIに進む前に、**1.8節(6つの事例)** をざっと目を通しておく。そうすると、後の各章でAIに何を、なぜリクエストするのか、文脈がぐっと掴みやすくなる。

Part II_{VI}(第6~17章) は、各パートが1つのアプリを中心に独立して構成されている。興味のあるパートだけ選んで読んでもよい。

- ・ Mandaiアプリが気になるなら -> Part II
- ・ AI翻訳と出力品質が気になるなら -> Part III
- ・ トレーディングダッシュボードが気になるなら -> Part IV
- ・ ドメイン知識をAIに渡す方法が気になるなら -> Part V
- ・ ターミナルアプリが気になるなら -> Part VI

各パートは「このアプリで学ぶループのパターン」の紹介から始まる。Part II以降のアプリ事例は互いに繋がる部分もあるが、必ず順番に全部読まないと理解できない構造にはなっていない。興味があるアプリから読めるよう、各パートの最初の章で必要な文脈を改めて整理する。

Part VII(第18章) は、この方法を自分のアプリへ適用するときに参照する章だ。初回に一度、実際に自分のアプリを始めるときにもう一度読む。

全体の構成

本書は7つのパートで構成される。各パートで実際のアプリを1つずつ作りながら、AIと一緒に開発する方法を順に身につける。**太字で示した名前は**、自分で作ってみるプロジェクトだ。

Part	章	テーマ
I	1 –5	AIを使う理由、フィードバックループ、Copilotのワークフロー、開発ツール、Tauri 2の基礎
II	6 –9	Mandai —設計、DB、AI API、AIのミスへの対応
III	10 –11	readbooks.ai —プロンプティング、出力の検証
IV	12 –14	Trading Dashboard —規模の拡張、リアルタイム処理、デプロイ
V	15	RL & Backtest Lab —AIに専門ドメイン知識を注入する
VI	16 –17	TUI Apps —ターミナルUIアプリを素早く仕上げる
VII	18	自分のアプリへの適用

前提知識

本書を読み通すには、次の背景があるとよい。

- ・ **RustとTypeScriptのコードを読めること**。自分で書ける必要はない。AIが作ったコードを見て「これは何をしているのか」をざっくり把握できれば十分だ。
- ・ **Gitとターミナルに慣れていること**。ブランチを切り、コミットを分ける動作が身についていれば、本書の作業の流れ(コードを書く -> AIにレビューさせる -> 修正を繰り返す)をそのまま適用できる。
- ・ **AIコーディングツールを一度でも使ったことがあること**。Claude Code、Cursor、GitHub Copilot、ChatGPTのいずれかを、実際の作業で使ってみた経験があるとよい。まだなければ、おもちゃのプロジェクトで1週間ほどAIと一緒に作業してから戻ってこよう。

逆に、次は**なくてもよい**。Tauri 1 の経験、SvelteKit の経験、Rust の経験—どれも無くてもかまわない。必要な内容は各章の中で説明する。

技術スタックを一目で

本書で使う中心的な技術だ。それぞれを深く知っている必要はない。**いまは何の役割をするものなのかだけ掴んで進めばよい**。

技術	役割	使う場所
Tauri 2	Web 技術 (HTML/CSS/JS) でデスクトップアプリを作るフレームワーク。インストーラが小さい。	2~14 章
SvelteKit 2 + Svelte 5	本書の画面 (フロントエンド) フレームワーク。	2~14 章
Rust	Tauri のバックエンド言語。自分で書くことはほぼない。読める程度で十分。	2~17 章
DuckDB	サーバー不要、ファイル 1 つで動く分析用データベース。	7~13 章
ratatui	ターミナル画面に UI を描くための Rust ライブラリ。	16~17 章
Python	強化学習の訓練スクリプト。Rust アプリとは別プロセスで動かす。	15 章のみ
shadcn-svelte	Tailwind ベースの Svelte UI コンポーネント集。	6~14 章

技術	役割	使う場所
Claude Code	ターミナルで動かすAIコーディングツール。本書で最もよく使う。	全章
Cursor	AI自動補完を備えたコードエディタ。ファイル1つを素早く編集するとき に使う。	全章

各技術の細かな動作や違い(たとえばTauriとElectronの比較、Svelte 5のruneの文法など)は、本文で初めて使うときに改めて説明する。

開発環境

本書のすべてのコードとスクリーンショットは、次の環境で作成した。同じバージョンを強制する必要はないが、詰まったときに見比べる基準点になる。

項目	バージョン
OS	macOS 15.7.5 (Sequoia)
Rust	1.95.0
Node.js	24.15.0 LTS
pnpm	10.33.2
Tauri	2.11.0
SvelteKit	2.59.0 + Svelte 5.55.0
DuckDB	1.5.2
Python	3.14.4 (強化学習の実習章のみ)
Claude Code	2026年5月時点の最新版
Cursor	3.0

項目	バージョン
VSCode	最新 (1.99+)

注意: pnpm 9.xは2026年4月30日でサポートが終了した。pnpm 10を使うこと。
Node.js 22 LTSでも動くが、本書はNode.js 24 LTSを前提に書かれている。

WindowsとLinuxでも、ほとんどの例題はそのまま動く。プラットフォームで異なる部分があれば、その章で個別に明記する。

著者からのメモ

本書には、私がAIと一緒に開発した経験を、起きた順に並べている。

プロンプトを書き直して再試行を繰り返した末に、結局自分でコードを直してしまった記憶もある。それをやってはいけない。AIが、もっともらしく見えて実は間違っているコードを返してきたのに、実行もせずそのままコミットした記憶もある。そうするとエラーが雪だるま式に積み重なり、コードを丸ごと捨てる羽目になる。本書の学習スタイルは、そんなふうに自分が実際にたどった思考の流れに沿っている。

会社勤めをしながら、生後150日の子を育てつつアプリを6つ作った作業記録でもある。昼休みにカフェでプロンプトを書き、退勤の電車でコードを確認し、夜中の授乳が終わった後にコミットを送った。

本書に出てくる場面のいくつかは、流れを整えるために少し再構成してある。教訓とコードは本物だ。

時間があつたから書いた本ではない。**時間がなくてもAIエージェントがあれば足りる、**と伝えたくて作った記録である。

この章で必ず覚えておくこと

まえがきは長かったが、本文に進む前に必ず覚えておくべきことは4つだけだ。

- ・ **核となるのはRequest -> Build -> Verifyのサイクル**だ。コードを最初から最後まで読むのではなく、実際に実行して動作を確認する。
- ・ **6つの実際のアプリ**でこのサイクルを練習する。すべて著者が毎日使っているアプリだ。
- ・ **Part Iは順番に**、Part II以降は興味のあるアプリから選んで読んでもよい。
- ・ **すべての技術を前もって学んでおく必要はない**。詰まった瞬間にAIに尋ね、自分で実行して確かめる。それ自体が学習の過程になる。

それでは第1章で、「AIがコードを書くのに、詰まるのはなぜ人なのか」から見ていこう。

朴 承煥 (パク・スンファン) 2026年

第1章. スピードよりも大切なのは検証だ—主導権を握るべき理由

Part I —基礎(1～5章) AIと一緒に開発する方法を扱う。なぜAIを使うのか(1章)、どのループで動くのか(2章)、どの道具をどう使うのか(3～4章)、Tauri 2の核心パターン(5章)。この五つの章が本書全体の土台になる。最初の通読では、道具の名前をすべて覚えるよりも「依頼して、作って、検証して、また依頼する」というループを身につけることに集中する。

最初はAIに「作ってくれ」と頼めば終わりだと思っていた。出てきたコードをそのまま貼り付け、画面が表示されたら次の機能に移った。数日間は本当に速く感じた。自分でタイピングしていた時間が消え、数時間かかっていた下書きが数分で出てくる。だが、上がった速度と同じだけ誤った判断も急速に積み重なっていった。

たとえば、こんな一文の依頼だけでも動くコードが出てきた。

トレーディングダッシュボードを作って。Python
FastAPIバックエンドにWebSocketで接続して、
リアルタイムチャート。Tauri 2で作って。

画面もそれらしく見えた。問題はその後だった。接続が切れたのに再接続されない。状態が絡まったのに、どこで間違ったのか追えない。AIに直してくれと頼むと、また それらしいコードが出てくるが、同じ種類の問題が形を変えて繰り返された。

気づいたことは単純だった。自分で確認していないコードは、自分が制御しているコードではない。AIが書いたか、人が書いたかは関係ない。どんな動作を期待していたのか、どんな条件で失敗するのを確認せずに統合したなら、そのコードはシステムに紛れ込んだブラックボックスも同然だった。速く生成されるからといって主導権を握っているわけではなかった。

そこで質問が変わった。「これを作ってくれ」よりも「この動作が本当に正しいかをどう確認するか」を先に問うようになった。結果を実行し、失敗する条件を絞り込み、その証拠を次の依頼に込める。このループを繰り返した。この章はその転換についての話だ。AIがコードを速く書くという事実よりも大切なことがある。その速さの中で、開発者がどうやって主導権を失わないかだ。

この章の流れ

この章では、AIコーディングにおいてスピードよりも検証がなぜ重要なのか、そして開発者の役割がどう変わったのかを順に見ていく。

- ・ 計画なしのAIコーディングはなぜ長く持たないのか
- ・ なぜ今なのか
- ・ 開発、それ以前は: 自分でコードを書いていた
- ・ 今は: AIが書いて、自分が決める
- ・ 「読むだけでよい」という言葉が半分しか正しくない理由
- ・ 役割の転換: コードレビューから要件整理へ
- ・ ワンシーン—韓国語PDFパーサーのエピソード
- ・ 本書に繰り返し登場する六つの事例
- ・ 本書を読み終えたあとに残るもの

学習目標

- ・ AIが単なる自動補完ではない理由を理解する。
 - ・ 計画なしでAIを使ってコーディングするとどこで詰まるのかを知る。
 - ・ 「AIのコードは読むだけでよい」という言葉がどこまで正しく、どこから誤っているのかを見分ける。
 - ・ 開発者の役割がタイピングから検証と判断へと変わった理由を理解する。
 - ・ 本書で扱う六つの事例をあらかじめ眺めておく。
-

1.1 計画なしのAIコーディングはなぜ長く持たないのか

計画なしにAIでコーディングすると、最初はうまく回っているように見える。画面が表示され、ボタンが反応し、APIのレスポンスがコンソールに出る。だから検証を後回しにしやすい。「とりあえず次の機能までつないでから整理しよう」という考えが自然について回る。

問題は、そうやって積み上がったコードだ。あるとき何かが壊れたとき、どこが原因かすぐには見えない。たとえばWebSocket(ウェブソケット、双方向のリアルタイム通信)の再接続が失敗したとしよう。フロントエンドの状態の問題なのか、Rust側の問題なのか、Pythonバックエンドの問題なのか、一目では分からない。私もダッシュボードが止まったとき、しばらく見当違いのところを疑っていた。本当の原因は単純だった。失敗条件を確認しないままAIの結果をそのままコミットする、その作業のしかた自体が問題だったのだ。

それ以来、やり方を変えた。AIがコードを書いたとしても、次の段階に進むかを決めるのは開発者の役目だ。その判断は勘ではなく、検証結果に基づくべきだ。本書で繰り返し登場する **Request, Build, Verify ループ**(依頼 → 生成 → 検証)は、その主導権を保つための最低限の仕組みだ。スピードはAIに任せつつ、判断は開発者が握る構造である。

1.2 なぜ今なのか

AIは以前からあった。自動補完もあった。それならなぜ2026年なのか。三つのことが同じタイミングで噛み合った。

第一に、モデルが変わった。 2024年末から、Claudeのようなモデルは複数のファイルを同時に読み、理解する能力を持ち始めた。2026年には、ファイルを読み、コードを直し、テストを実行する一連の作業を、一つの会話の流れの中でつながられるようになった。単なる自動補完ではなく、小さな開発単位をまるごと任せられる道具になったのだ。

第二に、Tauri 2が正式リリース(GA)された。 インストールファイルが小さく、メモリ使用量も低い。Electronベースのアプリが100MBを軽く超えるのと比べれば差は大きい。技術の細かい話は5章で扱うが、ここで注目したいのはTauriの構造だ。画面はWeb技術で作り、ファイルアクセスやシステム権限が必要な処理はRust command(ラスト・コマンド)で扱う。役割がこう分かれていれば、AIに対しても「どちら側を直してほしい」と明確に指示できる。

第三に、Svelte 5 runesが登場した。 runes(ルーン)は `$state`、`$derived` のように `$` で始まるSvelte 5の新しいリアクティブ構文だ。詳しくは4章で扱う。今はひとつだけ覚えておけばいい。Reactで複数のフックを組み合わせる必要があった状態管理が、Svelte 5ではもっと短く単純なパターンに収まるという点だ。パターンが単純なほど、AIが取り違える余地も減る。フロントエンドのコード品質が上がる理由はそこにある。

1.3 開発、それ以前は: 自分でコードを書いていた

AIがなかった時代、開発のほとんどは指でやっていた。JIRAのチケットを読んでエディタを開くと、ブラウザにはStack Overflowのタブがいくつも開いていた。当たり前のコードも自分で打たねばならず、コンポーネント一つに半日かかることも珍しくなかった。あの頃の最高の生産性ツールは自動補完だった。最初の数文字を入れるとIDEが残りを埋めてくれる、それだけのことだった。

コードの90%は私の手を通っていた。関数名、変数名、繰り返される条件文、import文に至るまで全部入力していた。この方式には良いところもあった。手で書くと、コードの構造を長く見つめることになる。だが欠点もはっきりしていた。タイピングに時間

がかかりすぎて、設計を十分考える前に実装を進めてしまう。「とりあえず動くようにしてあとでリファクタリングしよう」という言葉は計画ではなく、時間がないという告白だった。そして、たいていの場合、その「あと」は来なかった。

1.4 今は: AIが書いて、自分が決める

2026年に変わったのは、AIが私の仕事を肩代わりするようになったことではない。AIがタイピングを担い、自分が方向を決めるようになった。手は暇になったが、何を依頼するか、どこで止めて検証するかは、絶えず判断しないといけない。判断の頻度はむしろ増えた。

実際の流れを一回の作業で見てみよう。朝、ノートにアイデアを書いた。「Mandal-Artの目標ツリー + Pomodoro + AIコーチング -> ローカルDuckDBに保存 -> 全部Tauri 2で包む」。昼食のあとClaude Codeのターミナルを開き、最初のプロンプトを投げた。

このプロンプトは「何を作るか」と「どんな制約に従うか」を短く書いたものだ。

Tauri 2 + SvelteKit 2 + Svelte

5でMandal-Artの9x9グリッドコンポーネントを作って。

セルをクリックすると編集モーダルが開くようにして。中央のセルが変わったら

AIが周りの空きセルの候補を提案するようにして。

Svelte 5 runesのみ使用。ファイル別にコードを分けて。

3分でコンポーネント、ストア、AI提案用のTauriコマンドの下書きが出てきた。出てきたものを動かしてみると、二つほど気に入らなかった。第一に、新しい構文を使えと言ったのに古い構文が混ざっていた。第二に、AI呼び出しがフロントエンドから直接行われていた—これではAPIキーが露出する。

観察した問題をそのまま次のプロンプトに入れた。

二つ修正してほしい。

1. `$state`, `$derived`, `$effect`だけを使うこと。

``let``で宣言するリアクティブ変数なし。

2. AI呼び出しをTauriコマンドに移すこと。

APIキーがフロントエンドに露出してはいけない。

さらに3分の修正で、両方とも解決した。コミット。以前なら6時間かかっていた作業が20分以内で終わった。

ただし、その20分は「AIが勝手に全部やってくれた」という意味ではない。望むものを言葉で表現し、結果を動かしてみて、間違っているところを見つけてもう一度依頼し

た。タイピング量は減ったが、何を望むかを整理し結果を判断するのは依然として自分の役目だった。

同じ作業でAIが最初の試みで当てたこと、外したこと、私が直させたことを整理するとこうなる。

カテゴリ	例
当てた	9×9グリッドのレイアウト、Tailwindクラスの組み合わせ、キーボードナビゲーションの骨組み
外した	古いSvelte構文が混ざる、フロントエンドから直接AI呼び出し、型処理が雑
直させた	新しいrunes構文に統一、AI呼び出しをTauriコマンドへ移動、エラー型の整理

この3行の表が機能ごとに繰り返される。私の一日は実のところ、この表を埋めることだ。

要点は、AIが最初から完璧だということではない。短く依頼し、短く修正し、短く検証するループを高速に繰り返せるという点だ。以前より速くなった理由もそこにある。一発で当てる能力よりも、外したときにすぐ範囲を絞ってもう一度試す能力のほうが大事だった。

1.5 「読むだけでよい」という言葉が半分しか正しくない理由

今振り返ると、**バグを探すためにコードを一行ずつ読む必要はなかった**。でも最初はそれが分からなかった。

この言葉は半分は正しく、半分は誤っている。

正しい部分から見よう。もうRust構文をまるごと暗記する必要はない。AIがコードを書く。自分は読めばいい。「こここの処理は正しいか？」と問えば十分だ。

しかしそこで止まれば誤る。「読める」の基準が以前よりずっと高くなっているからだ。AIが渡したコードに問題があるかを判断できなければならない。今すぐRustを知らなくてもいい。5章までついてくれば「こんなコードが見えたら危ない」というシグナルくらいは目に入るようになる。その程度で十分だ。

必要な力は二つ。危険信号を見抜く力、そして「この結果は自分が望むものか」を判断する力だ。

注意: これはMandaiプロジェクトを始めるときに私が抱いていた考えだった。最後までやってみると、コードを読むことがボトルネックではなかった。コンパイラと動作テストだけで十分にバグを捕まえられた。Rustのチェックツール(*cargo clippy*)は危ないパターンを警告してくれ、動作テストは隠れた誤りをあぶり出す。どんなパターンがなぜ危険なのかを知っていることは今も大事だ。だがバグを探すためにコードを一行ずつ読む必要はなかった。2章では動作テストがコードレビューを置き換えるループを見せる。

では、検証はどうやるのか。コードを読むのではなく、**実行結果を証拠として次の依頼に入れる**。韓国語のPDFが文字化けするなら、化けた数行をそのままコピーして次のプロンプトに貼ればいい。1.7節で、この方法が実際にどう動くかを見ていく。

1.6 役割の転換: コードレビューから要件整理へ

2020年、私が一日でいちばん多くやっていた作業は**コードを書くこと**だった。2026年にいちばん多くやっているのは**プロンプトを書くこと**だ。

プロンプトを書くというのは、要件を書くということだ。会議のあとに残す短いメモとは違う。AIに渡すプロンプトは、はるかに**正確でなければいけない**。人は「うまくやって」という言葉のある程度自分で解釈する。AIはそうではない。理解したように見えても、しばらくしてまったく違うものを出してくる。

以前は作り終えてから検証していた。今は作り始める前に曖昧さを潰しておかなければならない。要件と自分の判断を先に分けておかないと、AIは空欄を自分のやり方で埋める。

そこで、毎日次の項目を頭の中で整理するようになった。

- ・ **何を:** 関数なのかコンポーネントなのかアプリ全体なのか。

- ・ **なぜ**: もっと大きな機能の中のどこに入るのか。
- ・ **制約**: 言語、ライブラリ、性能目標、ファイル構造。
- ・ **出力の形**: コードブロック、ファイル一覧、変更部分のみ、説明の深さ。

慣れれば、この四項目を一文に収められる。2章ではループの一部としてこれを扱う。

かつての**コードレビュー**は、他人が書いたコードを事後に読む作業だった。今の**要件整理**は、AIが何かを作る 前に 頭の中を整理する作業だ。間違っただけで直すのではなく、最初から間違える可能性を減らす。順序が逆になった—それが最大の変化であり、AIと働くと生産性が上がる理由だ。

1.7 ワンシーン—readbooks.ai のPDFパーサーのエピソード

readbooks.ai を Tauri 2 で作り直す作業の途中で、PDF からテキストを取り出す機能が必要になった。最初のプロンプトは短かった。

PDF からテキストを抽出する Rust ベースの Tauri コマンド。

Claude は pdf-extract ライブラリを使った 20 行ほどのコードを出してきた。動かしてみると、英語 PDF はうまくいった。韓国語 PDF は**文字化けした箱の列**しか出てこなかった。

以前なら検索欄に「rust pdf korean encoding」と打ち込んでいただろう。代わりに、観察した結果をそのまま持って、もう一度尋ねた。

韓国語 PDF が文字化けする。サンプル入力の 1 ページ目はこれ:
<化けたテキスト 30 行>

考えられる原因を 3 つ挙げて、それぞれの簡単な修正方法を教えて、そのうえで現実的なものを選んで、修正したコマンドを出して。

Claude は三つの原因候補を並べ、その中で可能性が最も高いものを選び、別のライブラリを使うか外部ツールを呼び出す方法を提案してきた。診断は的確だった。

ここで大事なことは、Claude が最初に外したという事実ではない。**自分が観察した結果を次のプロンプトに入れ直して依頼を具体化した**という点だ。以前なら「これをどう直すか」を検索窓に打ち込んでいた。今は**自分が見たものを AI に正確に見せて、その上で次の仮説を立てる**。

本書で繰り返し出てくるパターンはこれだ。短い最初の依頼、結果を証拠として再提示すること、候補となる仮説の列挙、現実的な選択。AIに答えを一方的に期待するやり方ではない。この流れが、繰り返し可能な作業スタイルの土台になる。

1.8 本書で繰り返し登場する六つの事例

序文で紹介した六つのプロジェクト—Mandai、readbooks.ai、Trading Dashboard、RL & Backtest Lab、TUI Trading Monitor、TUI Log Viewer—が本書全体を通して繰り返し登場する。ここではこの章のテーマとつながる部分だけ先に触れる。

六つのプロジェクトは、それぞれ別の問題から始まった。Mandaiは作業をキャンセルするたびにUIが固まった。readbooks.aiは先に見たように、韓国語PDFを処理すると文字が崩れた。トレーディングダッシュボードはリアルタイム接続が切れたあと再接続できなかった。RL Labはプロセスが止まってもメモリから片付けられず残り続けた。

共通点はひとつ。問題が目に見えるときだけAIは役に立った。何が間違っているのか分からなければ、AIに説明することすらできない。

本書の事例は成功談の寄せ集めではない。失敗をどう絞り込み、それをどう依頼に変え、どこで人が直接判断すべきかを見せるための素材だ。

1.9 本書を読み終えたあとに残るもの

本書を終えてもTauri 2のAPIをすべて暗記することにはならない。Rustの専門家になるわけでもない。それは目標ではない。

持ち帰るものは、**同じ方法で自分のアプリを作る能力**だ。具体的には五つある。

1. **依頼を正確に書く筋力**。頭の中の「こんな感じ」をAIが実行できるプロンプトに変える力だ。
2. **AI出力を判断する目**。正しいコードと間違ったコードを見分け、正しいけれど危ないコードも捕まえる。Tauri 2で初心者がよく踏む落とし穴を先に知っておく。
3. **開発ループを途切れさせないリズム**。最初の試みが完璧でないことを受け入れ、短いサイクルで終わらせる。

4. **六つの実プロジェクト参照**。AIが書いた部分、自分が直した部分が両方残っている。似たものを作るときにほぼそのまま使えるパターン集だ。
5. **インターフェイスを選ぶ判断**。ユーザー体験が大事ならGUI、スピードが大事ならTUI(ターミナルベースUI)だ。どちらを選ぶかで、その後のすべてのプロンプトの向きが変わる。

この五つは特定の技術スタックに縛られない。Tauriでなくても、Rustでなくても、残るのは「依頼して、判断して、検証する」やり方そのものだ。

この章で必ず覚えておくこと

- ・ AIコーディングの核心は「速く作る」ではなく「自分で検証する」だ。検証していないコードは自分のものではない。
- ・ 無計画なAIコーディングは最初こそ速いが、問題が起きた瞬間、どこから疑えばよいか分からなくなる。
- ・ 2026年にはモデル、Tauri 2、Svelte 5が同時に安定し、AIを実開発に組み込める条件がそろった。
- ・ 開発者の仕事はタイピングから、**依頼する -> 結果を確認する -> もう一度依頼する**へと移った。
- ・ 「AIのコードは読むだけでよい」は半分しか正しくない。構文を暗記する必要は減ったが、実行結果を証拠として次の依頼に戻す力が新たに必要になった。
- ・ 本書は六つの実例をたどりながら、その作業のしかたが実際にどう動くのかを見せていく。

2章では、このループを実際にどう回すのかを段階ごとに解きほぐす—各段階の開始条件、終わらせる条件、そして繰り返しがプロジェクトを揺さぶる悪い癖まで。

第 2 章. Request, Build, Verify ループの形

本書のすべての章は、同じループの上で動く。扱うコードは章ごとに違う。DuckDBのスキーマを作ることもあるし、WebSocketの再接続を直すこともあるし、PythonプロセスをTauriアプリの中で実行することもある。だが、作業のやり方は変わらない。要求し、AIが作った結果を受け取り、実行して確認し、失敗した部分だけを再び要求する。

第1章で見た核心的な難しさは、**AIがもっともらしい結果を、人が検証できる速度より速く作り出す**という点だった。この差をそのままにしておくと、検証されていないコードの上に次の機能が積み上がり、3つ目の機能を載せた瞬間に、どこが原因なのかを追跡しづらくなる。だからこの章では、ループを意図的に細かく定義する。各ステップに、開始条件と終わらせる基準を置く。後のプロジェクト章は、このループを実例に適用し

た記録である。ループを先に理解しておかないと、後の章でなぜ特定の要求をし、なぜ特定の地点で止まって検証するのが見えてこない。

この章の流れ

- ・ 学習目標
- ・ この章で使う技術
- ・ ひとつのループ、サイズが変わるだけ
- ・ ステップ1 —要求
- ・ ステップ2 —実装
- ・ ステップ3 —検証
- ・ ステップ4 —反復
- ・ 「完璧なプロンプトひとつでアプリ完成」が幻想である理由
- ・ 骨組み優先戦略(トップダウンAI開発)
- ・ 実例—MandaiのDBスキーマループ
- ・ 基本プロンプトテンプレート
- ・ ループを回すときの注意点

学習目標

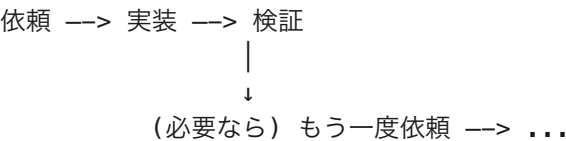
- ・ 本書全体で使う中心ループ **要求 -> 実装 -> 検証 -> 反復** を把握する。
- ・ 各ステップで守るべき原則と、よく足を引っ張るアンチパターンを押さえる。
- ・ 「完璧なプロンプトひとつでアプリ完成」がなぜ幻想なのかを見る。
- ・ **骨組み優先戦略**を学ぶ。全体構造から固め、細部機能へ下りていき、バグを直し、機能を検証してから磨く流れである。
- ・ Mandaiの初期DBスキーマで、ループを最初から最後まで実際に回してみる。
- ・ **何を / なぜ / 制約 / 形式** プロンプトテンプレートを自分で書いてみる。

この章で使う技術

技術 / ライブラリ	バージョン	この章での役割
Tauri	2.11.0	骨組み優先戦略の例で、Rust モジュール・コマンド構造を設計する対象フレームワーク
SvelteKit	2.59.0	骨組みプロンプトで、ルートとコンポーネント構造を設計するフロントエンドフレームワーク
DuckDB	1.5.2	MandaiのDBスキーマループ実習で、SQLスキーマ設計の対象となるデータベース
tokio	-	非同期コマンド例で名前のみ言及。実際のコード作成は後の章で
thiserror	-	骨組み優先戦略の例で名前のみ言及。実際のコード作成は後の章で

2.1 ひとつのループ、サイズが変わるだけ

本書全体でAIと作業するやり方は、ひとつにまとまる。



この一行が、本書全体の作業モデルだ。関数ひとつを書くときも、アプリ全体のアーキテクチャを設計するときも、同じように使う。変わるのは、一周にかかる時間だけだ。小さく回せば数分、大きく回せば一日以上。どちらにしても、**検証なしに次のステップへ進まない**というルールは同じである。

ループのサイズは、おおよそ3段階に分けられる。

- ・ **小さなループ(5～15分)**: 「このSvelteコンポーネントにキーボードショートカットを追加してくれ。」
- ・ **中くらいのループ(30分～2時間)**: 「readbooks.aiのPDFアップロードパイプラインをTauriコマンドに移してくれ。」(readbooks.aiは著者が作ったPDF翻訳アプリだ。**Tauriコマンド(Command)** とは、フロントエンドから `invoke("名前")` を呼ぶとRust関数が実行される通り道のことである。)
- ・ **大きなループ(一日～一週間)**: 「Trading Dashboardの19画面に共通するレイアウトとチャートコンポーネントの設計を確定してくれ。」

サイズが大きくなるほどステップに時間がかかり、大きなループの中には小さなループが入れ子になる。いま回しているループがどのサイズなのかを把握しておく、全体の流れを見失わない。

2.2 ステップ1 —要求

要求とは、AIに文脈・制約・目標を伝える作業である。自然言語で書くが、中身は技術仕様書に近い。良い要求は「何を作るか」だけを言わない。なぜ必要なのか、どこに付くのか、どんなやり方は避けるべきか、成果物がどんな形であるべきかも一緒に含める。この情報が抜けると、AIは空欄を自分で埋める。その仮定が実際のプロジェクトと食い違くと、次のループ2～3回分は抜けた文脈を埋めることに使われる。

良い要求は、たいてい次の3つを満たす。

1. **文脈を含める。**この機能はコードベースのどこに付くのか。周囲のコードは何をしているのか。
2. **制約を明示する。**「Svelte 5 runesのみ使用」「すべてを一つのファイルに押し込めないこと」。
3. **成果物の形を決める。**コードブロックか? ファイル3つか? 「コードのみ、説明なし」がその例である。

この3つが抜けるとどうなるか、2つのアンチパターンで見てみる。

短すぎる要求

最もよく見かけるミスだ。

TauriでDBコマンドを書いて

このような要求を受けると、AIは空いている情報を勝手に埋める。DBの種類、コマンド名、エラー処理のやり方まで、すべてAIが決める。問題は、その決定が間違っている、見た目はもっともらしいということだ。プロジェクトがDuckDBを使っているのに、SQLiteの例が出てくる、といった具合である。続く数ラウンドは機能開発ではなく、最初に抜けた文脈を後から補うことに使われる。

長すぎる要求

逆に、一度に多くのことをやらせるのも危険だ。

Tauri 2アプリを最初から作って。SvelteKit(Svelte 5)を使い、DuckDBを埋め込み、OpenAIを接続し、Mandal-ART UIを作り、Pomodoroを追加し、GTDワークフローも含めて。すべてのファイルを短く保ち、コメントを追加して。Tailwindも使って...

10項目を一度に要求すると、AIはたいてい10項目すべてを浅く処理する。各機能は単独で見るとそれっぽいが、互いに噛み合わない。DBスキーマはUIの状態と合わず、Tauriコマンド名はフロントエンドの呼び出し側と違って来る。5つの小さな要求に分けていれば、ステップごとに検証できた。

2.3 ステップ2 —実装

実装はAIが返してくる成果物だが、まだ「こうすれば良さそうだ」という主張に近い。コードかもしれないし、設計図やファイルツリーかもしれない。成果物が出てきたからといってループが終わったわけではない—検証ステップがその主張を事実として確定してくれるまでは。

確認のやり方は、成果物の種類によって違う。アーキテクチャの素案、DBスキーマ、ファイルツリーのような設計成果物は、まず**読む**。モジュールひとつを追加したり外したりするのは、コード200行を巻き戻すよりずっと安い。一方、実装コードは行単位で長く読むより、まず**実行する**。表面的に自然に見えるコードが、ランタイムで間違っている場合が多いからだ。

「実行する前にすべて読む」という習慣は、まえがきで述べたコードレビューのボトルネックを作る。AIがコードを生成する速度は、私が行単位で読む速度より速い。コンパ

イラと動作テストは、私が手作業で探そうとしていたバグを、より速く、より一貫して捕まえる。コードを理解する能力は依然として必要だ。ただ、すべての行を先に読む必要はない。**先に実行し、失敗した地点を中心に読む**。この「実行で主張を確定させる」段階を、次の節で詳しく見る。

2.4 ステップ3 —検証

検証とは、実行、テスト、レビューである。普通は次の順で進める。

1. **コンパイルと型チェック**。Rustは`cargo check`(コンパイルはするが実行ファイルは作らない高速チェック)、TypeScriptは`tsc --noEmit`(型チェックだけしてファイルを出力しない)。失敗したら、コンパイラ出力をそのまま貼って要求ステップへ戻る。
2. **制約が守られているか**。自分が指定した動作を直接実行してみる。
3. **既存機能は無事か**。隣接機能を実行して、リグレッションがないか確認する。
4. **エッジケース**。不正な入力、境界値、エラー経路の3つを自分で試す。重要な機能はAIに単体テストを書かせる。

ループが小さければ、手動実行1回で十分だ。ループが大きければ、テストを先に書いて、AIにそれに合わせて実装させる。ツールの使い方(`cargo check`、`tsc --noEmit`、テスト構成)は、環境設定と一緒に後の章で詳しく扱う。

コードを目で追って「合っていそうだ」と通すのは検証ではない。目には問題なく見えても、権限ファイルが抜けていたり、イベント名がフロントエンドと違っていたりするケースは、実際に実行しないと現れない。検証を飛ばして実装を2つ連続で入れると、最初のエラーが2つ目の変更の下に埋もれる。3つ目の機能まで載せると、どの問題がどの変更で生じたのかを追跡するのに、機能実装よりも時間がかかるようになる。**検証は最後に1回行う手続きではなく、実装と実装の間ごとにループを閉じる動作である。**

2.5 ステップ4 —反復

反復するときは、間違っている部分だけを再要求する。最初から全部書き直してくれとは頼まない。

上のコードで次の2点だけ直して。

1. <具体的なポイント1>

2. <具体的なポイント2>

それ以外はそのままにして。

「残りはそのまま」が肝だ。この一行がないと、AIは触らなくていい部分まで変えてしまう。そうすると、たった今確認したコードを最初から検証し直さなければならない。

同じ要求で3回失敗したら、反復を止めて要求を最初から書き直す。原因はたいてい3つのうちのどれかである。

- ・ 制約を抜かしている。
- ・ 文脈が足りない。
- ・ 一度に多くを要求しすぎている。

エラーを貼り続けるより、要件と制約を整理し直すほうが速い。

2.6 「完璧なプロンプトひとつでアプリ完成」が幻想である理由

ループを4ステップに切り分けるべき理由は、一度で終わらせようとする試みが、実際にはうまくいかないからだ。X(旧Twitter)には「プロンプトさえうまく書けばアプリができあがる」という投稿があふれている。やってみると違う。理由は3つある。

1. **文脈の限界。** プロンプトをいくら長く書いても、アプリ全体を一度に収められない。
2. **曖昧さが積み重なる。** 自然言語で書いた要件は、条件が10個積み上がると前のほうの条件が違うように解釈されることがある。
3. **検証が難しい。** 一度で出てきた成果物は最初から最後まで丸ごと確認する必要がある、問題が起きたときにどこで間違ったのか見つけにくい。

この3つの問題は、要求を小さく分ければ大半が解決する。だから私は、**短い要求を何度も使う**。一回の要求に入れる要件は3つを超えない。次の節の「骨組み優先戦略」は、この原則をより大きな規模、つまりアプリ全体のレベルで適用したものである。

2.7 骨組み優先戦略—トップダウンAI開発

本書でもっとも重要なアイデアだ。残りはすべて、この細部にすぎない。

Tauriアプリでも、TUIツールでも、バックエンドサービスでも—AIで何を作るにせよ、細部より骨組みを先に固める。全体戦略は5ステップだ。

- | | |
|-------------|---------------------------------|
| ステップ1: 骨組み | —アーキテクチャ、ファイル構造、インターフェース、データフロー |
| ステップ2: 詳細実装 | —各モジュールを1つずつ実装 |
| ステップ3: バグ修正 | —コンパイル、実行、すべてのエラー修正 |
| ステップ4: 機能QA | —各機能を最初から最後まで検証 |
| ステップ5: 仕上げ | —エッジケース、性能、UX |

大きな決定から小さな決定へ下りていく、漏斗の形をしている。

AIはこの順序を知らない。機能を要求すると、構造・ロジック・エッジケース・エラー処理を一度に全部作ろうとする。成果物は完成したように見えるが、もろい。一か所を変えると3か所が壊れる。

開発者が各ステップの範囲を自ら制限すると、結果の品質が上がる。骨組みを設計するときはコードを書かせず、モジュールを実装するときは他のモジュールを触らせない。

ステップ1: 骨組み—アーキテクチャプロンプト

最初のプロンプトは決して「Xを作って」ではない。「Xの骨組みを設計して」である。

以下はMandaiプロジェクトで実際に使った骨組みプロンプトだ。Mandaiは、GTD(やることを収集・処理・整理する生産性メソッド)タスク管理と、DuckDB(アプリ内部に埋め込む軽量データベース)を組み合わせたTauriアプリである。**コードなしで構造だけ受け取るのが、このプロンプトの目的だ。**

[文脈]

MandaiというTauri 2 + SvelteKit(Svelte 5 runes)アプリを作っている。
Mandal-Art (9x9目標グリッド)、GTDタスク、Pomodoroタイマー、
マルチAIチャット(OpenAI、Claude、Gemini)を組み合わせる。

[依頼]

プロジェクトの骨組みを設計して。

- src-tauri/のすべてのRustモジュールと各モジュールの役割(1行ずつ)
- src/routes/のすべてのSvelteルートと表示する画面

- すべてのTauriコマンド名、入力、戻り値の型
- DuckDBテーブル(名前+カラム。SQLはまだ書かない)

[制約]

- 実装コードを書かない
- SQLを書かない
- 構造だけを出力する

いちばん重要な行は制約だ。「実装コードを書くな」が抜けると、AIは聞かれなくてもコードと一緒に吐き出す。そうすると、また再びコードレビューモードへ引きずられ、骨組みを詰めるステップの利点を失う。

出てくるのは**設計図**だ。製品ではない。設計図は読みやすく、変えやすい。AIが望まないモジュールを追加してきたら一行消せばいい—コード200行ではなく。

骨組みレベルの変更は安い。コードレベルの変更は高い。コードを書く前に、必ず骨組みを確定する。

ステップ2: 詳細実装—一度にひとつのモジュール

ポモドーロの最小要求はこう書ける。「start_pomodoro(minutes)とstop_pomodoro(session_id)を実装する。0分と重複開始を拒否し、ID・開始・終了予定時刻を返す。同じIDの停止は二回呼んでも一度だけ終了する。tokio::timeとthiserrorを使い、正常終了・重複開始・不正ID・再起動の期待動作を先に表にする。」関数の数より入力・失敗・復旧条件を明確にする。

このモジュールが完成して検証されたら、次のモジュールに進む。同じパターンを繰り返す。**モジュールひとつを、深く、検証してから次へ。**

ステップ3: バグ修正—コンパイル、実行、修正

モジュールがいくつか実装されたら、プロジェクト全体をコンパイルする。この時点でビルドが失敗するのは普通だ。AIが各モジュールを切り離して書いたせいで、importパスが間違い、モジュール境界で型が合わず、関数シグネチャが変わっている。

バグ修正プロンプトは、機能実装プロンプトとは違う。**精密で短い。**ポイントは2つだ。

- ・ **コンパイラ出力をそのまま貼る。**意識しない。
- ・ **「この2つのエラーだけ」と釘を刺す。**抜けると、AIはまともな関数まで触り始める。

プロジェクトが次のエラーでコンパイルに失敗する：

```
error[E0308]: mismatched types
  --> src/pomodoro.rs:42:12
     expected `i64`, found `String`
```

このエラーだけを修正する。他は触らない。

ステップ4: 機能QA —各機能を最初から最後まで検証

コンパイルできて実行されたら終わり、ではない。機能を一つひとつ、実際のユーザーのように自分で試してみる。

QAループはこうだ。

1. 機能をひとつ選ぶ(例: 「タスクに紐づいたポモドーロセッションを開始」)。
2. 手動で実行する。動くか？
3. 動かなければ: 失敗の正確な地点を特定し、症状と一緒にAIへプロンプト。
4. 修正 -> 再検証 -> 次の機能へ。

バグレポートプロンプトのポイントは、**期待される動作と実際の動作を一行ずつ明確に書くこと**だ。

[バグレポート]

task_id = 5でPomodoroセッションを開始して停止すると、セッションはDuckDBに保存されるが、タスクのaccumulated_minutesが更新されない。

期待動作: task.pomodoro_minutesがセッション時間分増える

実際の動作: task.pomodoro_minutesが0のまま

stop_pomodoro関数を見せて。タスク更新はどこで起きるべきか。

最後の行が肝だ。AIにすぐコードを直させるのではなく、**間違っただけをまず指させる**。場所を取り違えたら、その場で止めてもう一度頼めばいい。

ステップ5: 仕上げ—エッジケース、性能、UX

仕上げは最後のステップだ。正常系がカバーしきれなかったエッジケース、実データでしか現れない性能の問題、1時間試してみて気づくUXの不便を、ここで処理する。プ

ロンプトはたいていエッジケースを番号で整理する(例: ポモドーロタイマーの (1) アプリ再起動時に中断されたセッションの復旧、(2) 重複開始の防止、(3) 0分セッションの保存禁止)。番号を振ると、応答が抜かした項目をすぐ見つけられる。

この順序が重要な理由

5つのステップは、**前のステップに依存する**。前のステップが安定していなければ、後のステップが揺れる。

- ・ バグを取る前にコードを磨くと、すぐ変わるコードを磨くことになる。
- ・ すべてのモジュールを実装する前にバグを取ろうとすると、次のモジュールが追加されるとそのバグが自然に消える場合がある。
- ・ 骨組みを確定する前に細部機能を実装すると、アーキテクチャが変わったときにモジュールを最初から書き直すことになる。

AI支援開発で時間をいちばん無駄にする原因は、ステップを混ぜることだ。順序を守れば、ステップごとに速くなるのではなく、**手戻りそのものが減る**。

注意:「これだけ早く直そう」という誘惑は必ずやってくる。その場で直さず、リストに書きとめておいて、合うステップで処理する。AI支援開発では、コンテキストスイッチのコストは思ったより大きい。

2.8 実例—MandaiのDBスキーマループ

理論は十分見た。実際のサイクルを見てみよう。MandaiのDBスキーマを最初に固める過程だ。2サイクル、約15分かかった。

DBスキーマを最初の例に選んだ理由は単純だ。**コード量が小さくて1画面に収まるので、ループ一周の全体が一目で掴める**。要求と成果物、検証と修正が同じページに見える作業は、ループを最初に身につけるのに最適である。

サイクル1 —要求

スキーマ全体を短い要求にまとめた。肝は、**9×9格子のセル構造、必須のカラムリスト、DuckDBで有効なSQL**だ。

Mandai用の初期DuckDBスキーマを下書きして。

Mandal-Artは9x9グリッド。中央3x3の各セルが下位3x3の中心になる。

各セルはtitle、description、done flag、累積Pomodoro分、created/updated timestampsを持つ。

DuckDBで有効なSQLだけを使う。インデックスも含める。1ファイルにまとめる。

サイクル1 —実装

Claudeが生成した初版から、肝となる部分を抜粋した。このコードがやっているのは、セルと親子関係、基本メタデータカラムを定義することだ。

```
CREATE TABLE cells (
  id INTEGER PRIMARY KEY,
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL,
  title TEXT, description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  updated_at TIMESTAMP DEFAULT now()
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

読むときに押さえるべき行は3つ。parent_id(格子の階層構造を作る)、position(セルの位置で、0〜8でなければならない)、updated_at(初期値がnow()で一度だけ設定される—自動更新ではない)。

サイクル1 —検証

確認してみると、3つの問題が見えた。

1. positionカラムに範囲制約がない。0〜8だけが有効なのに、スキーマが強制していない。
2. DuckDBはSQLiteと自動増分IDの文法が違う。初版にはSQLiteの文法が混ざっていた。
3. updated_atが自動的に埋まらない。アプリのコードで直接値を入れる必要がある。

3つを確認したあと、全体をもう一度要求はしなかった。合っている部分はそのままにし、間違っている部分だけを指摘した。

そのスキーマで3点を直して。

1. `id`をDuckDBの文法に合わせて直す。
 2. `position`に`0..=8`の範囲制約を追加する。中央は4。
 3. `updated_at`はアプリが書き込みごとに入れる。それをコメントに書く。
- `parent_id`構造はそのままにする。

修正版は3つすべてが反映された形で返ってきた。スキーマ初版として受け入れるには十分だった。DuckDBのSEQUENCEやCHECKといった具体的な文法は、第7章で別途扱う。

2サイクル、約15分。スキーマv0.1完了。最初の試みで完璧な結果は期待しなかった。素早く初版を受け取り、問題を具体的に指摘した。「`parent_id`構造はそのまま」と、触らせない部分も明確に指した。ドメイン知識は、コードを最初から自分で書くときだけ必要なのではない。AIが間違えて仮定した地点を絞り込むときにも必要になる。

このパターンをあらゆる作業に使えるよう整理したのが、次のテンプレートである。

2.9 基本プロンプトテンプレート: 何を / なぜ / 制約 / 形式

毎日使う4ブロックのテンプレートだ。空欄を埋めるように書けば、抜けた項目がないかを点検できる。

[何を]

1行: 何を作るか。

[なぜ]

なぜ重要か。含める機能とユーザーフロー。

[制約]

- 言語 / バージョン
- 禁止事項
- 性能 / セキュリティ条件
- 既存コードとのインターフェース

[形式]

- 成果物の形(コードのみ / 説明付き / diff / ファイルツリー)
- ファイルと関数の命名規則

Tauriコマンドの例で埋めてみるとこうなる。この要求が作るのは、**PDFアップロードを受けてローカルに保存するRustコマンドひとつだ。**

[何を]

PDFアップロードを受け取り、ローカルに保存するTauriコマンド。

[なぜ]

readbooks.ai翻訳パイプラインのステップ1。ステップ2-4(解析、翻訳、保存)がここにつながる。ユーザーがPDFをドラッグ&ドロップする。

[制約]

- Rust, tauri 2.x, async
- app_data_dir/uploads/<uuid>.pdfに保存
- 50MBより大きいファイルは拒否
- thiserrorでカスタムエラー型を定義

[形式]

- Rustコードブロック1つ、30行未満
- 説明はコードの上に2-3行まで

この構造どおりに書けば、要求が短くなっても結果の品質は変わらない。手に馴染んだら、4項目を1段落に圧縮してもよい。ただし、AIと新しい作業セッションを始めるときは、4項目を分けて書くほうがよい。

2.10 ループを回すときの注意点

- ・ AIの「ベストプラクティス」主張は、もう一度疑う。学習時点より後にAPIが変わっているケースが多い。最新の公式ドキュメントとクロスチェックする。
- ・ 同じ修正指示で2回同じ結果が返ってきたら、指示そのものが間違っている。表現を変えて再度要求する。
- ・ 重要なプロンプトは保存しておく。アプリごとにprompts.mdに書きとめておけば、2か月後の似た作業にそのまま再利用できる。

この章で必ず覚えておくこと

- ・ 本書のすべての作業は、**要求 -> 実装 -> 検証 -> 反復**ループに従う。作業規模に応じてループの単位だけが変わる。
- ・ **骨組み優先戦略**を使う。コードを書く前にアーキテクチャを確定し、モジュールをひとつずつ実装し、全体が入ってからバグを取り、機能を最後まで検証してから磨く。ステップを行き来すると手戻りが急速に増える。
- ・ 「完璧なプロンプトひとつ」は存在しない。**短い要求を何度も**やるほうが品質も速度もよい。
- ・ 検証なしに実装すると、反復が積み上がるほど技術的負債も積み上がる。
- ・ すべての要求に**何を / なぜ / 制約 / 形式**を入れる。慣れたら段落ひとつに圧縮してもよい。

次章からは、このループを実際のコード作業に適用していく。同じ4ステップが、毎回同じ形で回る様子を、自分の目で見ることになる。

第2サイクルの修正版SQL

DuckDB sequenceでIDを生成し position を0~8に制限する。updated_atは書込ごとにappが設定する。parent_idを維持し、不正positionと参照を試す。

```
CREATE SEQUENCE cell_id_seq;
CREATE TABLE cells (
  id INTEGER PRIMARY KEY DEFAULT nextval('cell_id_seq'),
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL CHECK (position BETWEEN 0 AND 8),
  title TEXT,
  description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  -- The application supplies updated_at on every INSERT/UPDATE.
  updated_at TIMESTAMP NOT NULL
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

第3章 AIコーディング ツーループロンプトだ けで回す開発ループ

第2章では開発ループとは何かを説明した。本章では、そのループを実際のツールの前でどう回すかを扱う。頭で理解することと、AIがそのまま追いかけるリクエストを自分で書くことは別の作業である。

最初はGitHub Copilotを単なる自動補完ツールとして使っていた。コードを数文字打つとCopilotが続きを提案し、Tabキーで受け入れる、という使い方だ。しかしTauriアプリのようにフロントエンド、Rustコマンド(ファイルアクセスのようなシステム操作を担うRust関数)、データベースを同時にかみ合わせる必要があるプロジェクトでは、それだけでは足りない。

いまはTabキーを押す前に、次の三つを先に決める。

1. AIに何を作らせるかを指示する。

2. どんな順序で結果を受け取るかを定める。
3. 受け入れる前に何を確認するかを定める。

この三つが、プロンプトだけで回す開発ループの中心にある。

本章の流れ

- ・ 3.1 本章の範囲
- ・ 3.2 プロンプトだけのループ—6ステップ
- ・ 3.3 毎日使うプロンプトパターン6種
- ・ 3.4 通しセッション—ポモドーロタイマーを作る
- ・ 3.5 まとめ

学習目標

- ・ 自分でコードを打たなくても品質を保てる**プロンプトだけの開発ループ**を組み立てる。
- ・ 安定して機能するプロンプトパターン6種を身につける。
- ・ 自然言語のプロンプトだけでTauriコマンドを作る流れを最後までたどる。
- ・ ツールが変わってもそのまま使える考え方を身につける。

3.1 本章の範囲

ツールの種類と選び方は第4章で扱う。本章ではツールの機能比較ではなく、ツールが変わってもそのまま使える**ループのパターン**に集中する。

例はCopilotを基準にするが、重要なのは製品名ではなく次の三つの習慣だ。

1. リクエストを小さな単位に分ける。
2. 文脈と制約を一緒に渡す。
3. 結果を実行して検証する。

この習慣はCursorでもClaude Codeでも同じように働く。

3.2 プロンプトだけのループコードを直接打たずに開発する

実装コードを一行も自分で打たずに、機能まるごとを作ることができる。ただし、開発者が手を放すという意味ではない。コードを書く代わりに、要件を明確に書き、AIが作ったコードをレビューし、実行結果を確認する側へ役割が移る。

ループはこのように流れる。

リクエスト -> レビュー -> 検証 -> 改善 -> 繰り返し

ステップ1: 自然言語プロンプトを書く

この段階は「AIと会話する」ことではなく、**仕様書を書く**ことだ。AIは曖昧な部分を空欄のまま残さず、自分の判断で埋める。その判断がプロジェクトの慣例と合わなければ、以降のステップで小さな衝突が続けて起きる。

そのため、最初のプロンプトには次の四項目を入れる。次のテンプレートはそのまま使ってよい。

Context: [コードベース内でこれがどこに位置するか]

Goal: [望む具体的な動作]

Constraints: [してはいけないこと、または従うべき特定のパターン]

Format: [出力の構造化方法 — 単一関数、複数ファイルなど]

四項目のうち一つでも空けると、その空白をAIの推測が埋めてしまう。

ステップ2: AIがコードを生成する

生成が完全に終わるまで待つ。途中で止めるとAIが組み立てている構造が切れて、次の回答に別の前提が混ざり込むことが多い。

方向そのものが明らかに違うなら、新しいプロンプトでやり直す。細かい誤りについては、生成が終わった結果を見てから修正リクエストを送る。

ステップ3: 結果をレビューし、動作を検証する

ここが最も重要なステップだ。コードを受け入れる前に、**実際に実行して動作を確認する**。AIが生成したコードは文法も整っていて命名も自然なので、目で見ただけでは問題を見逃しやすい。

確認すべき項目は四つある。

- ・ エラーなくコンパイルされるか。
- ・ 制約条件は守られているか。
- ・ 想定したエッジケースは通るか。
- ・ 隣接する機能を壊していないか。

ステップ4: 受け入れ、却下、または修正

検証が終わったら次の三つから選ぶ。

- ・ **受け入れる**: そのまま反映する。
- ・ **却下する**: アプローチが間違っている。別のプロンプトでやり直す。
- ・ **修正する**: 方向は合っているが詳細が違う。特定の部分だけ修正を依頼する。

生成されたコードを手作業で少しずつ直さない理由は、そうするとAIが次のラウンドでどの状態を基準にすべきか曖昧になるからだ。修正リクエストでループをきれいに保つほうがよい。

ステップ5: 検証

第2章と同じである。

1. **コンパイルチェック**: `cargo check`、`npm run type-check`
2. **手動実行**: 実際に機能を動かす。
3. **エッジケースのテスト**: 壊すように使ってみる。

検証に失敗したら、エラー出力をそのまま控えてステップ6に進む。手で当てずっぽうに直すより、エラーメッセージを根拠にしてAIに任せ直すほうがよい。

ステップ6: 証拠をもとに改善する

失敗を直すプロンプトには決まった形がある。

上記のコードで次のエラーが発生しました：[エラーをそのまま貼り付け]
この特定の問題だけを修正してください：

1. [具体的な問題1]
2. [具体的な問題2]

残りはすべてそのままにしてください。

最後の「**残りはすべてそのままにしてください**」が要だ。この一文がないと、AIは動いていたコードまで書き直すことがある。そうすると、いま終わったばかりの検証を最初からやり直すことになる。

3.3 プロンプトパターン—実戦の道宝箱

6ステップのループが大きな流れだとすれば、本節の6つのパターンは各ステップで取り出して使う道具である。**プロンプトは技術文書だ**。関数シグネチャやAPI仕様と同じくらい正確に書く必要がある。

パターン1: 骨組みプロンプト—コードよりアーキテクチャが先

実装を頼む前に、必ず構造設計を先に頼む。この段階で構造を誤ると、以降のすべてのラウンドがその上でずれていく。

次はポモドーロモジュールの骨組みを組むときに送るリクエストだ。要は「**構造だけ、コードブロック禁止**」の一行である。

[文脈]

Mandai 生産性アプリ。Tauri 2 + SvelteKit 2 + Svelte 5。DuckDB/バックエンド。

現行モジュール：mandal-art CRUD、GTDタスク。

追加対象：ポモドーロタイマー機能。

[依頼]

ポモドーロモジュールの骨組みを設計せよ：

1. Rust側：すべての関数名、入力型、戻り値型（実装なし）
2. Svelte側：すべてのコンポーネント、props、呼び出すTauriコマンド

3. DuckDB: テーブル名、カラム、型 (CREATE TABLE SQLはまだ禁止)
4. イベントの流れ: どのTauriイベントがemitされ、誰がlistenするか

[出力]

構造のみ。コードブロック禁止。実装禁止。

ここで受け取るのは**設計図**だ。テキストの一行を消すのは一瞬だが、すで書き上がった関数を作り直すには一時間かかる。

パターン2: 範囲を絞った修正

うまく動かないときに「動きません」とだけ書かない。**正確なエラー**を貼り付け、**何をどう変えるか**を範囲で指定する。

```
error[E0308]: mismatched types
  --> src/pomodoro.rs:42:12
   |
42 |         return sessions;
   |         ^^^^^^^^^ expected `Result<Vec<Session>, String>`,
   |                     found `Vec<Session>`
```

この戻り値の型不一致のみ修正せよ。

Ok() で包め。

クエリのロジックやSession構造体は変更するな。

このプロンプトが従う規則は三つある。

1. **コンパイラ/ランタイムの出力をそのまま貼る** — AIは行番号まで読む。
2. **修正範囲を明示する** — 「これだけ修正せよ」が他の箇所への波及を防ぐ。
3. **保存するものを明示する** — 「変更するな」が不要な書き直しを止める。

パターン3: スペック先行プロンプト—実装よりシグネチャが先

関数シグネチャ (関数名、入力、戻り値型) を先に固定してから実装を頼む。シグネチャと本体をまとめてAIに任せると、意図しないパラメータ型が混じってくる。フロントエンドとバックエンドが同じインターフェースを約束する場面では特に重要だ。

次の正確な関数シグネチャの実装を書け:

```
pub async fn stop_pomodoro(
    db: State<'_, DbPool>,
    session_id: i64,
) -> Result<PomodoroResult, AppError>
```

動作:

- pomodoro_sessionsからアクティブなセッションをクエリ
- 経過分を計算
- セッションの状態を 'completed' に更新し、ended_at をセット
- 紐付くタスクの accumulated_minutes を更新
- セッションのサマリーを含む PomodoroResult を返す

関数シグネチャを変更するな。パラメータを追加するな。

最後の二文が核心だ。シグネチャが揺れると、呼び出し側のコードも一緒に揺れる。

パターン4: 否定の制約—してはいけないことを書く

AIは学習データのパターンを既定値として使う。Tauri 2を頼んでもTauri 1のパターンが出てくるし、Svelte 5を頼んでもSvelte 4の構文が混ざる。望まないものを直接書いてしまうほうが確実だ。

Mandai用のDuckDBクエリコマンドを生成せよ。

必須:

- データベース接続に tauri::State を使う
- カスタムエラー型で Result<T, AppError> を返す
- パラメータ化クエリを使う

絶対禁止:

- unwrap() や expect() の使用一切禁止
- rusqlite の使用禁止(duckdb クレートを使うこと)
- 生のデータベースエラーをフロントエンドに返すな
- Tauri 1 のAPI使用禁止

必須 / 絶対禁止 の形式は、AIが機械的に追従できるチェックリストになる。絶対禁止リストを書くと、修正回数が大きく減る。

パターン5: ファイル参照で文脈を固定する

コードベースの慣例を言葉で説明するより、慣例が表れているファイルを直接指し示すほうがはるかに正確だ。AIはそのファイルを読んでパターンを推定する。

#file:src-tauri/src/db.rs (データベースモジュール) と
#file:src/lib/stores/timer.ts (フロントエンドのタイマーストア) を見て、

新しいTauriコマンド `get\_pomodoro\_stats` を書け:

- DuckDBから総セッション数、総分数、平均時間をクエリ
- timer.ts のTypeScriptインターフェースに合う構造体を返す
- db.rs の他のコマンドと同じエラーハンドリングパターンを使う

ツールごとにファイル参照の構文は違う。

ツール	構文
GitHub Copilot Chat	#file: パス(空白なし)
Cursor	@ファイル名
Claude Code	ファイル名に言及するだけで自動的に含まれる

パターン6: フェーズ認識プロンプト—現在のフェーズを明示する

作業を複数のフェーズに分けて進めるとき、いまどのフェーズにいるかをプロンプトに書いておくと、AIは範囲から外れない。

[フェーズ: バグ修正 —機能を追加するな]

プロジェクト全体をコンパイルしたあと、三つのエラーが残っている:

1. src/ai/mod.rs:15 — StreamExt のimport漏れ
2. src/mandal/grid.rs:88 — Cell 構造体に #[derive(Clone)] が必要
3. src/main.rs:42 — register_commands! マクロのモジュールパスが誤り

三つすべてを修正せよ。変更した行のみを前後3行のコンテキストとともに示せ。
リファクタリングするな、最適化するな、機能を追加するな。

冒頭の【フェーズ：バグ修正】タグが基準点だ。これがないと、AIはエラー一つを直すついでにモジュール全体を再設計してしまう。

パターンの組み合わせ

実際のプロンプトは大半が二つか三つのパターンを同時に使う。フェーズ認識、ファイル参照、スペック先行、否定の制約を一度に入れる、という具合だ。各パターンが防ぐ問題は次のとおり。

- ・ フェーズ認識 -> 範囲外への逸脱を防ぐ。
- ・ ファイル参照 -> プロジェクトの慣例を破ることを防ぐ。
- ・ スペック先行 -> APIインターフェースが想定外の方法へ流れることを防ぐ。
- ・ 否定の制約 -> すでに知っている落とし穴に再度はまることを防ぐ。

組み合わせた全文のプロンプト例はリポジトリの付録で確認できる。

3.4 通しセッションーポモドーロタイマーCommandを組み立てる

ここまでに見たパターンが、一回の作業の中でどう束ねられるかを見てみよう。Mandaiアプリのポモドーロ機能のためのTauriコマンドを、コードを直接打たず、AIとの対話だけで完成させる。全部で3ラウンドである。

ラウンド1 —最初のリクエスト

最初のリクエストでは、何を作るか、どんな引数を受け取るか、どんな動作とエラー処理を求めるかをまとめて書く。

`start\_pomodoro\_timer` という新しい Tauri コマンドを作ってください。

Context: DuckDB バックエンドを持つ生産性アプリ Mandai 用です。

Args: goal_id (String), duration_minutes (u32)

動作:

- pomodoro_sessions テーブルに新しいセッションを挿入
- バックグラウンドタイマーを開始

- 毎秒、残り時間を含む 'timer_tick' イベントを emit
 - 完了時に 'timer_done' を emit
 - session_id を返す
- エラー処理: Result<String, String> を返す、unwrap() なし

AIから返ってきた応答の構造だけを押さえる。次の抜粋は「このコマンドが何をしているか」を一望するためのもので—DBにセッションを入れ、バックグラウンドで毎秒イベントを送り、終わったら完了イベントを送る。実装の全体はリポジトリで確認できる。

```
#[tauri::command]
pub async fn start_pomodoro_timer(
    app: tauri::AppHandle,
    goal_id: String,
    duration_minutes: u32,
) -> Result<String, String> {
    let session_id = uuid::Uuid::new_v4().to_string();

    // 1. DBにセッションを挿入 (INSERT INTO pomodoro_sessions ...)
    // 2. バックグラウンドタイマーを spawn
    tokio::spawn(async move {
        for remaining in (0..=duration_minutes * 60).rev() {
            app.emit("timer_tick", /* TimerEvent */).ok();
            tokio::time::sleep(Duration::from_secs(1)).await;
        }
        app.emit("timer_done", session_id.clone()).ok();
    });

    Ok(session_id)
}
```

レビュー—何が合っていて何が抜けているか: 基本構造は合っている。ただし次の三点が抜けている。

1. TimerEvent structが定義されていない -> コンパイル不可。
2. タイマーを途中でキャンセルする手段がない。
3. tokio::spawn ブロックは別スレッドで動くため、goal_id をその中でそのまま使えない(所有権の問題)。

三つの問題のうち1番はコンパイルを止めており、2〜3番は動作上の問題だ。まずコンパイルを通すのが順序である。

ラウンド2 —欠けている型を解決する

まずコンパイルを阻んでいる `TimerEvent` 型から片付ける。

上記で使われている `TimerEvent struct` を定義してください。

含める要素: `session_id (String)`, `remaining_seconds (u32)`, `goal_id (String)`

Tauri イベント用に `Serialize + Clone` を付けてください。

返ってきた構造体は三つのフィールドを持ち、`Serialize`(JSON変換用)と`Clone`(同じイベントを複数箇所へ渡すため)のトレイトを備える。

確認する出力: この時点で `cargo check` が通るはずだ。

ラウンド3 —スコープ問題 + キャンセル機能

残る二つの問題を一度に解決する。すでに動いている部分には触れるなと明示して、ラウンド1の成果を保存する。

二つだけ修正してください:

1. `goal_id` を `spawn` の前に `clone` し、`TimerEvent` に含めよ
2. `cancel_pomodoro_timer` コマンドを追加: `session_id` を受け取り、DB で `status = 'cancelled'` に更新

タイマーループ内では 10秒ごとに DB の状態を確認し、`'active'` でなければ終了させること。

残りのロジックは変更するな。

追加された `cancel_pomodoro_timer` がやることは一行のUPDATEクエリだ—セッションの状態を `cancelled` に切り替えるだけである。実際のタイマー停止は、別スレッドで回っているループがこの状態変化を見て自分から抜け出すことで起きる。

確認する出力: `cancel_pomodoro_timer` を呼び出してから10秒以内に `timer_tick` イベントが止まるはずだ。

セッションのまとめ

- ・ **総ラウンド**: 3回
- ・ **所要時間**: 約20分
- ・ **自分で打った実装コード**: 0行

AIは基本パターンを最初から正しく組み立てた。追加の案内が必要だったのはマルチスレッド間のデータ共有とキャンセル機能だけだった。これがプロンプトだけで回すループの典型的な流れだ—大きな構造は一度で受け取り、細部は範囲を絞った後続ラウンドで整える。

3.5 まとめ

本章で必ず覚えておくべきことは二つある。

- ・ **6ステップのループ** —リクエスト、生成、レビュー、受け入れ/却下/修正、検証、証拠に基づく改善。
- ・ **6つのパターン** —骨組みプロンプト、範囲を絞った修正、スベック先行、否定の制約、ファイル参照、フェーズ認識。

6つのパターンのうち実戦でいちばん頻繁に使うのは二つだ—**骨組みプロンプト** と **否定の制約**。骨組みを先に組み、してはいけないことを明示するだけで、修正ラウンドは大きく減る。

この二つを身体に染み込ませると、コードを自分で打たなくても成果物の品質を保てる。

第4章では、実際に使うツールを選ぶ理由と環境設定を扱う。第5章ではTauri 2の基礎に入る。

最小検証コマンド

project内で実行し、終了codeだけでなく機能の実際の結果を確認する。

```
cargo check --manifest-path src-tauri/Cargo.toml  
pnpm check  
pnpm tauri dev
```

第4章. ツールと環境

—Claude Code、Cursor、Codex、GitHub Copilot

第3章ではプロンプトループと6つのパターンを扱った。本章ではそのパターンを実際のツールへつなげる。どのツールを使うかよりも、そのツールでどのようにループを保つかが本質となる。

ツール選びは比較表だけで結論を出しにくい。「Claude CodeとCursorのどちらが優れているか」という問いは、実際の開発を前へ進めない。もっとよい問いは別にある。

いまの作業の規模とリスクを見て、どのツールが詰まらずに結果まで連れていってくれるか、という問いだ。

同じAIでも、関数1つを直すときと、リポジトリ全体の構造を変えるときに必要な環境は違う。ここで言う**フィードバックループ**とは、AIに依頼し、結果を確認し、再び修正を依頼するという反復のサイクルを指す。ループが短いほどミスを早く捕まえられる。よいツールとは「いちばん賢いツール」ではなく、いまの作業のフィードバックループをいちばん短くしてくれるツールである。

この章を最後まで実践すると手元に残るもの: 動作するTauriアプリの土台、プロジェクト規約ファイル(CLAUDE.mdまたは.cursorrules)、そして2つのgitコミット。つまり、第5章から本格的にAIと一緒に開発を始めるときにすぐ使える環境が手に入る。

AIコーディングツールは実行される場所によって大きく3種類に分かれる。**ターミナル(CLI)**で動かすもの、**IDEの中で動くもの**、そして**クラウドで動くもの**だ。ターミナル系はコマンド1行でリポジトリ全体を扱いやすく、IDE系はコードのすぐ横で短い編集を素早く行える。クラウド系は自分のPCではなく外部環境で作業するため、ローカルに触れずに任せたいときに使う。

この章の流れ

- ・ 4つのツール、4つのアプローチ
- ・ いつ何を使うか
- ・ Tauri 2 + SvelteKit + Rustの最小環境
- ・ 最初の30分—スキャフォールドから最初のコミットまで
- ・ 規約ファイル(CLAUDE.md、.cursorrules)
- ・ AIに与える制約—スタックとバージョンの明示
- ・ もう少し知っておく: MCP、Agent、スラッシュコマンド

学習目標

- ・ 4つのツール(Claude Code、Cursor、Codex、GitHub Copilot)の違いを理解する。
- ・ ローカルにTauri 2 + SvelteKit + Rustが動作する最小環境を構築する。
- ・ CLAUDE.md / .cursorrules 規約ファイルを書いて、AI出力の品質を高める。

相互参照: 本章は第3章のプロンプトループと6つのパターンを受け、第5章のTauri 2の踏み込んだ解説へつながる。

4.1 4つのツール、4つのアプローチ

2026年時点で、実務でよく比較されるAIコーディングツールは4つある。名前が似ていても、違いはモデルよりも**実行場所**と**作業の進め方**に現れる。

ツール	実行環境	モデル	主な強み
Claude Code	ターミナル(ローカル)	Claude (Anthropic)	長いコンテキスト、リボジトリ全体の探索
Cursor	IDE(VS Code フォーク)	選択可能	高速なインライン編集と補完
OpenAI Codex	クラウドサンドボックス	OpenAI	隔離実行、ローカル環境に依存しない
GitHub Copilot	IDE + クラウド + CLI	選択可能	GitHubのIssue/PR連携、インライン補完

ツールごとに得意な居場所がある。順番に見ていく。

Claude Codeはターミナルで動き、プロジェクトフォルダ全体をコンテキストとして扱える。長い対話と複数ファイルを同時に相手にする作業に強い。

CursorはVS Codeをベースに作られたIDEだ。エージェント作業と補完を同じ画面で併用でき、短い編集を素早く回せる。

OpenAI Codexは2021年のコード生成モデルと名前が同じだが、まったく別の製品だ。クラウド上の隔離空間で実行され、ローカル環境に影響を与えない点が特徴になる。

GitHub Copilotは元々は補完ツールだったが、エージェントプラットフォームへと拡張された。IDE内のAgentモード、GitHubのIssueを自動処理するCloud Agent、ターミナル向けのCopilot CLI、そして従来のインライン補完という4つの使い方がある。最初は名前と役割だけ覚えておけば十分だ。

本書ではClaude Code + Cursorを中心に使う。GitHub CopilotとCodexは特定の場面で代替手段として触れる。

注意: 同じ作業に複数のエージェントを同時に走らせると競合が起きやすい。とくに同じファイルを2つのエージェントが触る場合だ。作業ごとに使うツールを1つに決めて、切り替えはきっぱり行うのがよい。

4.2 いつ何を使うか

ツール選びは好みではなく、**作業の単位**の問題だ。「この作業を一度にどれだけ広く見る必要があるか」で分ける。

Claude Codeが向く場面

複数ファイルにまたがる新機能の追加、大規模なリファクタリング、プロジェクト構造を最初に組む作業など。デバッグでログ、スタックトレース、ソースを一緒に追うときも同様だ。ファイル構造全体が見えるため原因を辿りやすい。ただし、扱う範囲が広いぶん、**検証基準**を明確に渡す必要がある。そうでないと、たくさんのファイルを変えたあげく要件と違う方向へ流れることがある。

Cursorが向く場面

1ファイル内の関数1つ、特定のコード片の説明、1行の補完のように、**修正箇所がすでに明らかな作業**。逆に変更範囲が複数のディレクトリへ広がると、カーソル中心の文脈はかえって狭く感じられる。

OpenAI Codexが向く場面

ローカル環境に触らずに隔離されたクラウド空間で実行したいとき。時間のかかる作業をローカルマシンに負荷をかけずに任せたいときに役立つ。

GitHub Copilotが向く場面

反復的なボイラープレートを書くなら、インライン補完が一番速い。GitHubのIssueを自動でPRに仕上げたいときはCloud Agentを使う。

筆者の場合、1日の作業の半分以上はClaude Code、次に多いのがCursorで、残りは状況に応じて補助的に使う程度だ。ゼロから新しく作るアプリほどClaude Codeの比率が上がり、既存画面の調整作業が多くなるほどCursorの比率が上がる。

4.3 Tauri 2 + SvelteKit + Rustの最小環境

ここではTauri 2の開発環境を初めて整えるときに打つコマンドをまとめる。それぞれのコマンドがなぜ必要かも合わせて説明する。

バージョン固定の方針

インストール前にバージョンを正確に固定する必要がある。**AI開発ではとくに重要だ。** AIはさまざまなバージョンのコードを学習しているため、バージョンを明示しないとTauri 1.x(現在は動作しない)、Svelte 4のstore、古いSvelteKitのルーティングといった古いコードを生成しかねない。そこで本書では次のバージョンを基準にする。

- Tauri: 2.11.0
- SvelteKit: 2.59.0
- Svelte: 5.55.0 (rune構文)
- Node.js: 24.x LTS
- Rust: stableチャンネル、1.95.0

必須インストール手順

各ステップは一度だけ実行する。コマンドが止まらず最後まで通れば成功だ。

1. Rustツールチェーン

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
rustup default stable
```

インストール後に `rustc --version` で1.95.0以上が表示されればOK。

2. Node.js (バージョン管理用のfnm)

```
curl -fsSL https://fnm.vercel.app/install | bash
source ~/.zshrc
fnm install 24
```

```
fnm use 24
fnm default 24
```

3. パッケージマネージャ (pnpm推奨)

```
npm install -g pnpm@latest
```

4. Tauri CLI

```
cargo install tauri-cli@2.11.0
pnpm add -D @tauri-apps/cli@2.11.0
```

5. プラットフォーム別の要件

プラットフォーム	インストールコマンド
macOS	<code>xcode-select --install</code>
Linux (Ubuntu/Debian)	<code>sudo apt install libwebkit2gtk-4.1-dev libayatana-appindicator3-dev librsvg2-dev patchelf</code>
Windows	Microsoft C++ Build Tools、WebView2 (Win 11では通常プリインストール)

AIツールの設定

```
# Claude Code CLI
npm install -g @anthropic-ai/claude-code
claude --version

# Codex CLI
npm install -g @openai/codex
codex --version
```

Cursor IDEはcursor.comからダウンロードする。既存のVS Code設定を自動で引き継いでくれる。

APIキーの環境変数はシェル設定ファイルに追加する。

```
export ANTHROPIC_API_KEY="your-key-here"  
export OPENAI_API_KEY="your-key-here"
```

注意: APIキーをコードに直接書いたり、gitにコミットしてはいけない。環境変数か、`.gitignore`に登録した`.env.local`を使う。AIツールがキーをコードへそのまま埋め込むよう提案してきたら、必ず断る。

SvelteKitを作ってからTauriを追加する

scaffolderとTauri CLIの版は別である。以前の`create-tauri-app@2.11.0`は公開scaffolderの版を指さない。まずSvelteKitを作り、Tauriを追加する。

```
pnpm dlx sv create my-app  
cd my-app  
pnpm install  
pnpm add @tauri-apps/api  
pnpm add -D @tauri-apps/cli@2.11.4 @sveltejs/adapter-static  
pnpm tauri init
```

SvelteKit minimalとTypeScriptを選ぶ。tauri initでは`http://localhost:5173`、`pnpm dev`、`pnpm build`、出力`../build`を指定する。WebViewへSvelteKit serverは同梱しないためstatic SPAにする。

```
import adapter from '@sveltejs/adapter-static';  
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';  
  
const config = {  
  preprocess: vitePreprocess(),  
  kit: {  
    adapter: adapter({ fallback: 'index.html' })  
  }  
};
```

```

    }
  };

  export default config;

```

root layoutでSSRを無効にする。

```

// src/routes/+layout.ts
export const ssr = false;

```

build.frontendDistが../buildか確認してpnpm tauri devを実行する。Tauri窓内にSvelteKit画面が開き、consoleにerrorがないことを確認する。

インストールが失敗したとき—よくある3つの原因

ウィンドウが開かない、あるいはビルドが失敗するとき、入門段階ではほぼ次の3つのどれかだ。

- ・ **Rustのパス未設定:** rustc --versionが動かないなら、source ~/.cargo/envをもう一度実行するか、新しいターミナルを開く。
- ・ **WebView2の欠落 (Windows 10):** Tauriはシステムのウェブビューを使う。Win 11では通常入っているが、Win 10では別途インストールが必要だ。Microsoftの公式サイトで「Evergreen Bootstrapper」を入手する。
- ・ **Linuxのライブラリ不足:** libwebkit2gtk-4.1-devのようなパッケージが抜けていると、pnpm tauri devの途中でコンパイルエラーになる。上のapt installコマンドを再確認する。

メモ: クロスプラットフォームビルド (macOSからWindows用ビルドなど) とリリース自動化は第12章で扱う。いまは**自分のPCでとにかく動く**ところまで確認できれば十分だ。

4.4 最初の30分—スキファールドから最初のコミットまで

4.3節のSvelteKitとTauriの作成手順を終えてから、以下を確認する。

すぐに確認 (5分)

1. 開発サーバーの確認

```
cd your-app-name
pnpm install
pnpm tauri dev
```

アプリが開いて「Welcome to Tauri!」が見えるはず。ブラウザコンソールにエラーがあってはいけない。

2. ビルドの確認

```
pnpm tauri build
```

初回はRustのコンパイルに2〜3分かかる。エラーなく終わればsrc-tauri/target/release/bundle/に実行ファイルができる。

3. VS Code連携の確認

CursorかVS Codeでプロジェクトを開く。次の3点が見えるはずだ。- src-tauri/フォルダでRust構文がハイライトされる - src/フォルダでSvelte構文がハイライトされる - TypeScriptエラーが出ていない

プロジェクト設定 (10分)

4. 規約ファイルを作る

まずプロジェクトルートにCLAUDE.mdを作る。Claude Codeはセッションを開始するとき、作業フォルダにあるCLAUDE.mdを自動で読み込み、システムコンテキストに取り

込む。つまり、毎回プロンプトに同じ規約を貼り付けなくても、このファイルに書いた内容が既定の約束として一緒についてくる。

プロジェクト名

スタック

- Tauri 2.11.0、SvelteKit 2 + Svelte 5、TypeScript、DuckDB
- Svelte 5 rune構文のみ (\$state, \$derived, \$effect)

構造

- フロントエンド: SvelteKit (src/)
- バックエンド: Rust Tauriコマンド (src-tauri/src/commands/)
- データベース: Tauriコマンド経由でのみDuckDBへアクセス

規約

- フロントエンドから直接APIを呼ばない
- 外部データは必ずTauriコマンド経由
- すべてのコマンドの戻り値にResult<T, AppError>を使う
- プロダクションコードにconsole.logを残さない

禁止事項

- SvelteのstoreやレガシーなReactive文を使わない
- Rustでunwrap()を使わない (適切なエラー処理を行う)
- ファイルパスのハードコードをしない (Tauri path APIを使う)

同じ場所に.cursorrulesも置く。Cursorは同じ役割で読み込んでくれる。中身はCLAUDE.mdと似た規約でかまわない。

5. gitの初期化と最初のコミット

```
git init
git add .
git commit -m "Initial Tauri + SvelteKit scaffold"
```

最初の生産的な変更 (15分)

6. 最初のカスタムTauriコマンドを作る

自分でコードを書く前に、AIがTauriのパターンをきちんと守れるかを先に確かめる。
次のプロンプトをAIに送る。

基本的なアプリのメタデータを返す「get_app_info」というシンプルなTauriコマンドを作ってください。

戻り値型: `Result<AppInfo, AppError>`。AppInfoには以下を含めてください:

- `app_name: String`
- `version: String`
- `platform: String`

CLAUDE.mdのプロジェクト規約に従ってください。

7. 新しいコマンドのテスト

```
pnpm tauri dev
```

フロントエンドからこのコマンドを呼んだとき、エラーが出なければOKだ。

AIのコードが動かないときに最初に見るべき場所 1. **ファイルの位置**: 新しいファイルがCLAUDE.mdで指定したフォルダ(`src-tauri/src/commands/`)に置かれているか? 2. **戻り値型**: `Result<T, AppError>`の形で終わっているか? `unwrap()`が紛れていないか? 3. **バインディング**: フロントエンド用のTypeScript呼び出しコードが`src/lib/api/`に一緒に生成されているか? 4. **バージョン**: AIがSvelte 4のstoreやTauri 1.xのパターンを混ぜていないか? この4つのどれか1つでもズレるとビルドが失敗する。コードを書き直す前に、まず上の項目を確認する。

8. 動くコマンドをコミットする

```
git add .  
git commit -m "Add get_app_info Tauri command"
```

確認チェックリスト

30分が経ったら、下の項目を点検する。

- ・ ☒ devサーバーとbuildコマンドの両方が動く
- ・ ☒ AIの動きを縛る規約ファイルが存在する

- ・ ☒ コミットが2つある (スキヤフォールド + 最初の機能)
- ・ ☒ AIが規約に従っていることを確認した
- ・ ☒ TypeScriptとRustのコンパイルエラーがない

得られるもの: 以降のAI作業はここで作ったパターンをそのまま引き継ぐ。

注意: 検証ステップを飛ばさないこと。設定の問題はコードが500行積み上がる前に潰しておく。AIがコンパイルも実行もできないコードを生成し続けると、原因を探すのに後でもっと時間がかかる。

4.5 .cursorrules / CLAUDE.md —規約ファイルの活用

この2つのファイルには、**AIが既定で従う規約**を書いておく。プロンプトの中で毎回説明しなくても、AIが自分で守ってくれる。4.4で新規プロジェクト用の基本テンプレートを見た。ここでは実際にリリースしたプロジェクトの規約ファイルがどう磨かれているかを見る。

CLAUDE.md (Claude Code用) —実例

リポジトリのルートに置けば、Claude Codeはセッション開始時に自動で読む。次はMandaiプロジェクトのCLAUDE.mdの抜粋だ。

```
# Mandai
```

```
## スタック
```

- Tauri 2、SvelteKit 2 + Svelte 5、TypeScript、DuckDB (duckdbクレート)
- AIプロバイダ: OpenAI、Anthropic、Gemini (統合コマンド経由)

```
## 規約
```

- Svelte 5 runeのみ (\$state, \$derived, \$effect)。レガシーstore構文は禁止。
- Tauriコマンドは src-tauri/src/commands/*.rs に置く。

- エラーは `thiserror` ベースの `MandaiError` に集約する。
- フロントエンドからのAI呼び出しは禁止。必ずTauriコマンド経由にする。

禁止事項

- コミットに `console.log` を残さない。
- プロダクションコードで ``unwrap()`` を使わない（テストはOK）。

このファイルがあれば、「Svelte 5 runeだけを使え」「unwrapを使うな」といった指示をプロンプトごとに繰り返さなくて済む。

.cursorrules —同じ概念、別のファイル名

考え方はCLAUDE.mdと同じだ。ファイル名だけが違う。Cursorは.cursorrulesを、Claude CodeはCLAUDE.mdを読む。両方のツールを併用するプロジェクトなら、2つのファイルを並べて置き、中核となる規約をそろえておけばよい。

アプリごとに追加の規約が必要になることもある。たとえばリアルタイムデータを扱うアプリなら、「WebSocket接続はTauriバックエンドで管理する」「フロントエンドはTauriイベントでデータを受け取る」といったアプリ固有の規約を足す。共通規約は新規プロジェクトのテンプレートからそのまま持ってきて、アプリ固有のパターンだけを追記すればよい。

規約ファイルの効果

- ・ プロンプトが短くなる。「このリポジトリでは…」という前提説明が消える。
- ・ ループの反復回数が減り、AIの誤生成も減る。
- ・ リポジトリをチームメンバーに引き渡せば、**AIの規約がそのままチームの規約**になる。

規約ファイルは依存関係を上げたり構造を変えたりするときに一緒に書き直す。ファイルが古くなるとAIは古いパターンのコードを書く。

4.6 AIに与える制約—スタックとバージョンの明示

規約ファイルが整ったあとの次の段階は、**プロンプトごとにどの制約を明示するか**だ。中核の規則は1つに集約できる。

AIに作業を任せるときは、**使う技術とバージョン、そして前提とすべきパターン**を明示する。

制約を渡さないと、AIは学習データの中で一番ありふれたパターン(たとえばElectron、React、PostgreSQL)でコードを作ろうとする。「Tauri 2を使う」「SvelteKit 2 + Svelte 5のrune構文だけを使う」「バックエンドはRust」といった一言二言をプロンプトの先頭に添えるだけで、誤った既定値を防げる。

制約を観測可能な完成条件へ変える。DBなら「二回目の起動で適用済みmigrationを再実行しない」、WebSocketなら「三回切断しても接続は一つ」、UIなら「キーボードだけで保存とエラー確認ができる」と書く。第7章はDB、第13章は復旧、第14章は配布を説明する。道具の名前より成功の観測方法が重要だ。

注意: ツールや技術選択の制約は**明示的に、繰り返し**与える必要がある。常に思い出させないと、AIは最もありふれたパターンを既定として使う。

4.7 もう少し知っておく: MCP、Agent、スラッシュコマンド

AIツールの周辺では新しい用語が次々に増えている。最初の通読では名前と役割だけ覚えておき、実際に触る場面が来たらまた見直せばよい。

MCP(Model Context Protocol) —AIが外部ツールやデータ(ファイルシステム、git、DB、ウェブ検索など)につながるときの標準アダプタ。つないでおけば、「このプロジェクトのcellsテーブルのスキーマを確認して」と短く頼んでもAIが直接読みに行ってくれる。スキーマをコピー&ペーストする手間が減る。

Agent —複数のステップを自分で計画して実行するAI。Claude Codeの既定の動き方もこれだ。「このリポジトリにテストを追加して」と頼めば、計画・実行・検証を自分で進める。ただし、ステップが一気に流れるとどこで間違えたか見えにくいので、出力がすべて見えるモードで使うのが安全だ。

スラッシュコマンド—`/commit`、`/review-pr`のように名前を付けて保存しておくプロンプトだ。同じ依頼を5回以上繰り返すようになったら作る価値がある。一度しか使わない依頼には作らない。

リポジトリ構成(モノレポ vs 分割)のような決定も、最初の通読では気にしなくてよい。最初のアプリは単一リポジトリで始め、共有したいコードがはっきり見えてからモノレポを考えても遅くない。このテーマは本書終盤の運用の章で改めて扱う。

4.8 ツール選びで守る5つの原則

1. **規約ファイルから書く。**新しいリポジトリを作ったら、コードより先に`CLAUDE.md`を書く。AIに送るプロンプトの品質はほぼこのファイルで決まる。
2. **月に1回は設定を見直す。**AIツールは速く変わる。いま最適だと思っている設定が、1か月後には古くなっていることがある。
3. **バージョンは必ず明示する。**バージョンを書かないと、AIが勝手にバージョンを選んでコードを生成する。それがプロジェクトに合わないバグの種になる。
4. **設定をすぐに試す。**初期設定が間違っていると問題が積み上がる。機能を開発する前に、`dev`、`build`、AI連携がきちんと動くかを先に確認する。
5. **規約ファイルも合わせて保守する。**依存関係を上げたり構造を変えたりするときは`CLAUDE.md`も合わせて直す。ファイルが古くなるとAIは古いパターンのコードを書く。

この章で必ず覚えておくこと

- ・ **ツールごとに得意な仕事が違う。**大きな構造の作業はClaude Code、1ファイルの編集はCursor、繰り返し入力補完はCodex。競合ではなく役割分担だ。
- ・ **AIに何をを使うかを明示する。**何も言わなければ、AIは学習データの中で最もありふれたパターン(React、PostgreSQL)を既定にする。

- ・ **バージョンを固定する。** Tauri 2.11.0、SvelteKit 2.59.0のように明示しないと、互いにかみ合わないコードが混ざる。
- ・ **CLAUDE.mdと.cursorrulesの2ファイルが一番効く。** 同じ規約をいちいち書き直さずに済むようになる。
- ・ **30分以内に、環境構築、規約ファイルの作成、最初のコミットまで終える。** このパターンが以後すべての作業の基準になる。
- ・ MCP、Agent、スラッシュコマンドは、いまは名前と役割を知っておくだけで十分。必要になったらまた見る。

一行でまとめるとこうなる。**AIコードの品質は、どのモデルかよりもどんな制約を与えるかで決まる。** 制約をうまく与えた普通のAIは、制約のない優秀なAIより役に立つ。

いま自分にできるようになること: - 作業の種類でAIツールを選ぶ (構造的な作業 vs こまかい編集 vs 繰り返しコード) - バージョンを固定して、完成度のあるTauri 2 + SvelteKit開発環境を整える - AIコードの品質を上げ、手直しの回数を減らす規約ファイルを書く - よくある間違いを避ける: バージョン不一致、制約の抜け、誤ったスキマフォルド - 新規プロジェクトの最初の30分で生産的なパターンを作る

次へ: 第5章ではTauri 2の基礎をじっくり扱う。AIベースのワークフローを支えるアーキテクチャ、コマンドシステム、状態管理パターンを見ていく。

相互参照: 本章で固めたワークフローのパターンは、第6章の高速プロトタイピングと第12章のプロダクション配布戦略へつながる。

第5章. Tauri 2の基礎

—中核パターンとAIの ミス防止

Tauri 2は本書で扱うフレームワークの中で、AIへの指示が最も難しい部類に入る。一見するとWebフロントエンドとRustバックエンドをつないだ単純な構造に見えるが、実際には2つのランタイム間の境界をどれだけ正確に守るかが、アプリの安定性と安全性を左右する。

Tauriは独立した2つのランタイムの上で動く。Rustがバックエンドプロセスを担当し、SvelteKitを動かすWebViewがフロントエンドを担当する。両者はメモリを共有せず、**IPCブリッジ(プロセス間通信の通り道)**を介してのみメッセージをやり取りする。データは境界を越えるたびにJSON形式へ変換されて渡される(直列化/逆直列化)。この境界は単なる区切りではなく、安全設計の核心であり、AIが最も多くミスをする地点でもある。AIはこの境界を無視して関数を直接呼ぼうとしたり、秘密の鍵をフロントエンド

に露出させたり、コンパイルは通るが負荷をかけると止まる非同期コードを書いたりする。

この章はTauri文法のまとめではなく、**境界の地図**に近い。どの仕事どちら側に属するかは、結局のところ開発者が判断するしかない。

Tauriに初めて触れるときは、「小さなWebアプリ + 安全なシステムバックエンド」として理解すると分かりやすい。画面はWeb技術で作り、ファイル・DB・プロセス・キーチェーンといった機密性の高い作業はRust命令で処理する。フロントエンドはリクエストを送るだけで、実際の権限はバックエンドが握る。

境界を分ける基準は一つだけだ。**ユーザー画面を変える仕事はフロントエンド、OSやデータに手を加える仕事はRust命令である**。ボタンの色を変えたり一覧をフィルタしたりする仕事はSvelteの中で完結させる。逆にファイルを保存したり、DuckDBを開いたり、APIキーをキーチェーンから読み出したりする仕事はRust命令で処理する。

レストランに例えると、フロントエンドは客が見るホールであり、Rustバックエンドは厨房と金庫だ。客はメニューを選び注文を出せるが、冷蔵庫を直接開けたり金庫からカードを取り出したりはできない。注文票は決まった窓口から厨房に入り、できあがった料理だけがホールに戻ってくる。Tauriの `invoke()` がまさにその注文票である。

この章の流れ

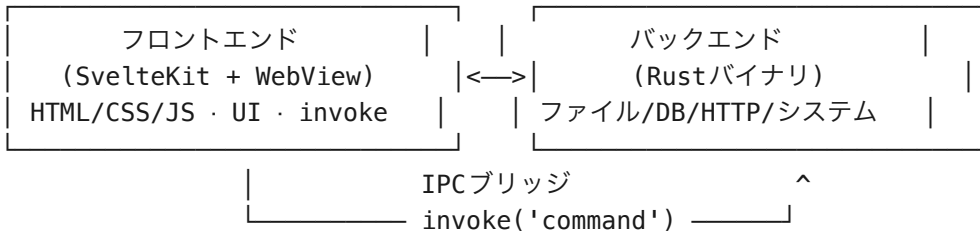
- ・ 5.1 アーキテクチャの概観—なぜ境界が重要か
- ・ 5.2 命令 (command) —フロントエンドがバックエンドを呼ぶ方法
- ・ 5.3 状態 (State) —命令の間でデータを共有する方法
- ・ 5.4 イベント—バックエンドからフロントエンドへデータを流す
- ・ 5.5 プラグイン—よく使う機能のセット
- ・ 5.6 権限と機能—Tauri 2の新しいセキュリティモデル
- ・ 5.7 フロントエンド—SvelteKit 2 + Svelte 5との接続
- ・ 5.8 プロジェクト構造
- ・ 5.9 メンタルモデルとプロンプティング戦略
- ・ 5.10 AIがよく犯すミス
- ・ 5.11 まとめとクイックリファレンス

学習目標

- ・ Tauri 2の2つのランタイム構造(Rustバックエンド + WebViewフロントエンド)を理解する。
- ・ `#[tauri::command]` でバックエンド関数をフロントエンドに公開するパターンを習得する。
- ・ `tauri::State<T>` で命令の間でデータを共有する。
- ・ イベントシステムでバックエンドの進捗をリアルタイムに画面へ流す。
- ・ Tauri 2の権限モデル(capabilities)を理解し、権限漏れエラーを素早く突き止める。
- ・ AIがTauri 2のコードでよく外すパターンを知る。

5.1 Tauri 2アーキテクチャの概観—明確な境界が重要な理由

2つのプロセスがどうつながっているか、まず図で見てみる。



フロントエンドで `invoke('get_user_data')` を呼ぶと、次の順で処理が走る。

1. `invoke()` が関数名と引数を JSON にまとめる。
2. IPC レイヤーがそのデータをバックエンドへ渡す。
3. Rust が `get_user_data` 命令を実行する。
4. Rust が結果を返す。
5. フロントエンドが応答を Promise として受け取る。

流れは HTTP API 呼び出しに似ているが、ネットワークは経由しない。同じアプリの中で WebView と Rust プロセスが内部の通り道でメッセージをやり取りする構造だ。

このアーキテクチャがAI開発と相性が良い理由は、**境界がきっぱり分かれている**からだ。「ユーザー設定を保存するバックエンド命令を書いて」と頼めば、AIが作るべきものがはっきりする。Electronはメイン/レンダラーの両方がJavaScriptなので、AIがフロントエンドとバックエンドを混ぜやすいが、Tauri 2はRustとJavaScriptという言語の違いが分離を強制する。

ネイティブUIではなくWebViewを使う理由

Tauri 2はシステムに組み込まれたWebViewを使う(macOSはWebKit、WindowsはWebView2、LinuxはWebKitGTK)。おかげでUIはHTML/CSS/JavaScriptで書けるし、プラットフォーム別のUIコードなしで1つのコードがすべてのプラットフォームで動く。

5.2 命令—#[tauri::command]パターン

命令はフロントエンドがバックエンド関数を呼ぶ方法だ。ファイルシステムへのアクセス、データベースクエリ、HTTPリクエストはすべて命令で処理する。

命令の名前を付ける原則

命令名をUIイベントにそのまま結びつけない。onSaveButtonClickは特定のボタンに紐づいた名前だが、save_user_settingsはボタンがなくなってもそのまま使える。**命令は業務の動作を表現すべきであり、画面のイベントを表現してはいけない。**

最小の例

このコードの役割: フロントエンドから名前を送ると、Rustが挨拶文字列を作って返す。Tauri命令に必要な3点(#[tauri::command]、invoke_handlerへの登録、フロントエンドからのinvoke()呼び出し)がすべて入っている。

```
// src-tauri/src/lib.rs
#[tauri::command]
```

```
pub fn greet(name: String) -> String {
    format!("こんにちは、{}!", name)
}

pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![greet])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

フロントエンドは同じ名前と呼ぶ。

```
// src/lib/api.ts
import { invoke } from '@tauri-apps/api/core';

export async function greet(name: string): Promise<string> {
    return await invoke('greet', { name });
}
```

確認ポイント: 3か所の名前がすべて一致している必要がある。Rust関数名、generate_handler![]への登録、フロントエンドのinvoke()の3つだ。どれか一つでもズレると、実行時にcommand not foundが出る。命令を追加するときに最もよく踏むバグがこの名前の不一致である。

失敗しうる命令

実際のアプリの命令はほとんどが非同期で、失敗の余地がある。ファイルが無い、権限が無い、ネットワークが切れる、といった具合だ。成功値だけを返さず、Result<T, String>で失敗を明示する。

```
#[tauri::command]
pub async fn read_config(path: String) -> Result<String,
String> {
    if !std::path::Path::new(&path).exists() {
```

```

        return Err(format!("ファイルが見つかりません: {}", path));
    }
    tokio::fs::read_to_string(&path).await
        .map_err(|e| format!("ファイル読み込み失敗: {}", e))
    }

```

read_configはResultの説明用である。製品では許可したapp-data配下でpathを正規化し、..やsymlinkの解決先が範囲外なら拒否する。frontendの任意の絶対pathを許してはいけない。

フロントエンドではこの失敗をtry/catchで受ける。

```

try {
    const text = await invoke<string>('read_config', { path });
} catch (error) {
    console.error('設定読み込み失敗:', error);
}

```

複数フィールドのデータ

設定、注文、セッションのように複数フィールドを持つデータは構造体として定義し、Serialize/Deserializeを付ける。Tauriが自動で変換してくれるので、フロントエンドとバックエンドが同じデータの形を共有できる。

```

use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
pub struct UserSettings {
    theme: String,
    auto_save: bool,
    recent_files: Vec<String>,
}

#[tauri::command]
pub async fn save_user_settings(settings: UserSettings) ->
Result<(), String> {

```

```
// ファイルやDBに保存
Ok(())
}
```

確認ポイント: 新しい命令(`save_user_settings`)も`invoke_handler![greet, save_user_settings]`のように登録リストへ追加する必要がある。

AIプロンプティングのコツ: 「ユーザー設定を管理する命令を作って」と頼むより、構造体定義(フィールド名と型)と一緒に渡すと結果がぐっと安定する。

5.3 状態管理—`tauri::State<T>`

複数の命令でデータを共有したいときは、Tauri組み込みの状態管理を使う。グローバル変数より整理しやすく、複数のスレッドから同時にアクセスしても安全だ。

状態(State)はアプリ内の共用の保管庫だと考えればよい。複数の命令が同じ設定、DB接続、キャッシュを使う必要があるとき、毎回作り直さずに保管庫から取り出して使う。ただし、同時に複数の命令が手を伸ばす可能性があるので、鍵の仕組みが必要だ。

基本パターン: `RwLock`

このコードの役割: 命令たちが共有するキャッシュをアプリ起動時に1回だけ登録し、各命令は`State<'_, AppState>`でその値を取り出して使う。

```
use std::sync::RwLock;
use std::collections::HashMap;
use tauri::State;

pub struct AppState {
    pub cache: RwLock<HashMap<String, String>>,
}

#[tauri::command]
pub async fn set_cache_value(
```

```

        key: String, value: String, state: State<'_, AppState>
    ) -> Result<(), String> {
        let mut cache = state.cache.write()
            .map_err(|e| format!("Lock エラー: {}", e))?;
        cache.insert(key, value);
        Ok(())
    }

// アプリ起動時に状態を登録
fn run() {
    tauri::Builder::default()
        .manage(AppState { cache: RwLock::new(HashMap::new()) })
        .invoke_handler(tauri::generate_handler!
            [set_cache_value])
        .run(tauri::generate_context!())
        .expect("error");
}

```

`RwLock`はデータを安全に共有するための鍵だ。書くときは`write()`で1か所だけ入れ、読むときは`read()`で複数の場所が同時に入れる。

ロックは型名ではなく臨界区間で選ぶ

`std::sync::RwLock`と`tokio::sync::Mutex`のどちらかが常に正しいわけではない。短い同期キャッシュ操作には標準ロックが簡単だが、非同期処理を待つ間も排他所有が必要ならTokioロックが適する場合がある。不要なロックを`.await`の先へ持ち越さない。

次は共有状態をロックしたままネットワークを待ち、ほかの命令を待たせる例だ。

```

// Holds shared state while waiting for network I/O.
use tokio::sync::Mutex;
pub struct BadState { data: Mutex<Vec<String>> }

#[tauri::command]
pub async fn bad_command(state: State<'_, BadState>) ->
Result<(), String> {

```

```

    let mut data = state.data.lock().await; // asynchronous
    lock
    data.push("item".to_string());
    send_to_server(&data).await?;           // still holds the
    lock
    Ok(())
}

```

短時間で更新し、snapshotをコピーして先に解放する。send_to_serverはアプリ固有のI/O関数を表す。

```

// Release the lock before awaiting I/O.
use std::sync::RwLock;
pub struct GoodState { data: RwLock<Vec<String>> }

#[tauri::command]
pub async fn good_command(state: State<'_, GoodState>) ->
Result<(), String> {
    let snapshot = {
        let mut data = state.data.write()
            .map_err(|e| format!("Lock poisoned: {}", e))?;
        data.push("item".to_string());
        data.clone()
    }; // lock released here
    send_to_server(&snapshot).await?;
    Ok(())
}

```

読み取りが多いならRwLock、排他アクセスならMutexが自然だ。遅い同期DB操作はworker threadやspawn_blockingへ分離し、非同期executorを止めない。

初期化コストの大きいリソース

DuckDB接続のように作るのに時間がかかるリソースは、アプリ全体で1回だけ作りたい。こうしたケースは第7章でDuckDB例とともに改めて扱う。今は「1回だけ作って複数の命令が共有するパターンが別にある」程度に覚えておけばよい。

5.4 イベント—emitとlisten

命令はフロントエンドがバックエンドに問いかける方式だ。ところがバックエンドの方から先にフロントエンドへ知らせたい場合もある。AI応答のストリーミングや、長時間動くプロセスの進捗のような場合だ。フロントエンドが「終わった?」と問い続ける代わりに、バックエンドがイベントを押し出す(push)。

バックエンドからフロントエンドへ送る

このコードの役割: 10秒間、毎秒進捗をフロントエンドへ送り、最後に完了イベントを送る。

```
use tauri::{AppHandle, Emitter}; // Emitter provides emit()

#[tauri::command]
pub async fn start_long_process(app: AppHandle) -> Result<(),
String> {
    tokio::spawn(async move {
        for i in 0..10 {
            tokio::time::sleep(tokio::time::Duration::
from_secs(1)).await;
            let _ = app.emit("process_progress", (i + 1) as f64
/ 10.0);
        }
        let _ = app.emit("process_complete", ());
    });
    Ok(())
}
```

失敗したら見るポイント: use tauri::Managerを忘れるとapp.emit()がコンパイルできない。

フロントエンドはlistenで受ける。リスナーはもう必要ないときに解除しないとメモリリークになる。

```
import { listen } from '@tauri-apps/api/event';

$effect(() => {
  const unlistenPromise = listen<number>('process_progress',
    (event) => {
      console.log('진행:', event.payload);
    });

  // Clean up even if registration completes after disposal.
  return () => { void unlistenPromise.then((unlisten) =>
    unlisten()); };
});
```

ストリーミングのパターン

AI応答を画面に流すときも同じパターンだ。モデルがトークンを1つ生成するたびにイベントで流し、終わったら完了イベントを送る。

```
#[tauri::command]
pub async fn stream_ai_response(prompt: String, app: AppHandle)
-> Result<(), String> {
  let mut stream = client.chat().create_stream(prompt).await?;
  while let Some(chunk) = stream.next().await {
    if let Some(content) = chunk?.choices[0].delta.content {
      let _ = app.emit("ai_chunk", content);
    }
  }
  let _ = app.emit("ai_complete", ());
  Ok(())
}
```

このパターンは第6章(タイマー通知)、第8章(AI応答ストリーミング)、第10章(プロセス出力キャプチャ)で繰り返し登場する。

5.5 プラグイン—tauri-plugin-*エコシステム

Tauri 2はよく使う機能をプラグインとして提供する。各プラグインはRust命令と(必要なら)フロントエンドJavaScript APIをセットでまとめてくれる。

AIアプリでよく使うプラグイン

プラグイン	用途
tauri-plugin-shell	プロセス生成、Pythonスクリプト実行
tauri-plugin-store	ユーザー設定などの保存
tauri-plugin-dialog	ファイル/フォルダ選択ダイアログ
tauri-plugin-fs	ファイルの読み書き
tauri-plugin-http	RustからのHTTPリクエスト
tauri-plugin-websocket	WebSocket接続

プラグイン追加は2ステップだ。Cargo.tomlに依存を加え、Rustコードで登録する。

```
fn run() {  
    tauri::Builder::default()  
        .plugin(tauri_plugin_store::Builder::new().build())  
        .plugin(tauri_plugin_dialog::init())  
        // ...  
}
```

セキュリティ注意: *tauri-plugin-store*はデータをディスクにJSONとして保存する。APIキーのように露出してはいけない値は別途暗号化するか、*tauri-plugin-stronghold*でシステムのキーチェーンに保存する。

5.6 権限とcapability —Tauri 2の境界

capabilityはWebViewが呼べるTauri core・plugin APIを制限し、窓ごとに権限やpath・URL scopeを分ける。generate_handler!登録の独自Rust命令は既定でアプリの窓から呼べるが、その内部コードをcapabilityが自動検査するわけではない。独自命令の不安全な実装を代わりに守る装置ではない。

```
{
  "$schema": "../gen/schemas/desktop-schema.json",
  "identifier": "main",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "fs:allow-read-text-file",
    "fs:allow-write-text-file",
    "dialog:allow-open",
    "dialog:allow-save",
    "http:default",
    "store:default"
  ]
}
```

default権限の内容はpluginで異なる。名前から広さを推測せず、生成schemaや公式表を確認する。fs:allow-read-text-fileのように必要な動作とpath・URL scopeを選ぶ。

plugin権限と独自命令を区別する

次はfilesystem pluginを呼ぶfrontend codeなので、権限とpath scopeが必要だ。

```
import { readTextFile } from '@tauri-apps/plugin-fs';
import { BaseDirectory } from '@tauri-apps/api/path';

const text = await readTextFile('notes/today.md', {
  baseDir: BaseDirectory.AppData
});
```

権限がなければPromiseが拒否される。plugin登録、識別子、許可pathを合わせて確認する。error文は版によって変わるため、そのまま識別子へ転記せずschemaから選ぶ。同じ処理を独自Rust命令で行うならpath検証、許可リスト、認証を内部で実装する。

5.7 フロントエンド—SvelteKit 2 + Svelte 5 + Tauri

TauriアプリのフロントエンドはWebView内で動くSvelteKitアプリケーションだ。Tauriブリッジパッケージを追加するだけでシステムAPIにアクセスできる。

```
pnpm add @tauri-apps/api
```

核となるインポートは3つだ。

```
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import { getCurrentWindow } from '@tauri-apps/api/window';
```

Svelte 5 runeとTauriイベントの相性

このコードの役割: バックエンドが送る進捗を\$stateで受け取り、そのまま画面に反映する。\$derivedで表示テキストを自動的に計算する。

```
<script lang="ts">
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';

let progress = $state(0);
let isProcessing = $state(false);

async function startProcess() {
  isProcessing = true;
```

```

    try {
      await invoke('start_long_process');
    } catch (error) {
      isProcessing = false;
      console.error(error);
    }
  }
}

let progressPercent = $derived(Math.round(progress * 100));

$effect(() => {
  const subscriptions = [
    listen<number>('process_progress', (e) => { progress =
e.payload; }),
    listen('process_complete', () => { isProcessing = false;
}),
  ];
  return () => {
    for (const pending of subscriptions) {
      void pending.then((unlisten) =>
unlisten()).catch(console.error);
    }
  };
});
</script>

<div>{isProcessing ? `처리 중... ${progressPercent}%` : '준비'}</
div>

```

APIラッパーで命令呼び出しをまとめる

`invoke()`をコンポーネントの中で直接呼ぶと、命令名のタイプがあちこちに散らばる。すべての命令呼び出しを`src/lib/api/`ディレクトリの関数で包めば変更に強くなる。

```

// src/lib/api/files.ts
import { invoke } from '@tauri-apps/api/core';

```

```
export const FilesAPI = {
  readDirectory: (path: string) =>
  invoke<FileItem[]>('read_directory', { path }),
  readFile: (path: string) => invoke<string>('read_file', {
    path }),
  writeFile: (path: string, contents: string) =>
    invoke<void>('write_file', { path, contents }),
};
```

コンポーネントはFilesAPI.readFile(...)を呼ぶだけでよい。命令名が変わっても1か所直すだけで済む。

5.8 プロジェクト構造—リファレンスレイアウト

本書の6つのプロジェクトすべてで同じ構造を使う。

```
my-tauri-app/
├── src-tauri/                                # Rustバックエンド
│   ├── src/
│   │   ├── lib.rs                          # エントリポイント
│   │   ├── commands/                      # files.rs, ai.rs, database.rs ...
│   │   ├── state/                         # app_state.rs
│   │   └── utils/
│   ├── capabilities/main.json
│   ├── Cargo.toml
│   └── tauri.conf.json
├── src/                                      # SvelteKitフロントエンド
│   ├── lib/
│   │   ├── api/                          # files.ts, ai.ts ...
│   │   ├── stores/                       # app.svelte.ts, files.svelte.ts
│   │   └── components/
│   ├── routes/
│   └── app.html
├── package.json
└── vite.config.ts
```

Claude Codeに作業を頼むときは、ディレクトリも一緒に指定する。

「ユーザー設定を保存する命令を`src-tauri/src/commands/database.rs`に追加して、`src/lib/api/index.ts`にラッパー関数も一緒に作って。」

そうすればAIがコードをどこに置くか迷わない。

5.9 AI開発のためのTauri 2メンタルモデル

Tauri 2は役割の境界が強制される。だからAIと組んで開発するときに、むしろ有利だ。構造そのものが次の規則を強制する。

- ・ フロントエンドはファイルシステムに直接アクセスできない -> 命令を通る
- ・ バックエンドはDOMを直接いじれない -> イベントで知らせる
- ・ データは境界を越える際に必ず変換される
- ・ 権限はコードの中にあらかじめ宣言されていなければならない

規則がコードに溶け込んでいるので、間違っって作られたコードは最初から動かない。

プロンプティング戦略

同じ機能でも、プロンプトの書き方で結果が変わる。

- ☐ 「ファイルを保存する機能を追加して」
- ☒ 「`src-tauri/src/commands/files.rs`に、テキスト内容をファイルパスに保存するRust命令を追加して。不正なパスと権限問題のエラー処理も入れて。次に、`src/lib/api/files.ts`にこの命令を呼ぶTypeScriptラッパーも作って。」

どちら側でどの仕事をするかを明示するほど、構造に沿ったコードが出てくる。

5.10 検証チェックリスト

- ・ `save_user_settings`を含む新命令を`generate_handler!`へ登録する。
- ・ frontend pluginの権限・path・URL scopeを確認し、独自Rust命令は入力を自ら検証する。
- ・ lockを短く保ち、アクセス要件から同期・非同期を選ぶ。

- ・ 処理開始前にlistenerを登録し、登録が遅れても破棄時に解除する。
- ・ RustとTypeScriptのpayload型を揃える。この例のprocess_progressは[0,1]の数値、process_completeは処理中flagを解除する。
- ・ 高頻度streamは20～50msごとにまとめるかTauri Channelを使う。
- ・ SvelteKit 2とSvelte 5を使い、static adapterとnative専用routeのSSR無効化を確認する。
- ・ 命令失敗、素早いmount/unmount、繰り返し実行を試す。

任意の教材archiveにはtauri/ch05-local-notes/src/App.svelteがある。Svelte 5 + Vite + Tauri 2の独立実習で、MandaiやSvelteKit appの全体ではない。本文が命令・app専用file・event解除・error処理を説明し、READMEに導入と完成条件がある。

第6章 Mandai の設計

—AI にどれくらい大きな単位で任せられるか

Part II —Mandai (6～9章) ここからは実際にアプリを作る。1～5章が原則と道具の話だったとすれば、Part II はそれらが一つのプロジェクト全体でどう機能するかを見せる章だ。設計 (6章) -> DB (7章) -> AI API (8章) -> 失敗と教訓(9章)の順に、ループが実際にどう回るかをたどっていく。最初の通読では Mandai のすべての機能を覚える必要はない。「AI にアプリ全体を一度に任せず、画面・データ・コマンドに分けて任せる」という構造さえ掴めればよい。

プロジェクトを始めるときに最初に答えるべき問いは「どんなプロンプトを使うべきか」ではない。その手前にある問いがある。**自分は何を作るのか。**

この問いに答える前に AI に頼むとどうなるか。AI は方向を持たないまま、素早く何かを作り出してくる。ファイル名はそれっぽく、フォルダ構造もあるように見えるが、各部分が互いに噛み合わない。

逆に、大きすぎる単位で任せるとどうなるか。アプリ全体を一度に渡せば AI は消化しきれない。かといって細かく分けすぎると、今度は文脈が途切れる。AI は前に何を作ったか知らないまま次の部分を書く。

Mandai はこのバランスを探るのにちょうどよいプロジェクトだった。Mandal-Art、GTD、Pomodoro、AI チャットはそれぞれ独立した機能のように見える。だが実際の製品の中では、一つの目標管理の流れとしてつながっていなければならなかった。この章は Mandai を設計しながら、AI が一度に受け取れる大きさを探っていく過程の記録である。

この章にはコードがほとんど登場しない。代わりに Mandai の骨格が固まっていく順番をそのままたどる。**自分が送ったもの、AI が返したもの、そして その返答が設計をどう変えたか**を一緒に見ていく。ここで得てほしいのは完成したフォルダ構造ではない。AI と設計をやり取りするときどんな問いが必要か、その感覚だ。

料理にたとえてみる。AI に「夕飯を一式作って」と頼めば、メニュー、材料、調理順序を AI が勝手に決める。逆に「玉ねぎを 1cm に切って」だけを繰り返せば、全体の料理像は見えない。良い単位は「カレーを作るから、まず材料リストと調理手順を提案して」くらいだ。全体の方向は決まっていて、一度に処理する範囲は小さい。Mandai の設計も同じやり方で進めた。

この章の流れ

Mandai というアプリの形をまず固め、AI に任せられる作業単位を少しずつ絞っていく。

- ・ Mandai は最初どんなアプリだったか
- ・ 最初の失敗—「全部を一度に作って」
- ・ サイズ調整—AI が一度に処理できる単位
- ・ AI の問い返しを設計ツールとして使う
- ・ 三層に分ける: 画面・データ・コマンド
- ・ AI チャUNKサイズの法則

- ・ビルドの途中で計画が変わるとき

学習目標

- ・ AI にどれくらい大きな作業を一度に任せられるか、その限界がどこにあるかを把握する。
- ・ AI が問い返してくる質問を、設計判断のための道具として活用する方法を身につける。
- ・ アプリを画面・データ・コマンドの三層に分ける戦略を学ぶ。
- ・ Mandai の初期設計で AI と一緒に構造を組み立てるパターンを実際にたどる。

6.1 Mandai は最初どんなアプリだったか

Mandai は四つの機能の一つにまとめた生産性ツールだ。並べてみるとよくあるアプリに聞こえるかもしれない。だが要は、この四つを別々に使うのではなく、一つの流れに繋ぐことにある。目標、今日のタスク、集中時間、AI との会話がそれぞれ分断されず、互いに繋がる。

- ・ **Mandal-Art (9×9 の目標グリッド)** : 中央の目標が一つ、サブ目標が八つ。各サブ目標はさらに 9 マスのグリッドへ展開される。 $9 \times 9 =$ 計 81 マス。
- ・ **GTD**: 今日のタスクリスト。各タスクは 81 マスのどれかにバインドされる。
- ・ **Pomodoro**: 25 分集中、5 分休憩。完了したセッションを DuckDB に記録する。
- ・ **AI API 統合**: 一つのチャット画面から OpenAI、Claude、Gemini を切り替えて使う。ストリーミングに対応する。

四つを一つに束ねた理由は単純だ。**Pomodoro のセッションは、自分の 81 マスのどれかに繋がっているときだけ意味を持つ**。集中時間を記録するだけでは足りない。どの目標のどの領域に時間を使ったのかも残らなければならない。そうして初めて、自分が実際にどこにエネルギーを注いでいるかを後から見返せる。

既存のアプリにはこの繋がりがなかった。Todoist には目標ツリーがなく、Forest にはタスクがなく、Notion には Pomodoro が標準の流れとして組み込まれていない。

だから自分で作った。この決定がこの後のすべての設計の基準になる。単なるタイマーアプリではなく、目標構造と時間記録を束ねるアプリであることを、AI に何度も伝え続ける必要があった。

6.2 最初の失敗—「全部を一度に作って」

最初のプロンプトはこうだった。

「Tauri 2 + SvelteKit 2 + Svelte 5 で、Mandal-Art、GTD、Pomodoro、AI チャットを組み合わせたアプリを作って。DuckDB を使って、AI プロバイダを 4 つ繋いで、ストリーミング対応。」

Claude の応答は予想通りだった。30 行のフォルダ構造が出てきて、300 行のコードブロックが `lib.rs` にまるごと入ってきた。半分はコンパイルが通らず、残りの半分は自分が望んだものではなかった。Mandal-Art のセル ID は適当に決められ、Pomodoro のテーブルはタスクテーブルと噛み合っていなかった。

その出力をそのまま使うことはできなかった。だが完全に無駄でもなかった。自分の設計にどんな抜けがあるかを暴いてくれたからだ。

AI が出してきた結果には、核となる問題が三つあった。コードそのものよりも、**どの判断が誤って下されたか** を見るのが大事だ。

問題 1. セル ID に UUID を使った

AI はセルごとにランダムな長い識別子 (UUID) を付けた。そして「このセルはどの目標に属するか」を書く欄を空白でも許される作りをしていた。二つとも間違っていた。

- ・ **UUID は位置情報を失う。**「中央の三番目の領域、五番目のマス」という意味が消える。
- ・ **親目標が空でも通るので、どこにも属さないセルが生まれてしまう。** $9 \times 9 = 81$ マスという構造そのものがデータとして強制されない。

自分が望んだのは、人間が読める ID と、必ずどこかの目標に属するセルだった。

問題 2. Pomodoro セッションがタスクと繋がっていなかった

AI が作ったセッションテーブルには、「どのタスクのセッションか」を指す繋がりが欠けていた。代わりに、自由に書ける文字列のメモ欄しかなかった。

この一つの欠落でアプリ全体の意図が壊れる。「Pomodoro セッションは 81 マスのどれかに繋がっているときだけ意味を持つ」という核の判断が消えてしまったのだ。セッションの記録は残るが、どの目標のどのマスに使った時間か、後から繋ぎ直せない。

問題 3. フォルダ構造は立派だが中身が空だった

`models.rs`、`migrations.rs`、`connection.rs` といったファイルが自動で作られた。名前は立派でも、マイグレーションファイルは「あとで埋める」というコメントだらけで、DB パスはハードコードだった。見栄えのする骨格の中に中身がなかった。

要求が雑だった分、AI はその隙を自分で埋めた。問題は、AI が埋めた値が製品の核となる意図と合わなかったことだ。セル ID 体系とセッションとタスクの繋ぎは、アプリの中心構造である。最初に間違えれば、画面を全部作った後でも何度も最初に戻る羽目になる。

注意: 要求が曖昧だと AI は隙を自分の判断で埋めて、それらしい答えを作る。AI は何が大事かを知らない。最初の応答には少なくとも三つの誤った判断が入っていると思っておくとよい。

教訓は単純だ。**AI に「アプリ全体」を一度に任せるな。** AI に渡す前に、まず自分の手で作業を細かく分けなければならない。分けていない要求は AI に自由を与えるのではない。製品の中核の判断を AI に推測させているだけだ。

6.3 サイズ調整—AI が一度に処理できる単位

二度目のセッションはこう始めた。

「アプリの計画は終わった。これからは正確に一つずつ聞いていく。まず全体構造を聞いてから、聞きたいことを最大 5 つ投げて。コードは書くな。」

この一行で Claude の応答の仕方が変わった。Claude はコードの代わりに質問を出してきた。質問は五つだった。

1. Mandal-Art セル ID の体系は？(例: 1-3-5 vs center.sub3.cell5 vs UUID)
2. Pomodoro セッションが終わったら、どのタスクに紐付けるか—開始時に固定か、終了時に選ぶのか？
3. AI チャット履歴はタスクごとか、セルごとか、それともアプリ全体か？
4. オフラインをサポートするか？するなら AI 呼び出しはどう動かすか？
5. 設定 (API キーなど) はどこに保存するか—keyring、普通のファイル、それとも DuckDB？

五つのうち三つは、まだ決めていない事柄だった。 AI が自分の隙を先に見つけてきたわけだ。最初のセッションで AI が勝手に埋めてしまっていたのが、まさにこの部分だった。

Claude が自分から質問してこないときは、こちらから誘導する。「コードは書くな。この設計で、まだ決まっていないところを 3 つ探して。」この一文で、Claude はコードを書く代わりに設計の隙を探し始める。

セル ID 体系—二度の問い返しで確定

最初に詰めたのはセル ID 体系だった。自分はこう答えた。「セル ID は {center_idx}-{sub_idx}-{cell_idx} 形式に従う。文字列。9×9 = 81 個。」

Claude がさらに問い返した。「中央そのもの (center) はどう扱う？0-0-0 のように仮想セルとして扱うのか、それとも別テーブルか？」

見落としていた点だった。中央の目標はセルではないのに、無意識にセルのように扱っていた。「中央は別の goal テーブルにする。セルは goal_id FK で参照する。」

Claude がまた問い返した。「目標が別テーブルなら、アクティブな目標が複数あるときは？セルは正確に一つの目標に属するのか、それとも複数の目標にまたがって使えるのか？」

一度答えると新しい質問がまた出てくる。これが繰り返されるパターンだ。こちらが答えるたびに新しい質問が 2〜3 個出る。AI が設計の隙を探し続ける。

自分の答え: 「一度にアクティブな目標は一つだけ。セルは goal_id FK で正確に一つの目標に属する。」

二往復でセルのスキーマが確定した。最初のセッションで Claude が一人で書いた 300 行よりずっと実りがあった。

Pomodoro セッション—分析シナリオまで追いかける

次に、Pomodoro セッションがどのタスクに紐付くかを決めた。自分はこう答えた。「Pomodoro は開始時にタスクを選ぶ。」

Claude が問い返した。「タスクなしで 25 分セッションを回すケースは—例えばメール処理など?」

もっともな指摘だった。「ではタスクは任意。task_id は nullable で、note カラムが代わりに入る。」

Claude がさらに突いてきた。

「task_id が nullable なら、分析はどう扱う?『今週、メールに 3 時間』と『今週、未追跡の作業に 3 時間』を見たいか?」

分析のユースケースは考えていなかった。

自分の答え: 「そう、notes は検索できる必要がある。category enum を追加: 'task', 'email', 'meeting', 'other'。」

この一つの決定が、後のセッション履歴画面の全体構造を決めた。AI が問い返したおかげで、後でやるべき判断を今のうちに前倒しできた。

AI チャット—選択肢を提示してもらう

三つ目の論点は AI チャットの範囲だった。最初は「AI 履歴はアプリ全体でフラットに」と答えた。

Claude はそのまま受け取らず、シナリオを投げ返してきた。

「セル 2-3-4 で『この問題をどう解こうか?』と聞いたとする。後でセル 5-1-2 にいるときに、履歴から同じ質問を見る。文脈が合わなくなっている。

選択肢: A) セルごとの履歴 (セルあたり 1 スレッド = 81 スレッド) B) 目標ごとの履歴 (目標あたり 1 スレッド) C) コンテキストタグ付きのアプリ全体履歴

(#cell:2-3-4、#task:write-tests) D) ハイブリッド: アプリ全体だが、コンテキストフィルター付き」

Claude が自分の思いつかなかった選択肢を提示した。答えは C だった。フラットに保存し、タグを豊富に付け、ビューでフィルタする。

このループ一つでチャット用データベーススキーマ全体が定義された。テーブルは三つ: messages、contexts、そして結合テーブルの message_contexts。

注意: 最初に思いつくデータ構造はたいてい間違っている。AI は抜けていた利用シナリオを見つける。「ではどうするか?」を最低 3 回掘り下げてからスキーマを確定させる。

6.4 AI の問い返しを設計ツールとして使う

これまでに見た三つのループには共通点がある。こちらが答えると Claude がまた問い返し、その質問が設計の空白を浮かび上がらせる。この流れは意図的に作り出せる。

自分が使っているテンプレートはこうだ。

「この設計の穴を監査して。ルールに従うこと: 1. コードは書くな。2. 自分が判断を先送りしている 3 箇所を見つけて。3. それぞれに 2~3 個の代替案を提示して。」

このテンプレートを使うと、Claude が代替案の生成器の役割を担うようになる。自分が知らなかった選択肢を並べてもらい、その中から一つを選び、なぜ選んだかをノートに残す。そのノートがアーキテクチャドキュメントになる。

実際にどう動くか見てみよう。Claude にこの設計を渡した。

「Mandai は 5 つの画面を持つ: Today、Archive、Goals、Settings、Chat。Today 画面はアクティブな目標の 81 マスを 9x9 グリッドで表示する。ユーザーがマスをクリックするとそのタスクが見え、タスクをクリックすると Pomodoro を開始する。」

監査の結果、決めていない箇所が 3 つ出てきた。

空白 1: ナビゲーション状態 - 案 A: ブラウザ風ルーティング (/today、/archive など) - 案 B: タブベースのナビゲーション (SvelteKit stores) - 案 C: モーダルオーバーレイ (メイン画面一つ、残りはポップアップ)

空白 2: セルの操作モデル - 案 A: シングルクリックで開く、ダブルクリックで編集 - 案 B: すべての操作を右クリックのコンテキストメニューに - 案 C: ホバーで操作ボタンを表示

空白 3: タイマーの状態保持 - 案 A: アプリを閉じたらタイマーをリセット（セッション限定） - 案 B: 再起動しても残るタイマー（DB に保存） - 案 C: アプリを閉じる前にタイマーが警告（ユーザー選択）

三つともまだ決めていない状態だった。設計らしく見えたものが、実は暗黙の仮定だらけの草稿だった。

自分の決定: 1. **ナビゲーション:** ブラウザ風ルーティング。戻るボタンが効き、ディープリンクができ、アプリらしさが出る。2. **セルの操作:** シングルクリックで開く、右クリックでコンテキストメニュー。初めての人にもすぐ見付き、パワーユーザーには効率的。3. **タイマー復元:** アプリを開き直したら、進行中だったセッションを継続できるようにする。ただし、セッション中にウィンドウを閉じようとしたら、その前に警告する。うっかりアプリを閉じたという理由だけで、25 分の集中記録が消えるのは避ける。この監査を Mandai に 10 回ほど繰り返した。結果として出てきた決定リストは、おおむね次のようになる。

- ・ セル ID: {center_idx}-{sub_idx}-{cell_idx} 文字列
- ・ goal テーブルは cell テーブルとは別
- ・ Pomodoro task_id は nullable、category enum を含む
- ・ AI 履歴はコンテキストタグ付きのアプリ全体（#cell、#task、#goal）
- ・ ナビゲーション: SvelteKit ルーティング（/today、/goals など）
- ・ セルクリック: シングルで開く、右クリックでコンテキストメニュー
- ・ タイマー復元: DB ベース、アプリを開き直したら進行中だったセッションを継続できる
- ・ オフライン: AI 呼び出しが失敗したらローカル下書きに保存、再接続時に再試行を提示
- ・ API キー: OS のキーリング（tauri-plugin-store ではない）
- ・ マイグレーション: DuckDB の _migrations テーブル、アプリ起動時に確認

このリストを **Cursor の .cursorrules に入れておいた**。それ以降、Claude はこの決定を破るコードを書かない。

監査一回あたり、主要な設計領域につきおよそ 10 分かかる。だが後の数時間を節約してくれる。セッション 2 で見つかった設計の穴が、セッション 20 ではバグにならない。

6.5 三層に分ける: 画面・データ・コマンド

AI に機能を頼むときには三層テンプレートを使う。

- 1. **画面** —ユーザーが見てクリックするもの。
- 2. **データ** —DB テーブルと TypeScript の型。
- 3. **コマンド** —画面からデータへ繋ぐ Rust 関数 (`#[tauri::command]`)。

三つを一度に頼むと AI が中心を見失う。だから順番が大事だ。**先に画面を確定させ、その画面に必要なデータを抜き出し、最後にそのデータを扱うコマンドを並べる。**

Today 画面でこのパターンがどう機能したかを見ていく。

層 1: 画面の定義

レイアウトはコードではなく ASCII モックで描いた。図を描くツールがなくても文字で表現でき、AI が混乱しない。以下は Claude にそのまま渡したモックだ。

```
+-----+
| MANDAI                                     [タイマー] 18:42 |
+-----+
| 9x9 グリッド | 選択されたセルのタスク |
|               |                       |
| [完了][待機][進行] | [ ] タスク 1      0/3 |
| [待機][完了][待機] | [ ] タスク 2      2/3 |
|               | [完了] タスク 3    3/3 |
|               | [+ 新しいタスク] |
+-----+
```

操作ルールはテキストで合わせて送る。

- ・ マスをクリック -> 下にタスクを表示
- ・ タスクをクリック -> Pomodoro 開始
- ・ セッション中は右上にタイマー表示

- ・ プログレスバーで各マスのタスク完了状態を表示

このモックを見た Claude が二つ確認してきた。

「1. 選択されたマスがないときは何を表示する—81 個全タスクか、空画面か？2. タスクのプログレスバーはいつ更新する—セッション中にリアルタイムか、セッション終了後か？」

決めていなかった点だ。こう答えた。「選択されたマスなし = 空のタスクリスト、『マスをクリックしてタスクを表示してください』というヒントを出す。プログレスバーはセッション完了後にだけ更新する。」

層 2: データ要件

画面が確定したところで、Claude が必要なデータを抜き出した。

「この画面に必要なもの: - アクティブな目標: *goal_id*、*title* - その目標の 81 セル全部: *cell_id*、*title*、*task_progress* (完了 / 全体) - 選択されたセルのタスク: *task_id*、*title*、*completed*、*cell_id* - アクティブセッション: *session_id*、*task_id*、*started_at*、*target_minutes* - セッションカウント: 今日のカウント、今週のカウント」

一点だけ変えた。「*task_progress* は事前計算しない。DB は生のカウント *completed_tasks* と *total_tasks* だけを返す。比率はフロントエンドで計算する。」DB が表示形式まで決めると、画面を変えるたびにクエリも書き直すことになるからだ。

Claude が追って問い返した。「タスクカウントには完了したタスクも含むか、それともアクティブなものだけか？『今週』の定義は何か—月曜始まりか、ローリング 7 日間か？」

見落としていたエッジケースだった。「すべてのタスクをプログレスに含める、完了済みでも未完了でも。『今週』はユーザーのタイムゾーンの月曜 00:00 から日曜 23:59 まで。」

層 3: コマンドの列挙

画面とデータが決まると、Tauri コマンドの一覧も自然に整理された。画面に必要なデータの種類だけコマンドが生まれる。

Claude が提案したコマンドは六つだった。名前を見るだけで、何を取ってくる関数かが一行で分かる。

コマンド	戻り値	用途
get_active_goal	Goal	現在アクティブな目標を一つ
get_goal_cells(goal_id)	Vec<CellWithProgress>	81 マスのテキストと進捗カウント
get_cell_tasks(cell_id)	Vec<Task>	選択したセルのタスク
get_active_session	Option<PomodoroSession>	進行中のセッションがあれば
get_session_counts	SessionCounts	今日と今週のセッション数
start_pomodoro(task_id, minutes)	String	新しいセッションを開始

このうち get_session_counts だけはシグネチャを調整した。今日や今週といった固定の期間しか受け取れないと、後で「過去 30 日」のような範囲を追加しにくいからだ。期間を enum（複数の選択肢を一つにまとめた型）で受け取る形に変えた。

考え方は単純だ。呼び出す側がどの期間を求めるかを値で伝えれば、コマンドはその期間のセッション数を数えて返す。enum の中に Today、ThisWeek、Custom（開始と終了の日付を直接指定）の三つの選択肢を置く。**enum に新しい選択肢が増えてもコマンドのシグネチャはそのままにできる**。後で「過去 30 日」のような画面が増えても、コマンドには手を入れず enum に項目を追加するだけで済む。

コマンドの入力・出力・エラーを一組で定義する。create_goalならtitleを受け、id・title・created_atを返し、空の題名とDB障害を区別する。JSONの状態はrunning・paused・completedのような固定値にし、RustとTypeScriptで揃える。画面の翻訳文字列を状態値にすると言語変更で契約が壊れる。

これがパターンだ。**画面 -> データ -> コマンド**。各層は前の層から出てくる。AI は一度に一層ずつ扱うときに最もうまく動く。三層をまとめて渡すと結果が混ざる。

コマンド 6 個は、すなわち **Request-Build-Verify ループ 6 回** である。コマンド当たり最大 30 分。画面一つ = 半日。

注意: 層を行き来すると AI は混乱する。一つのプロンプトで画面レイアウトと DB スキーマを聞くと、半分は UI フレームワーク、半分は SQL の応答が返ってくる。会話当たり一層だけにする。

6.6 AI チャンクサイズの法則

この過程を整理して、三つの法則を立てた。

法則 1. 一度に画面一つ、またはコマンド一つ。 アプリ全体 ≫ 機能一つ ≫ 画面一つ ≫ コマンド一つ。右に行くほど AI に任せやすい。

法則 2. コードを頼む前に質問を引き出せ。 最初の 5~10 分は Claude にこちらへ質問させる時間に使う。すると設計で抜けていた部分が見えてくる。その時間が、後で倍のデバッグ時間として返ってくるのを防ぐ。

法則 3. 決めるのは自分、理由は `.cursorrules` に残す。 AI は代替案をうまく作るが、何を採るかは決められない。自分で選び、なぜ選んだかを文書に残す。そうすれば次のセッションで Claude が同じ問題で揺れない。

チャンクサイズは大きくなるほど失敗率が急速に上がる。6 つのプロジェクトを通じて得たおおまかな感覚はこうだ。

- ・ **アプリ全体 (およそ 1000 行、3~4 時間) :** ほぼ失敗する。試さない。
- ・ **機能単位 (例: 「Pomodoro 全部」、1~2 時間) :** 初期の探索程度にだけ使う。
- ・ **画面一つ (30~45 分) :** 標準的な作業単位。一番よく使う。
- ・ **コマンド一つ (5~15 分) :** 画面レベルで詰まったときに、もう一段分割する。
- ・ **関数一つ (2~5 分) :** 特定のロジックをデバッグするときだけ。

判断に迷ったら、こう自問する。**このチャンクを一文で説明できるか。独立してテストできるか。コーヒ一杯のあとでも要件を覚えていられるか。判断を一つだけ下しているか。** どれか一つでも答えが「いいえ」ならチャンクが大きすぎる。

逆に、AI がチャンクごとに同じ質問を繰り返したり、二つのチャンクがロジックの大半を共有しているなら、チャンクが小さすぎる。一つにまとめて扱う。

最適なチャンクサイズは、**一度で動くコードが出てくる最大の単位** だ。ドメインやモデルによって変わるが、ほとんどの場合は思っているより小さい。

6.7 ビルドの途中で計画が変わるとき

6.2～6.6 節の計画の進め方は、設計が最初から最後まで安定していることを前提にしている。だが現実には、実際に作る中で新しい要件に気付くことが多い。AI も予想しなかったエッジケースを出してくる。最初の計画が崩れるのは避けられない。

Mandai を開発する間、これが二度起きた。開発の流れを止めずに設計をどう変えていったかを順に見せる。

ピボット 1: セル ID 衝突問題

ビルド 3 週目に、セル ID 体系で問題が出た。当初の計画ではセル ID は {center_idx}-{sub_idx}-{cell_idx} 形式で、各インデックスは 1～9 だった。(最終的な実装では 0～8 ベースに変わる—7 章のスキーマ参照)

問題: 目標切り替え機能を実装するうちに、ユーザーが 1-3-5 のようなセルを持つ目標を複数作れることに気付いた。セル ID の値が目標ごとに変わらないので、「目標 A のセル 1-3-5」のタスクと「目標 B のセル 1-3-5」のタスクが混ざる問題が出た。

スキーマ全体を書き直すこともできたが、そうすると DB コマンド 15 個を最初からやり直すことになる。代わりに AI に選択肢を聞いた。

「設計に問題がある。1-3-5 のようなセル ID が目標間で衝突する。既存のタスクとセッションのテーブルを変えずに、一意なセル参照が必要だ。選択肢は何か？」

Claude が提示した三つ:

- ・ A) **goal_id でプレフィックス:** goal_uuid_1-3-5 (一意だが見栄えが悪い)
- ・ B) **複合キー:** cell_id は '1-3-5' のまま保ち、常に (goal_id, cell_id) のペアで照会する
- ・ C) **階層型 ID:** {goal_idx}-{center_idx}-{sub_idx}-{cell_idx}

案 B はテーブル構造に手を入れず、クエリ条件を足すだけで済む。この決定を以後の作業が破られないように .cursorrules に 4 行を加えた。

- ・ セル ID は文字列形式 '1-3-5' をそのまま保つ。
- ・ セル ID は一つの目標の中でだけ一意である。
- ・ すべてのセル照会は goal_id 条件と一緒に書く。

- ・ インデックスは (goal_id, cell_id) の複合で張る。

この 4 行が、これから書かれるすべてのセル照会を縛る。新しい画面を作るときに Claude が WHERE cell_id = ? だけを書けばルール違反のコードになり、ルールファイルがそれを止める。

マイグレーションが二日ではなく 2 時間で終わった。変更の範囲を最小に抑えたおかげだ。

ピボット 2: リアルタイムタイマー問題

当初の計画では Pomodoro タイマーはブラウザの中だけで動く。アプリを閉じたらセッションも一緒に消える。

問題: ユーザーがセッションの途中で、うっかりアプリを閉じることが何度もあった。25 分のセッションが Cmd+W 一発で消える。生産性アプリとしては受け入れがたい挙動だ。

普通はアプリが終了しても残るバックグラウンドサービスで解決する。だが Tauri 2 はあらゆるプラットフォームでバックグラウンドサービスを同じようには扱えない。

ここで要件を「アプリを閉じてでもタイマーが動き続けなければならない」とだけ書くと、設計の向きがぼやける。より正確な表現は「アプリを開き直したときに、進行中だったセッションを復元する」に近い。要は、タイマーが独立して走り続けるのではなく、セッションの開始時刻を保存しておき、次に起動したときに残り時間を計算するということだ。

Claude は DB を単なる保存先ではなく **状態の復元手段** として使うパターンを提案した。流れはこうだ。

1. セッション開始時に started_at タイムスタンプ付きでセッションレコードを作る。
2. 30 秒ごとに last_heartbeat を更新する。
3. アプリ再起動時に ended_at IS NULL のセッションがあるか確認する。
4. あればダイアログを出す: 「進行中のセッションがありました。続ける (残り X 分) / 完了として記録 / 削除」
5. 「続ける」を選んだら now - started_at で残り時間を計算してタイマーを再開する。

このパターンが動く理由は単純だ。毎秒の残り時間を保存する代わりに、**開始時刻を一度だけ** 保存する。アプリが閉じていた時間も `now - started_at` 一回の計算で埋まる。バックグラウンドサービスがなくても、セッションの連続性が保たれる。

実装は一晩で終わった。「進行中だったセッションを続けますか？」と尋ねる復元ダイアログはユーザーの反応がよかった。機能が一つ増えた以上に、ウィンドウをうっかり閉じて作業の流れが切れない、という信頼を与えてくれた。

ピボットが多すぎるとき

一つのプロジェクトでピボット二回は普通だ。五回ならば、最初の設計が根本的に間違っているサインだ。両者を区別する基準は次の通り。

正常なピボット（ビルドを続ける）： - 問題が一つの制約から来ている（ID 衝突、永続化要件、など） - 影響範囲が狭く、修正の方向がはっきりしている - AI が具体的な解決策を 2〜3 個提案できる - 修正が結果として設計全体を改善する

危ないピボット（計画をやり直す）： - 複数の問題が同時に絡んで出てくる - AI の応答に解決策より問い返しのほうが多い - 「簡単な」修正が DB テーブル 5 個以上に触れる - 同じ核の概念を繰り返し説明する羽目になる

危ないピボットの兆候が見えたら、**6.4 節（設計の監査）に戻る**。現状から、空白を探すテンプレートをもう一度走らせる。ピボットが繰り返されるということは、たいてい奥のアーキテクチャに問題があるという合図だ。

6.8 まとめ

この章で必ず覚えておくべきことは単純だ。**AI に一度に大きすぎる仕事を任せるな。コードを頼む前に質問を受け取れ。決めるのは自分、その理由は文書に残せ。**

要点: **AI を使うとピボットのコストは下がる。なくなるわけではない。**早めに、頻繁にピボットする—ただし、設計そのものが間違っているときには必ず気付かなければならない。

その身軽さは適切なチャンクサイズから生まれる。AI はどんなサイズのチャンクでもビルドできるが、チャンクサイズが結果物の品質を決める。大きすぎれば存在しない制約を作り出す。小さすぎれば文脈が消える。

学んだこと:

- ・ **AI が誤って推測したところに設計の隙が見える。**「全部作って」プロンプトで Claude が勝手に埋めた箇所は、まさにこちらが明示しなかった要件を指す。
- ・ **初期段階ではコードより質問のほうが価値がある。**「自分に 5 つ質問して、コードは書くな」というプロンプト一つで、Claude をコード生成器から設計の完成度チェッカーに変えられる。
- ・ **AI の問い返しがエッジケースを暴く。** AI が応答するたびに新しい質問が 2〜3 個生まれる。これはバグではなく、自分が明示していなかったシナリオを指してくれる合図だ。
- ・ **アーキテクチャ監査テンプレートが先送りしていた判断を炙り出す。**「自分が判断を先送りしているところを 3 つ探して」というプロンプトを使うと、AI が該当箇所を指して選択肢を並べてくれる。選ぶのは自分の役目だ。
- ・ **層を分けると AI の応答が散らからない。** 画面 -> データ -> コマンドを別々の会話に分けて、AI に一度に一つだけ集中させる。
- ・ **チャンクサイズの基準: 画面一つ = コマンド 4〜6 個 = 半日。** 勢いを保ちつつ方向も変えられる単位だ。
- ・ **小さなピボットは正常だ。** AI 支援開発では方向修正は失敗の合図ではなく、想定された流れだ。ただし同じ場所で小さなピボットが繰り返されるなら、表面の下に構造の問題があるという合図だ。
- ・ **判断を文書化すれば揺れない。** 設計の選択とその理由を `.cursorrules` に残す。AI はセッションをまたいでも一貫性を保ち、同じ議論を蒸し返さずに済む。

次のステップ: これで大きな機能を AI が消化できるチャンクへ分ける仕組みが整った。AI の問い返しを設計ツールとして活かす方法、勢いを失わずに計画を変える方法も身についた。7 章では実際の DuckDB スキーマ設計にこの過程を適用する—最初の完全な Request-Build-Verify ループだ。

第7章 DBスキーマー最初のRequest・Build・Verifyループ

データベース設計はAIと一緒に開発するとき、いちばん最初に行き詰まる地点だ。AIがSQLを書けないからではない。単純なCREATE TABLEなら、AIは最初の返答でそのまま作ってくれる。本当に難しい部分は別にある。最初のループで決めたスキーマ(テーブル構造)が、その後のすべての機能の土台になるという点だ。この段階でテーブル間の関係を間違えると、後で画面、コマンド処理コード、マイグレーション(既存データを新しい構造へ移す作業)、分析クエリまで一度に直すことになる。

Mandaiのスキーマは3回修正した末に落ち着いた。最初のバージョンが明らかに間違っていたわけではない。コンパイルも通ったし、実行もできた。だが機能を3つ追加した後ようやくどこが使いにくいのが見えてきた。この章で大事なのは最終的なスキ

ーマだけではなく、AIにどんな要求を投げ、レビューで何を発見したかという過程である。読者が自分のプロジェクトで同じループを直接適用できるようにするためだ。

なぜこの章が「ループの話」なのか

まずは3回のループがどこで終わり、どこでもう一周必要だったのかを一目で見ってから始めよう。本文が抽象的に感じたらこの表に戻ってほしい。

ラウンド	自分の要求	Claudeの成果物	検証で見つけた問題
1	7テーブルのスキーマ草案	基本的なCREATE文	セッション終了時刻、セルIDの検証、ソフト削除戦略が抜け
2	3つの穴を埋める + インデックス	制約と複合インデックスを追加	_migrationsに一意性保証なし
3	Rust製マイグレーションランナー	一発で動作	なし

スキーマ設計は2〜3回のループ、定型パターン(マイグレーションランナー、クエリヘルパー、型レイヤ)は1回のループで終わる—この区別がこの章のいちばんのメッセージだ。

どこまで自分で手を動かして写すか

初読はスキーマ決定と失敗処理を追う。migrationは版の確認、未適用SQLの実行、成功した版の記録の順である。JSONパラメーターはnull・数値・文字列・booleanを区別して未対応値を拒否する。例はこの契約を理解するためのもので、商用アプリ全体をコピーする手順ではない。

この章の流れ

- 1. なぜDuckDBなのか —分析クエリに強いデータベースを選んだ理由

2. **1回目のループ** — スキーマ草案を受け取り、抜けた決定を洗い出す
3. **2回目のループ** — セッション終了、セルIDの検証、ソフト削除(データを消さずに消した印だけ残す方式)を埋める
4. **3回目のループ** — マイグレーションシステムを一度に仕上げる
5. **検証** — AIが作ったテストで制約が実際に効くか確認する
6. **接続** — RustコマンドとTypeScript型レイヤでフロントとつなぐ
7. **ループパターン** — どの作業が1回で終わり、どれが2~3回必要かを分類する

学習目標

この章を終えたら次のことができる。

- ・ DuckDBがSQLiteと何が違うか、なぜAIプロンプトにその違いを明示すべきかを説明する。
- ・ スキーマ設計を一発で終わらせようとせず、抜けた決定を段階的に埋めていくループを適用する。
- ・ CHECK制約(誤った値が入らないように防ぐルール)と外部キー、インデックスをクエリパターンに合わせて設計する。
- ・ TauriコマンドとTypeScript型レイヤをつなぎ、型安全なDBアクセスを作る。

7.1 DuckDBを選んだ理由

まず見るべきもの—7テーブルの関係

カラム名より先に関係を見よう。Mandaiの中心となる流れは次のとおりだ。

goals(目標) — 1:N — cells(81マス) — 1:N — tasks(タスク) — 1:N — sessions(Pomodoro)

chats(AIチャット) —
 コンテキストとしてgoal/cellに接続 |
 settings(キー=値) — アプリ全体の設定
 |

```

    _migrations(バージョン追跡)
    |

```

目標(goals)1つに対して $9 \times 9 = 81$ 個のセル(cells)がぶら下がり、各セルに今日のタスク(tasks)が紐づき、タスクごとにPomodoroセッション(sessions)が積み上がる。AIチャット(chats)はそこにコンテキストとして付く。この関係図を頭に置いておけば、なぜ外部キーとインデックスが必要なのか、なぜ1つのテーブルだけを見て決められないのかが自然に理解できる。

なぜDuckDBか

最初はSQLiteでも十分に思える。単純なデータ保存だけならSQLiteの方が馴染みもあり資料も多い。ところがMandaiは時間が経つほど分析的な問いを投げ始める。「ここ3か月でいちばん集中したセルは?」「曜日ごとの集中時間の中央値は?」といった問いだ。データを保存するのではなく、分析する仕事だ。DuckDBはこの分析側に特化している。

DuckDBは列指向エンジンだ。一般的なDBが1行ずつデータを格納するのに対し、DuckDBは1列ずつ格納する。そのおかげで合計や平均のように、1列まるごとを舐める集計クエリが速い。SQL構文もSQLiteとそれほど変わらないので、新しく覚えることも少ない。

データベースの章が難しく感じる理由は別にある。テーブルをどう設計するかが、そのままアプリの方針になるからだ。たとえばtask_idを空でも許す値にするか、データを消すときに実際に削除するか印だけ残すか—こうした選択が画面、分析クエリ、マイグレーションにまで影響する。単なる好みの問題ではない。

DuckDB vs SQLite —AIがよく間違える部分

注意: ほとんどのAIモデルはSQLiteの構文をデフォルトで使う。DuckDB固有の機能を明示しないと、DuckDBの性能を活かせない平凡なSQLが返ってくる。

Claudeに「データベーススキーマを作って」と最初に依頼したとき、返ってきた答えはSQLiteに近かった。分析に有利な型や関数が抜けていた。

領域	DuckDB	SQLite
タイムスタンプ	TIMESTAMP	DATETIME
大きな整数	BIGINT	INTEGER
分析関数	percentile_cont(), stddev() が内蔵	ほとんど自前実装
配列	ネイティブサポート	なし

プロンプトに「DuckDBの構文を使い、SQLiteのパターンでは書かないこと」と一行加えるだけで、AIが選ぶ型やサンプルクエリが変わる。この差を体感することが、この章でもっとも実用的な収穫の1つだ。

7.2 1回目のループーリクエスト草案を書く

スキーマをClaudeに依頼する前に、TauriからのDuckDB接続をどうするか決める必要があった。コマンドごとに新しい接続を開くのか、アプリ全体で1つの接続を共有するのかで、エラー処理が変わってくるからだ。

src-tauri/Cargo.tomlに依存を追加する。bundledフィーチャを使うとDuckDBがアプリと一緒にコンパイルされ、ユーザー側に別途インストールしてもらう必要がなくなる。

[dependencies]

```
duckdb = { version = "1.5", features = ["bundled"] }
```

接続は一度開いて共有する

接続はアプリ全体で1つだけ作り、それを共有する。次のコードはアプリ起動時にDuckDB接続をただ1回だけ作り、グローバル状態として保持してからマイグレーションを実行する。

```
static DB: OnceLock<Mutex<Connection>> = OnceLock::new();

pub fn init_db() -> Result<(), String> {
    let conn = Connection::open_in_memory()
        .map_err(|e| format!("DB接続失敗: {}", e))?;
    DB.set(Mutex::new(conn)).map_err(|_| "DB already
initialized".to_string())?;
    run_migrations().map_err(|e| format!("マイグレーション失敗:
{}", e))?;
    Ok(())
}
```

ここでOnceLockは「アプリ全体でただ1回だけ初期化される値」を作るRustの型、Mutexは「同時にひとつだけアクセスを許す」ためのロックだ。接続をコマンドごとに開くと遅いし、同時に複数の接続が同じファイルに触ると管理が複雑になる。だからアプリ起動時に1回開き、以降はすべてのコマンドがその接続を少しの間だけ借りる形にする。

5章との違い: 5章ではRwLockを勧めていた。ここでMutexを使うのは、DuckDBの接続自体が同時アクセスを許さないからだ。一般的なデータ構造にはRwLock、同時に1つしかアクセスできない資源にはMutex、と覚えておけばよい。

注意: 実際の配布アプリではConnection::open_in_memory()を使ってはいけない。アプリを閉じるとデータがすべて消える。実ファイルのパスを指定し、初回実行でファイルがない場合の処理も入れる必要がある。

スキーマのリクエストを送る

接続方式を決めた後、Claudeにスキーマを頼んだ。このプロンプトではテーブル一覧を並べるだけでなく、**アプリがそのデータをどう使うか**まで一緒に書いた。テーブル名だけ放り投げると、AIはそれが単なる保存用なのか分析用なのかわからず、「標準的なCRUD」用のカラムだけを選んでしまう。使われ方の流れを書いておいてはじめて、外部キー、タイムスタンプ、インデックス候補まであらかじめ考慮してくれる。

DuckDBスキーマを書いて。Mandaiアプリ用。テーブルは7個。

- goals: 中央目標
- cells: 9x9 = 81セル、goalへのFK
- tasks: 今日のタスク、cellへのFK(任意)
- sessions: Pomodoroセッション、taskへのFK(任意)
- chats: AIチャットメッセージ
- settings: キー値
- _migrations: マイグレーション追跡

明示的な外部キー。DuckDB文法。CREATE TABLEのみ。INSERT例は不要。

コンテキスト: これは生産性アプリ。

目標は9x9セルグリッドを持つ(Mandalaチャートのように)。

ユーザーはセルにタスクを追加し、タスクに対してPomodoroセッションを実行し、進捗について

AIとチャットする。分析クエリは時間範囲別にセッションを集計し、集中時間比率を計算する必要がある。

草案レビュー—何が抜けていたか

Claudeの草案ではgoalsとcellsの2テーブルは外部キー、NOT NULL、デフォルト値まで自然に入っていた。良い部分だ。だがレビューしていくと、決定が抜けている箇所が3つ見えてきた。

1. **sessionsにstart_atはあるがend_atがない。** 総集中時間を計算する手段がなく、タイマーが途中で終わるとセッションが永遠に開いた状態で残る。
2. **cells.idに生成ルールがない。** 6章で{center_idx}-{sub_idx}-{cell_idx}形式(例: 0-1-4)と決めたのに、そのルールがスキーマに反映されていない。
3. **ソフト削除の方針に一貫性がない。** goalsにはdeleted_atがあるがtasksにはない。タスクを削除したら、紐づくセッションはどうなるのか?

3つとも6章で決めずに先送りしたものだ。自分が忘れていて、Claudeも素通りしていた。レビューする中で、見落とししていた決定が浮かび上がってくる。1回目のループの役割はまさにそれだ。

7.3 2回目のループー穴を埋める

6章で決めたルールの再掲: セルIDは`{center}-{sub}-{cell}`形式で、各インデックスは0~8の範囲。例: `0-1-4`は中央エリア0、その中のサブエリア1、その中のセル4を指す。このルールをDBの制約に落とし込むのがこの節のポイントだ。

2回目のループでは草案を一から作り直すことはしなかった。良かった部分は残し、抜けていた3つの決定だけを具体的に指摘してフィードバックした。

3点の修正が必要:

1. ``sessions``に``end_at`` `TIMESTAMP NULL`と``ended_reason`` `TEXT` (`completed` / skipped` / interrupted)を追加。`end_at` NULLはセッションがまだ実行中であることを意味する。`
2. ``cells.id``にCHECK制約を追加:
形式は`"0-1-4"`のように``{center}-{sub}-{cell}``。
3. ソフト削除は``deleted_at`` `TIMESTAMP NULL`に統一。``tasks``と``goals``だけ。
セッションは履歴データなので残し、セルは構造データなので削除しない。

この3テーブルだけを書き直して。クエリパターンに合うインデックスも提案して。

「ソフト削除を入れて」とだけ言うと、Claudeは方式を自分で選んでしまう。正しいアプローチとその理由まで一緒に書いて、はじめてその方針どおりに従ってくれる。

修正された核のテーブル

修正版で重要なのは、`sessions`と`cells`に入ったCHECK制約だ。誤ったデータがそもそも入らないように、**DBが直接ブロックする**。

```
CREATE TABLE cells (  
  id TEXT PRIMARY KEY,  
  goal_id INTEGER NOT NULL,  
  center_idx INTEGER NOT NULL,  
  sub_idx INTEGER NOT NULL,  
  cell_idx INTEGER NOT NULL,
```

```

FOREIGN KEY (goal_id) REFERENCES goals(id),
CHECK (id = printf('%d-%d-%d', center_idx, sub_idx,
cell_idx)),
CHECK (center_idx BETWEEN 0 AND 8),
CHECK (sub_idx BETWEEN 0 AND 8),
CHECK (cell_idx BETWEEN 0 AND 8)
);

CREATE TABLE sessions (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NULL,
  start_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  end_at TIMESTAMP NULL,
  ended_reason TEXT NULL,
  target_minutes INTEGER NOT NULL DEFAULT 25,
  FOREIGN KEY (task_id) REFERENCES tasks(id),
  CHECK (ended_reason IS NULL OR ended_reason IN
('completed', 'skipped', 'interrupted')),
  CHECK (end_at IS NULL OR end_at >= start_at),
  CHECK (target_minutes > 0)
);

```

注目すべき行: 末尾に並んだCHECK制約だ。セルインデックスが0～8の範囲を外れることを禁じ、セッション終了時刻が開始より早くなれないようにし、目標時間を常に正の値に強制する。自分から頼んでいない安全策をClaudeが追加してくれた。

インデックス—よく実行するクエリから先に決める

インデックスは、よく実行するクエリを先に決めてからそれに合わせて作る。次はMandaiでいちばんよく使う3つのクエリに合わせたインデックスだ。

```

CREATE INDEX idx_cells_goal ON cells(goal_id);
CREATE INDEX idx_sessions_task_start ON sessions(task_id,
start_at DESC);
CREATE INDEX idx_sessions_completed ON sessions(ended_reason,
start_at);

```

自分一人で作っていたら、カラムごとに別々のインデックスを作っていただろう。Claudeは実際のクエリパターンを先に考えたうえで、複合インデックスを組み立てた。

注意: 最初からインデックスを増やしすぎる必要はない。インデックスが増えるほど書き込みが遅くなり、ディスク容量も消費する。DuckDBは一部のインデックスオプション(特定条件のみを対象とするpartial indexなど)に対応していないので、こうした絞り込みはクエリ側のWHERE節で処理する。

2回目のレビューでもう1つ問題が見えてきた。_migrationsテーブルに実行順序の保証がない。マイグレーションランナーが2回走ったとき、重複挿入を防げない。

7.4 3回目のループ—マイグレーションシステム

3回目のループでは、スキーマそのものではなく、それを実行する仕組みを作る。**マイグレーションランナーとは?**「どのSQLを、どの順番で、しかも一度だけ実行するか」を管理するコードだ。アプリのアップデートでスキーマが変わるとき、既存ユーザーのDBを自動で新しい構造に合わせてくれる。

リクエストはシンプルだった。「DuckDBのConnectionを受け取り、まだ適用されていないマイグレーションだけを順番に実行し、適用済みの記録を_migrationsに残す関数。中心ロジックは10行以下で」。

結果は**1発で動いた**。

```
pub fn run_migrations(conn: &Connection) -> duckdb::Result<()> {
    conn.execute_batch(include_str!("../migrations/
000_meta.sql"))?;

    const MIGRATIONS: &[(&str, &str)] = &[
        ("001", include_str!("../migrations/001_init.sql")),
        ("002", include_str!("../migrations/
002_add_indexes.sql")),
    ];

    for (version, sql) in MIGRATIONS {
```

```

let applied: i64 = conn.query_row(
    "SELECT COUNT(*) FROM _migrations WHERE version = ?",
    [version], |row| row.get(0))?;
if applied == 0 {
    conn.execute_batch(sql)?;
    conn.execute(
        "INSERT INTO _migrations(version, applied_at)
        VALUES (?, CURRENT_TIMESTAMP)",
        [version])?;
    }
}
Ok(())
}

```

注目すべき2行: - `include_str!("...")`: SQLファイルをコンパイル時にバイナリへ直接埋め込むマクロ。実行中に別途ファイルを読み込まなくて済む。 - `if applied == 0`: 「すでに適用されたバージョンはスキップする」という核となる保証。

この種のコードは形が決まっているので、Claudeは一発でうまく書いてくる。**AIに一度で任せるのに向いた領域だ**。最初の通読では、コードを一行ずつ精読するより、この関数が「バージョン確認 → 未適用SQLの実行 → 記録の追加」の3ステップであるという形だけ覚えれば十分だ。

注意: この方式はマイグレーションを追加するたびに再コンパイルが必要になる。デスクトップアプリなら問題ないが、サーバー環境ならファイルを動的に読み込む方式の方が向いている。

`migrations/000_meta.sql`には`_migrations`テーブルの定義だけが入り、`migrations/001_init.sql`にはこれまで作ったテーブル式が入る。

ここから先、7.5～7.8は**スキーマの次の段階**だ。テストで制約を検証し(7.5)、Rustコマンドとして公開し(7.6)、よく使うクエリパターンを整理し(7.7)、最後にTypeScript側で型安全に受け取る薄いヘルパー(7.8)を作る。それぞれは短い。

7.5 スキーマのテスト—AIで品質検証する

クエリレイヤに移る前に、スキーマがきちんと働くかを確認めた。AIに「エッジケースと制約違反に絞ったテスト用INSERTを作って」と頼んだ。ポイントは、**エラーになるのが正常**な次のようなケースだ。

-- 失敗するべき: 不正なセルID形式

```
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx,
text)
VALUES ('invalid', 1, 0, 0, 0, '悪いID');
```

-- 失敗するべき: セルインデックス範囲超過

```
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx,
text)
VALUES ('9-0-0', 1, 9, 0, 0, '範囲超過');
```

-- 失敗するべき: 終了時刻が開始時刻より前

```
INSERT INTO sessions (task_id, start_at, end_at) VALUES
(1, '2026-04-17 10:00:00', '2026-04-17 09:30:00');
```

この3つのクエリはどれもCHECK constraint failed系のエラーになるはずだ。1つでも成功してしまったら、対応するCHECK制約が正しく効いていない。

正常作成、重複、不正参照、migration再実行、中間失敗を試験する。例として、子セルのある目標は削除を拒否し、子も消す明示要求がある場合だけ一つのトランザクションで削除できる。クラッシュ後はrunningを「復旧必要」とし、終了時刻を捏造しない。実際の方針は要件で決める。

7.6 一発で完成したコマンド—db_query_json

スキーマが固まると、すぐにRustコマンドをつないだ。このコマンドはフロントエンドから送られたSELECTクエリを実行し、結果をJSON配列の文字列で返す。シグネチャは次のとおり。

```
#[tauri::command]
pub fn db_query_json(sql: String, params:
Vec<serde_json::Value>) -> Result<String, String>
```

JSONはnull->SQL NULL、boolean->BOOL、整数->整数型、有限の小数->実数型、文字列->TEXTの順に変換する。配列・オブジェクトはJSON列に限り直列化し、それ以外はエラーにする。大整数の精度を失わないよう小数より整数を先に判定する。値はSQLへ連結せずバインドする。表・列名は値のバインドで守れないため許可リストから選ぶ。書き込み用コマンドdb_executeも似たシグネチャを持ち、query_rowの代わりにexecuteを呼び出し、影響を受けた行数を返す。

注意: 汎用クエリコマンドは強力なぶん危険でもある。フロントエンドがDELETEやDROPを含むどんなSQLでも実行できてしまう。プロダクションでは読み込みコマンドと書き込みコマンドを必ず分けるべきだ。

7.7 Mandaiのための DuckDB クエリパターン

スキーマとコマンドが揃ったら、実際のクエリを組み立てていった。最初の通読では、DuckDBの強みが1行で見える次の例だけで十分だ。

```
SELECT
    date_trunc('month', start_at) as month,
    COUNT(*) as total_sessions,
    COUNT(*) FILTER (WHERE ended_reason = 'completed') as
    completed
FROM sessions
WHERE start_at >= '2026-01-01'
GROUP BY month
ORDER BY month;
```

注目すべき行: COUNT(*) FILTER (WHERE ...)の部分。同じ結果をSQLiteで出そうとすると、CASE WHENを長々と書くことになる。DuckDBでは条件付き集計を短く読みやすく書ける。

先に集計単位を決める。曜日別の集中時間中央値は完了セッションの時刻を指定時間帯の曜日へ変換し、durationを曜日でまとめてpercentile_cont(0.5)を使う。完了率は完了数/開始数とし、分母0は計算不可にする。日付境界がUTCか現地時刻かを明示して画面間の合計を合わせる。第8章はAI API統合へ進む。

ソフト削除とJOINの落とし穴

複数のテーブルをJOINするとき覚えておくのは1つだけでいい。ソフト削除を使っているなら、すべての分析クエリのJOIN条件かWHERE節にdeleted_at IS NULLを入れる忘れないこと。忘れると、削除済みのデータが統計に混ざる。

```
LEFT JOIN tasks t ON c.id = t.cell_id AND t.deleted_at IS NULL
```

7.8 TypeScript型レイヤーAIが確かに役立った場所

DBコマンドの上にTypeScript型レイヤをかぶせる作業では、AIはほとんど手直し不要のコードをすぐに出してきた。このレイヤはフロントエンドでクエリ結果を型安全に扱うための、薄いヘルパー3つを提供する。

```
import { invoke } from "@tauri-apps/api/core";

export async function query<T>(sql: string, params: unknown[] = []): Promise<T[]> {
  const json = await invoke<string>("db_query_json", { sql, params });
  return JSON.parse(json) as T[];
}

export async function one<T>(sql: string, params: unknown[] = []): Promise<T | null> {
```

```

    const rows = await query<T>(sql, params);
    return rows[0] ?? null;
}

export async function execute(sql: string, params: unknown[] = []): Promise<number> {
    return invoke<number>("db_execute", { sql, params });
}

```

3つの関数の役割はシンプルだ。- query<T> —複数行を配列で返す - one<T> —1行だけ返す(なければnull) - execute —書き込み専用、影響を受けた行数を返す

ジェネリック<T>を受け取るので、query<Goal>(…)のように呼べば自動的にPromise<Goal[]>が返る。呼び出し側のコードもすっきりする。

```

interface Goal { id: number; title: string; created_at: string; }

const goals = await query<Goal>("SELECT * FROM goals WHERE deleted_at IS NULL");
const activeGoal = await one<Goal>("SELECT * FROM goals WHERE id = ?", [goalId]);

```

このTypeScriptレイヤーはループが1度も必要なかった。リクエスト1回で終わりだ。Tauriのinvoke、ジェネリック、JSONパースを組み合わせる決まりきったパターンだからだ。判断が要らない作業ではAIは速くて正確だ。逆に7.2～7.4節のように何かを決めなければならない作業では、AI単独では答えを出せない。その決定は開発者がするしかない。

注意: このレイヤーは実行時にデータを検証しない。SQLが想定と違うカラムを返してもエラーにならず、ランタイムでプロパティが*undefined*になって現れる。プロダクションではZodのようなランタイム検証ライブラリと併用するのが安全だ。

7.9 この章のループまとめ

冒頭で見たループ表をもう一度眺めると、どの作業が1回で終わり、どれが何度もループを必要としたか、パターンがはっきりしてくる。1回で終わったもの: `run_migrations()`、`db_query_json`、TypeScriptヘルパー、テストケース。2〜3回のループが必要だったもの: スキーマ設計、制約の仕様、インデックス戦略。

Mandai全体を通じてこのパターンは繰り返された。コードに決まった形があり境界が明確なら、AIは一発で仕上げる。設計上の決定が混じってくると、2〜3回のループが要る。この区別を知っていれば、プロンプトを送る前に何往復必要かをあらかじめ見積もれる。

DB作業のパターン分類

- ・ スキーマ設計 = 2〜3回のループ(ビジネス上の決定が多い)
- ・ クエリ関数 = 1回のループ(確立されたパターン)
- ・ 型レイヤ = 1回のループ(決まった形)
- ・ テスト生成 = 1回のループ(AIがエッジケースをよく拾う)
- ・ マイグレーションシステム = 1回のループ(Rustのパターンが固まっている)

7.10 Rust側のコマンドレジストリ

最後に `lib.rs` へコマンドを登録する。5章で見た `invoke_handler![...]` に、この章で作った3つのコマンド(`db_query_json`、`db_execute`、`run_migrations_cmd`)を追加し、`setup`ブロックで `init_db()` と `run_migrations()` を呼べば終わりだ。Tauriがこの構造を強制するため選択の余地はほとんどなく、新しいコマンドが増えるたびに同じ場所に1行ずつ足していけばいい。正確な骨組みは5章の登録コードをそのまま踏襲する。

注意: `init_db()` や `run_migrations()` が失敗すると、アプリ起動時にパニックする。プロダクションではデータベース初期化のエラーを適切に扱う必要がある。

開発中のデバッグTips: パニックすると、メッセージが一瞬で流れていきがちだ。ターミナルを開いたまま `cargo tauri dev` を走らせれば、初回起動の失敗メッセージを取りこぼさず確認できる。メッセージが見えない場合は、`init_db()` の `map_err` の中に `eprintln!` を一時的に差し込み、どの段階で詰まったかを絞り込むとよい。

7.11 次の章

次章ではOpenAI、Claude、Geminiの3つのAIプロバイダとストリーミング連携を実装する。スキーマより複雑だ。SSE形式、エラー形式、タイムアウトがプロバイダごとに違い、一部の実装は5回以上ループを回した。

この章で必ず覚えておくこと

この章を一度読み終えたら、次の5つだけ覚えておけば十分だ。

1. **スキーマ設計は一発では終わらない。** 最初の草案で抜けた決定が浮かび上がるのが普通だ。その隙間を2回目、3回目のループで埋めていく。
2. **AIはSQLiteをデフォルトで使う。** DuckDBを使うならプロンプトに明記すること。そうしてはじめて分析関数、豊富な型、JSON関数といった強みを活かせる。
3. **CHECK制約は、誤ったデータが入らないようDBが直接遮るルールだ。** セルインデックスの範囲、セッション終了時刻のような基本ルールは、最初からかけておく。
4. **接続管理は1つに統一する。** `OnceLock<Mutex<Connection>>` のパターンで、アプリ全体で単一の接続を共有する。コマンドごとに接続を開き直さないこと。
5. **AIが一発で片づける作業と、ループの要る作業を見分ける。** 定型パターン(マイグレーションランナー、クエリヘルパー、型レイヤ)は1回で終わる。設計上の決定(スキーマ、制約、インデックス戦略)は2~3回のループが必要だ。

学んだこと: データベーススキーマの作業は、AIと協業するときに予測可能なパターンに従う。設計の決定はAIと2〜3回やり取りが要るが、実装の細部は一発で終わることが多い。次章のストリーミングAI APIは反復回数がより多く、解決策もより複雑だ。

第8章. AI API統合—AI を使ってAIコードを書く

3つのAIプロバイダ (OpenAI、Claude、Gemini) を1つのUIに接続する作業は、AIコーディングツールがもっとも役に立つと同時にもっとも罫の多い領域だ。プロバイダごとに認証方法、レスポンス形式、エラーの形がすべて違う。ドキュメントに書かれているスペックはAIがうまく処理する。しかし、ドキュメントに載っていない部分、つまりユーザーがキャンセルボタンを押したときにHTTPリクエストがどう切れるか、ストリームが止まったら何を表示するかといった問題は別だ。

この章の全体像は一文で済む。**差異はRust側の一箇所に集め、フロントエンドは3つのイベントだけを受け取る。**

```
[フロントエンド (SvelteKit)]
  ask_ai_stream(provider, model, messages)
  |
```

```

      v
[Tauri コマンド] --> [Provider レイヤ(Rust)]
                    - OpenAI
                    - Claude  HTTP ストリーム -> テキスト断片を抽出
                    - Gemini

      |
      v
[フロントエンド listen]
  ai_chunk(テキスト断片) / ai_done(完了, 使用量) / ai_error(エラー)

```

画面上でユーザーが目にするものは、もっと単純だ。チャット入力欄に質問を入れると、回答エリアに文字が少しずつ追加されていく。たとえばプロバイダが "Hel"、"lo"、" world" の3つの断片を送ってくれば、フロントエンドはそれをつなげて "Hello world" として表示する。この章のコードは、まさにこの「断片を受け取って連結していく流れ」を安定したものに仕上げる作業だ。

プロバイダごとにレスポンスやエラーの形式が違っても、フロントエンドは ai_chunk(テキスト断片)、ai_done(完了)、ai_error(エラー)の3つのイベントだけ知っていればよい。差分を吸収する仕事はすべてRustのプロバイダレイヤが引き受ける。

ここで本書ではじめて出てくる用語を一つ押さえておこう。**SSE(Server-Sent Events)** とは、サーバーが一度開いておいたHTTP接続を通じてレスポンスを少しずつ流していく方式だ。AIプロバイダは、トークンが生成されるそばから1つずつ送るためにSSEをよく使う。フロントエンドはレスポンス全体を待たずに、届いた断片をすぐに画面に追記できるので、「タイピングされているような」体験が生まれる。

最初に実装するとき、もっとも重要な判断は「**差異をどこに隠すか**」だった。差異をUIのあちこちに散らすと、ボタン・チャット欄・エラー-toastがプロバイダ別の条件分岐で複雑になる。差異をRustの一箇所に集めれば、フロントエンドは同じイベントを受け取るだけで済む。ユーザーにとって必要な情報は、レスポンスが来ているか、終わったか、失敗したか、それだけだ。

この章の流れ

- ・ 要件を先に文章でまとめる
- ・ 複数プロバイダの抽象化—Rustのtraitパターン
- ・ イベント構造とストリームのキャンセル(本筋)

- ・ providerのイベントを共通のdelta・done・errorへ変換し、stream IDごとに取消と終了を処理する。
- ・ 第9章につながるまとめ

学習目標

- ・ 複数のAIプロバイダを1つのインターフェースにまとめるRustのtrait(インターフェースに相当する概念)パターンを理解する。
- ・ ai_chunk / ai_done / ai_error の3つのイベントでストリーミングを扱う流れに慣れる。
- ・ プロバイダごとに異なるSSE・エラー・認証の差を、なぜ一箇所で吸収するのかを知る。
- ・ APIキーの保管といった、基本的なセキュリティ原則を押さえる。

一読目に見るべきこと

この章ではRustの非同期、SSE、プロバイダごとのレスポンスの違い、キャンセル処理までが一気に登場する。最初からすべてを理解しようとする则難しい。一読目では、次の4点だけ押さえれば十分だ。

- ・ UIはプロバイダごとの差を知らなくてよい。
- ・ RustのプロバイダレイヤがOpenAI、Claude、Geminiの差を吸収する。
- ・ フロントエンドは ai_chunk、ai_done、ai_error の3つのイベントだけを受け取る。
- ・ APIキーやコスト上限といった運用面の細部は重要だが、アプリ全体の流れを掴んでから読み直せばよい。

tokio::select!、プロバイダごとのSSE終了シグナル、レートリミットを今すぐ覚え切する必要はない。この章の核は、「**差異は一箇所に隠して、画面はシンプルに保つ**」ことだ。

8.1 要件を先に文章でまとめる

AIに「3つのプロバイダをつないでくれ」とだけ頼むと、結果がばらつく。プロバイダごとに異なるサンプルコードが混ざり、エラー処理もまちまちになる。そこで、コードを書き始める前に要件を整理した。

- ・ フロントエンドはプロバイダを意識しなくてよい。使うコマンドは1つだけ:ask_ai_stream(provider, model, messages)。
- ・ ストリーミングはTauriの emit で ai_chunk イベントを発生させる。フロントエンドは listen で受け取る。
- ・ トークン使用量はストリーム終了時に ai_done イベントで一度だけ送る。
- ・ すべてのエラーは ai_error という1つのイベントに統合する。
- ・ APIキーはOSのキーリングから読む。コマンド引数で渡さない。
- ・ cancel_ai_stream(stream_id) コマンドで進行中のリクエストを止める。

この6行をプロンプトの一番上に貼り付けた。Claudeはセッションを通じてこの仕様に従い続けた。**要件を短く、明確に、先頭に固定すると、修正回数が半分になる。**ストリーミングのように状態が複雑な機能ほど、始める前にイベント名と責任の境界を釘づけにしておく必要がある。

設計の中心にある原則は、「**フロントエンドをプロバイダの変化から守る**」ことだ。モデル名が変わっても、新しいプロバイダが追加されても、チャットUIには手を入れずに済むようにする。

セキュリティ:環境変数に入れたAPIキーは、実行中のすべてのプロセスから見える状態になる。OSのキーリング(macOSのKeychain、WindowsのCredential Manager、LinuxのSecret Service)を使うべきだ。Tauriの *tauri-plugin-keyring* がこの部分を標準化してくれる。AIへのプロンプトでは、キーリングを使う旨を明示的に書いておく必要がある。

8.2 複数プロバイダの抽象化—Rustのtrait

なぜtraitが必要なのかから短く押さえよう。プロバイダごとのコードをコマンドハンドラの中にそのまま並べていくと、徐々に次のような形に膨らんでいく。

```
if provider == "openai" { ... 100行 ... }
else if provider == "claude" { ... 100行 ... }
else if provider == "gemini" { ... 100行 ... }
```

プロバイダが増えるたびに同じ関数がさらに長くなり、一箇所を直すと別の分岐が壊れやすくなる。共通インターフェースでまとめておけば、新しいプロバイダは同じ約束に合わせて別ファイルに実装するだけで済む。

Rustの **trait**はTypeScriptのインターフェースと似た概念で、「このリストに並ぶ機能をすべて実装したものだけがプロバイダとして使える」という契約書の役割を果たす。このプロジェクトのプロバイダtraitは、メソッド4つだけ用意すればよい。

TypeScriptに置き換えるなら、こんな形に近い。

```
interface ChatProvider {
  streamChat(input: ChatInput): Promise<void>;
  parseChunk(event: unknown): string;
  parseUsage(event: unknown): Usage;
  mapError(error: unknown): AiError;
}
```

Rustのtraitも同じ約束を強制する。違うのは文法だけだ。初心者は「プロバイダごとに必ず同じ4つの機能を実装させる契約」として理解すれば十分だ。

メソッド	役割
stream_chat	HTTPリクエストを送り、SSEイベントを受け取ってemitする
parse_chunk	1つのイベントからテキスト断片(delta)を取り出す
parse_usage	トークン使用量データを取り出す
map_error	HTTPステータスとレスポンスを共通エラー形式に変換する

この4つを決めておけば、コマンドハンドラはプロバイダ文字列を見て該当インスタンスを選び、stream_chat を呼ぶだけで済む。擬似コードでは次のひと塊になる。

```
// コマンドハンドラ: 分岐は一度だけ、あとはtraitメソッドを呼ぶだけ
```

```
let p: Box<dyn ChatProvider> = match provider {
    "openai" => Box::new(OpenAi),
    "claude" => Box::new(Claude),
    "gemini" => Box::new(Gemini),
    _        => return Err(unknown_provider()),
};
p.stream_chat(model, messages, stream_id, app).await
```

新しいプロバイダを足すときは、同じ4つのメソッドを実装した型を追加して、matchの行を1行増やすだけだ。コマンドハンドラの本体やフロントエンドには手を入れなくてよい。

共通providerは要求と取消信号を受け、テキスト差分・完了・エラーを返す。例えば `Delta { stream_id, text }`、`Done { stream_id }`、`Error { stream_id, code, message }` とする。元の応答はアダプター内で変換し、UIをprovider名で分岐させない。鍵やHTTP応答原文をエラー文へ出さない。

8.3 イベント構造

traitが「差異を一箇所に集める器」だとすれば、イベント構造は「その器が外に見せる一定の形」だ。この2つは1組になっている。traitがあってもイベント名が毎回変わるならフロントエンドは揺れるし、イベントが統一されていても内部に抽象化がなければハンドラが膨らんでいく。

最初の段階では、実際のHTTP通信を後回しにして、**Tauriのコマンドとイベント発行の構造だけを先に** 固めた。要点は2つだ。

- ・ バックエンドはコマンドが呼ばれると `tokio::spawn`(Rustの非同期ランタイム `tokio` が提供する、タスクをバックグラウンドに流す関数) でタスクを起動し、すぐに戻る。**JavaScriptで Promise を作ってバックグラウンドに流すのと似た役割**だ。だからフロントエンドはレスポンスを待たずに済む。
- ・ フロントエンドは `listen` で `ai_chunk`、`ai_done`、`ai_error` の各イベントを購読する。

フロントエンドのリスナーの主要な流れは、4行ほどに収まる。

```
const unChunk = await listen<{ delta: string }>("ai_chunk", e =>
  buf += e.payload.delta);
const unDone = await listen<{ usage: Usage }>("ai_done", e => {
  usage = e.payload.usage; });
await invoke("ask_ai_stream", { provider: "claude", model,
  messages });
```

ここでの肝は、**buf がdeltaを順に連結していく形**だ。ai_chunk イベントはメッセージ全体を一度に送らない。新しく届いたトークンの断片だけを渡す。フロントエンドがその断片を順番につなぎ合わせて、完全なメッセージを作り上げる。こうすることでペイロードが小さくなり、ストリームがずっと速く感じられる。

```
ai_chunk: "Hel"      buf = "Hel"
ai_chunk: "lo"       buf = "Hello"
ai_chunk: " world"   buf = "Hello world"
ai_done              レンダリング終了、usage を保存
```

そして同じ正規化の一部として、**フロントエンドが受け取るペイロードの形式は4種類に固定**する。ai_chunk は { delta }、ai_done は { usage }、ai_error は { provider, kind, message }、そしてキャンセル通知は { stream_id } だ。プロバイダが増えても、この形は変わらない。Claudeは一度 ai_done に model や finish_reason まで詰め込もうとしたが、「フロントエンドで使わないフィールドはひとまず外して、必要になったら足そう」と整理した。**AIはとにかくフィールドを増やそうとする**。放っておくとペイロードが膨らみ続ける。

リスナーを登録したら、必ずクリーンアップ(cleanup)しなければならない。listen が返してくれる関数(unChunk、unDone)を呼ばないと、次のメッセージで同じコールバックが二重三重に積み上がって呼ばれ、画面に同じテキストが重複して描画される。ページを離れてもリスナーは生き残り、メモリを掴んだままになる。**クリーンアップのタイミングは2箇所だ—ストリーム終了直後(ai_done または ai_error 受信時)と、コンポーネントがunmountされる時**(Svelteなら onDestroy、Reactなら useEffect のcleanup)。

ストリームのキャンセル

ここまでの構造には、ひとつ問題が残っている。**一度始まったストリームを止める方法がない** ことだ。ユーザーが「停止」をクリックしても、ページを離れても、バックグラ

ウンドのタスクは走り続ける。APIクレジットを消費し、すでに閉じられたフロントエンドにイベントを送り続ける。

解決の核は一行で済む。**ストリーム1本ごとにIDとキャンセル用チャンネルを別々に用意する**。アプリ全体でチャンネルを1つしか持たないと、キャンセル信号がすべてのストリームを叩き起こしてしまい、どれを止めるべきか判別できない(こうした、2つの処理が絡み合う状況を **race condition** と呼ぶ)。

具体的には、ストリーム開始時にUUIDで `stream_id` を作り、IDごとにキャンセル用チャンネルを登録する。`cancel_ai_stream(stream_id)` が呼ばれたら、そのIDのチャンネルにだけ信号を送る。タスク側では `tokio::select!` という道具で、「キャンセル信号の到着」と「HTTPストリームの完了」のうち先に発生した方をそのまま処理する。`tokio::select!` は **JavaScriptの Promise.race と似た仕組み**で、複数の非同期処理のうち最初に終わったものを拾う。キャンセルが先に届いた場合、バックエンドは `ai_cancelled` で通知する。フロントエンドはこのペイロードで `stream_id` を受け取り、どのストリームの終了かを識別する。

stream ID生成、リスナー登録、要求開始の順にする。登録前に応答すると先頭トークンを失う。`callback`は自分の`stream_id`だけを扱い、完了・エラー・取消・画面終了で同じ冪等`cleanup`を呼ぶ。登録が遅れて既に画面が破棄されていたら即座に解除する。第5章の`disposed`も同じ原理だ。

注意点:ストリームが自然に終わる瞬間とキャンセル信号がぶつかると、`ai_done`と`ai_cancelled`のイベントが同時に発生することがある。フロントエンドは最初の完了通知を受け取った後にやってくる追加イベントを無視する必要がある。

8.4 SSEパースで詰まったところ

ここから8.6までは、「一読目を終えて実際にコードを書き始めると直面するディテール」だ。最初に読むときは、各節とも全体像を1行ずつ拾って次へ進むだけでよい。

全体像はシンプルだ。**プロバイダごとに終了シグナル、イベントの種類、認証情報の渡し方が違う**。OpenAIは終了が`data: [DONE]`という文字列なのでJSONとして解釈すると壊れる。さらに`usage`は最後のチャンクにしか入っていない。Anthropic(Claude)はSSEの`event:`行で分岐する必要があり、出力トークン数は累積値ではなく現在の応答の値なので足し合わせてはいけない。Geminiは既定がSSEではなく **JSON lines**ス

ストリーム(JSONオブジェクトを改行で区切って送る方式)なので、URLに `?alt=sse` を付ける必要があり、認証もヘッダーではなくクエリパラメータ (`?key=...`) で渡す。

アダプターは共通要求を provider 別 JSON へ変え、応答を共通イベントへ戻す。ネットワークのチャンクと SSE イベントは一致しないため、空行区切りまでバッファーへため、分断された UTF-8 も処理する。HTTP は `request`、JSON は `serde_json` を使えるが、フィールド名と終了記号は API の版に合わせる。

注意1: Anthropic の呼び出しには `anthropic-version` ヘッダーが必須だ。これが抜けると HTTP 400 で失敗する。バージョン文字列はコードにハードコードせず、公式ドキュメントで現在の値を確認しよう。

注意2: Gemini は API キーをクエリパラメータで渡す。サーバーログや HTTP デバッグツールに残るおそれがある。本番運用では OAuth2 の利用を検討すべきだ。

注意3: トークン数の正確さはプロバイダによって違う。トークン数に直接依存する課金ロジックを作らない方がよい。

一読目ではこの3つの注意事項を暗記する必要はない。重要なのは、**プロバイダごとにストリーム形式が違うので、その差をUIではなくプロバイダレイヤで処理しなければならない** という点だ。

8.5 エラー処理—プロバイダごとに違う失敗をするとき

3つのプロバイダは、いずれもエラー応答の形式が異なる。OpenAI は `error.type` と `error.code`、Anthropic は `error.type`、Gemini は `error.status` と `error.code` といった具合に、フィールド名も位置もばらばらだ。

この違いをそのままフロントエンドに渡すと、UI 側で分岐コードが増えていく。**エラーを正規化する目的はただ一つ**。プロバイダがどんな形で失敗しても、フロントエンドは `{ provider, kind, message }` という同じ形だけを受け取る。kind は `auth`・`rate_limit`・`context`・`server`・`unknown` といった少数の分類に絞り、HTTP ステータスコードとプロバイダ別のエラータイプ文字列を見て、そのいずれかに振り分ける。

HTTP 401/403は認証・権限、429は制限、5xxはサーバー、timeoutは時間切れ、解析失敗は形式エラーと区別する。Retry-Afterを守り再試行に上限を置く。不正な鍵を変更せず反復しない。画面には安全な説明、ログには要求IDと種類を残す。

注意: AIプロバイダのエラーメッセージには、APIキーの一部や内部サービス名といった機微情報が含まれることがある。ユーザーに表示したりログに記録したりする前に、必ずサニタイズすること。

8.6 リクエスト制限とコスト管理

運用段階で追加検討すべきこと: AI APIは呼び出しごとに料金が発生し、プロバイダごとに「分あたりのリクエスト数」と「分あたりのトークン数」という2つの上限が別々に設定されている。一般的には **token bucket** アルゴリズムでこの2つの上限を同時に管理し、同じ場所にモデル別の単価を置いて累積コストを追跡する。価格表や上限は頻繁に変わるので、コードにハードコードせず設定ファイルに分離する。そして請求計算には事前見積もりではなく、レスポンスの *usage* の数値を使う。一読目では「リクエスト数とトークン数の2つの上限を別々に見る」だけ覚えておけば十分だ。実装の細部は運用編の章で扱う。

8.7 レスポンスの正規化—フロントエンドに統一された形式を渡す

レスポンスペイロード形式の正規化そのものは、8.3でイベント構造と一緒に整理した。ここでは、その決定がなぜ揺らいではいけないかを、もう一度確認する。

ペイロードの形を崩すと、フロントエンドとバックエンドが不必要に絡み合う。5番目のプロバイダを追加するときに、チャットUIにまで手を入れることになる。だから新しいフィールドの追加は保守的に扱う—「このフィールドを今、画面のどこで使うのか」に答えられないなら、いったん保留する。正規化の価値は、形式が変わらないという約束 それ自体から生まれるからだ。

8.8 この章のループまとめ

この章のあいだに領域別の修正回数を表にしてまとめておいたが、次章の冒頭で同じデータをもっと詳しく分析する。ここで覚えておくべき一行はこれだ。設計がしっかり決まっていれば、ほとんどは1〜2回で通り、プロバイダごとのプロトコルの例外でだけ修正回数が増えた。第9章ではこの傾向を一般化する。

8.9 AIがコードを書くときの特徴

この節は第9章のテーマと重なる。AIが得意な領域(慣用パターン・抽象化の設計・サンプル付きのデータ変換)と苦手な領域(プロバイダごとのプロトコル例外・終了シグナル・認証情報の位置)の区別、そして「苦手な領域にはAPIドキュメントの要約をプロンプトに添える」という対処法は、次章で事例とともに本格的にまとめる。ここでは一行だけ残しておく—要件が明確で例が十分にあるとき、AIによるコード生成はもっともうまく機能する。

8.10 この章で必ず覚えておくこと

- ・ 一文要約: 差異はRustの一箇所に集め、フロントエンドは `ai_chunk` / `ai_done` / `ai_error` の3つのイベントだけを受け取る。
- ・ 核となる構造: プロバイダ `trait`(メソッド4つ) / Tauri イベントベースのストリーミング(`delta` 累積) / `stream_id` 単位のキャンセル(`tokio::select!`)。
- ・ プロバイダ特有の罫: OpenAI は `[DONE]` が文字列で、`usage` は最後のチャンクにしか入らない。Anthropic は `event:` 行で分岐し、出力トークン数は累積ではない。Gemini は既定が `JSON lines` で、`?alt=sse` が必要、認証はクエリパラメータ。
- ・ 運用面: API キーはOSのキーリングに、レートリミットは2種類の上限(リクエスト数・トークン数)を別々に、価格表はコードに埋め込まない。
- ・ 安全面: リスナー登録後のクリーンアップ(ストリーム終了時・コンポーネント `unmount` 時)、キャンセルと正常完了が重なったら最初の信号だけを採用、エラーメッセージはサニタイズしてからユーザーに見せる。

- ・ AI活用:慣用パターン・抽象化設計・データ変換は得意。プロトコル例外は苦手 -> **要件を先に文章で整理し、例外処理にはAPIドキュメントの要約を添える。**

次章(第9章)では、この表で見えた傾向を一般化する。「Claudeがうまく処理する領域と詰まる領域はどこで、学習データにないパターンに直面したらどう対応するか」。そして、この章で固めた **traitベースの抽象化、3つのイベントペイロード、stream_id単位のキャンセル** というパターンは、第9章以降で外部APIを追加で接続するたびに同じ形で繰り返し現れる。7ラウンド分の試行錯誤の跡が、次の章の出発点になる。

第9章 Mandai 開発で犯した失敗 — AI が間違えた箇所

AI と共同作業すると、結果はふたつに分かれる。AI が正解を出したものと、間違えたものだ。多くを学べたのは後者だった。検証せずにコードを承認した箇所、機能を3つ積み上げてから初めて構造の問題に気づいた箇所 — そうした失敗が、次の試行の材料になった。

これまでの章は AI と共同作業する流れを扱ってきた。本章はその裏側を見せる。**AI が繰り返し間違えた領域**と、それをどう発見して回避したかの記録だ。一度や二度の修正で済んだタイプミスや import 漏れは除外している。5回以上修正したか、要件そのものを書き直す必要があった問題だけを選んだ。繰り返し現れた落とし穴は次の4つだった。

1. **状態の同期** — フロントエンドとバックエンドが異なる状態を見てしまう。

2. **暗黙のルール** — 自分の頭の中だけにあり、AI には知りようのない決まり。
3. **エッジケースへの無関心** — AI は基本的に正常系(エラーなく流れる一般的な経路)を好む。
4. **バージョンの混同** — AI が古い API と新しい API のパターンを混ぜてしまう。

用語を短く整理しておく。**状態の同期**とは、同じ情報を二箇所(たとえば画面と DB)に保持しているときに、両方の値が常に一致するようそろえる作業のこと。**ドメインモデルの逸脱**とは、AI がプロジェクト固有のルールではなく、一般的によく使われるパターンを選んで適用してしまう現象を指す。このふたつの用語は本章で繰り返し登場する。

注意: 検証せずに AI の「ベストプラクティス」を信じてはいけない。AI は、チュートリアルでは動くが本番サービスでは破綻するパターンを自信たっぷりに提示してくる。常に動作全体のフローをテストすること。

本章は AI を批判するための章ではない。むしろ逆だ。AI が間違いやすい場所を知っておけば、より正確な依頼を投げられる。「タイマーを作って」と「アプリを落としても進行中のタイマーが復元されること」は、まったく別の依頼だ。後者には状態の所有権、保存タイミング、復元ロジックがすべて含まれている。実アプリで必要になる要件はたいていこの種のものだ。一文具体的になるだけで、実装全体の方が変わる。

読み方ガイド: 本章は 9.1 が最も長く、それ以降は短い。9.1 さえ通過できれば、あとは一気に読める。9.1 が長くなったのは、本章の核心的な気づきがすべてそこに詰まっているからだ。

本章の流れ

最も重い事例から始め、徐々に軽い事例へ降りていく。最後に予防のための道具をまとめて整理する。

- ・ 9.1 ボモードロタイマー — 状態が二箇所に存在するときに何が起きるか
- ・ 9.2 Mandal-Art Cell ID — 一般的なパターンがドメインルールを上書きする瞬間
- ・ 9.3 生産性スコアの計算 — エッジケースを明示すると AI が改善する
- ・ 9.4 バージョンの混同 — Tauri 1.x と 2.x の API を混ぜてしまうとき
- ・ 9.5 防ぐ方法: プロンプト・レビュー・テストの3点セット
- ・ 9.6 まとめと次章

学習目標

- ・ AI 開発で繰り返し現れる4つの失敗パターン(状態同期、暗黙のルール、エッジケース、バージョン混同)を把握する。
- ・ それぞれの失敗をどう発見し、プロンプトで解決するかを実例から学ぶ。
- ・ AI の失敗を防ぐためのチェックリスト、レビュー用プロンプト、テストプロトコルを自分で作って活用する方法を身につける。

9.1 ポモドーロタイマー — 状態が二箇所にある

本節の核心的な教訓は一行で言える: 進行中タイマーの「残り時間」をどこに保存しようとして、必ず `start_at` から再計算できる状態でなければならない。5回の試行錯誤の末にこの結論にたどり着いた。以下の話はその過程だ。

Mandai のポモドーロ(25分の集中+短い休憩を繰り返す作業法)は、見た目には単純そうに見える。スタートボタンを押し、25分を数え、終わったら DuckDB にセッションを保存すればいい — そう感じる。だが実際は違う。画面に表示される数字の裏には、開始時刻、目標時間、紐づくタスク、終了理由、通知、そしてアプリを開き直したときにどう戻すかというルールが付いてくる。さらに、ユーザーはタイマー稼働中にアプリを閉じることもあれば、ノートPC が突然スリープに入ることもある。

最初はこうした条件をプロンプトに十分書き込まなかった。そのため AI が最初に作ったタイマーはデモレベルで、実際の条件を一つずつ追加するたびに新しい問題が露わになった。結果として5回作り直すことになった。修正のたびに、「タイマー」という単語が単純なカウントダウンを指していないことがはっきりしてきた。タイマーは状態を保存し、必要に応じてその状態を呼び戻せる、ひとつの小さなシステムだ。

本当の問題

中心的な問題は、ポモドーロの状態が二箇所に分かれて存在することだ。

- ・ **フロントエンド:** 画面上で毎秒減っていく残り時間。

- ・ **バックエンド** (DuckDB の sessions レコード): 永続ログ。start_at、end_at、ended_reason。

Claude は最初、**タイマーをフロントエンドだけに置き**、セッション終了時点で DB に保存する実装をした。デモ上は問題がないように見える。カウントダウンが進み、0 になったら完了レコードが残る。しかしアプリを閉じてもう一度開くと、実行中だったセッションはどこにも残っていない。必要だった要件は「アプリを落としても進行中のタイマーが続くこと」だった。つまり、タイマーの状態は画面ではなく DB から読み込んで復元できなくてはならない。

最初の試行(失敗)

最初は「25分のカウントダウンを表示して、終わったら DB に保存しろ」という程度の短いプロンプトを投げた。Claude が生成したコードはタイマー状態をコンポーネント変数に置くだけで、**セッションが終了したタイミングでのみ** DB に1行記録する形だった。

```
// 誤り: タイマー状態がコンポーネントにしか存在しない
let remainingSeconds = 25 * 60;
// 完了時点で一度だけ DB INSERT -> 実行中の永続性がない
```

デモでは動く。だがユーザーがセッションの途中でアプリを閉じた瞬間、進行中だった作業時間は一切記録されない。チュートリアル用のタイマーは画面が点いている間だけ動けばよいが、実アプリのタイマーは画面が消えた後も現状をどこかに残しておく必要がある。

この最初の試行に続いて、4回の修正があった。開始時点で DB に事前 INSERT する、再起動時に開いているセッションを片付ける、無条件に片付けるのではなく時間ベースで復旧する、そしてタイミング測定そのものを Rust バックエンドに移す。それぞれの修正は直前の試行のすき間から生まれている。詳細はこの節の末尾の「核心的な教訓」でまとめる。ここでは、その途上で出会った3つの落とし穴だけを取り上げる。

落とし穴1 — CHECK 制約違反

進行中のセッションを開始時点で INSERT しようとしたとき、Claude は `ended_reason='running'` を入れようとした。第7章の `sessions` テーブルでは `ended_reason` カラムに入る値が `'completed'`、`'skipped'`、`'interrupted'` の3つに制限されている(特定の値だけを許す DB ルール、CHECK 制約)ため、これは拒否された。

解決策は単純だ。開始時の `ended_reason` は NULL のままにしておき(= 進行中)、完了・中断・スキップの時点で値を埋める。第7章のスキーマをそのまま生かせる方法だ。

落とし穴2 — 過剰なクリーンアップと時間ベースの復旧

「再起動時に `ended_reason` が NULL のセッションを片付けてほしい」と頼んだら、Claude はアプリ起動時に開いているセッションをすべて `interrupted` に印を付ける関数を作った。ところがこれが攻撃的すぎた。**アプリが少しの間バックグラウンドに戻って戻ってきただけのケースでも、タイマーが中断扱いになってしまった。**ユーザーが5分後に戻ってきても、タイマーはすでに死んでいる。

ここで気づいた。アプリを閉じる、最小化する、ノートPC がスリープに入る、プロセスが強制終了される — これらはすべて別々の状況だ。しかし DB 上ではその差が `end_at IS NULL` という1つにしか表れない。

そこで方向を変えた。無条件に片付けるのではなく、**開始時刻を基準に再計算する。**
`start_at` から経過した時間が目標時間を超えていれば自動で終了処理し、まだ残っていれば残り時間を計算し直してタイマーを生き返らせる。要点は1つだ — 保存された「残り時間」を信用せず、毎回 `start_at` から計算し直す。

確認すべき出力: アプリを閉じて5分後に開き直すと、画面の残り時間が5分減っているはず。減っていなければ、保存値をそのまま使っているということだ。

落とし穴3 — バックグラウンドのスロットリングとタスクの紐付け

最後のループでさらに2点が明らかになった。

バックグラウンドスロットリングとは、アプリがバックグラウンドにある間、ブラウザエンジン (WebView) が JavaScript タイマーの実行頻度を落とす現象を指す。`setTimeout` も同じ影響を受ける。解決策はタイミング測定を **Rust バックエンドに移すこと**だ。Tauri のバックエンドはブラウザのスロットリングを受けない。バックエンドで時間を測り、終わったらフロントエンドにイベントを送る構造になる。

タスクの紐付けは別種の決定問題だ。ユーザーがタイマー実行中にタスクを切り替えたら、`sessions.task_id` も追従するべきか？ 明示的なルールがないと、AI は毎回違う推測をする。決定: **セッションは開始時点のタスクに固定する**。生産性レポートが解釈しやすくなり、統計に一貫性が出る。

確認すべき出力: アプリをバックグラウンドに置いて25分待つと、通知音が鳴るはず。

修正後のプロンプトテンプレート

5回のループを経た末、最初からこう書いていれば早く済んだはずというプロンプトはこれだ。

以下の要件で Pomodoro タイマーを実装せよ:

永続性:

- 開始と同時に DuckDB にセッションを保存 (`end_at=NULL`, `ended_reason=NULL`)
- 再起動時、経過時間が `target_minutes` 未満ならアクティブセッションを復元
- `target_minutes` を超えている場合のみ `ended_reason='interrupted'` でマーク

タイミング:

- Rust バックエンドの `tokio::time` を使用 (ブラウザスロットリングの影響なし)
- フロントエンドは表示のみ、実際のタイミングはバックエンドが管理
- バックエンドタイマー満了時に Tauri イベント `'timer_completed'` を発行

タスク紐付け:

- セッションの `task_id` は開始時点のアクティブタスクに固定
- セッション中のタスク変更は `session.task_id` に影響させない

テスト:

- [] タイマー開始 -> アプリを閉じる -> 再度開く -> 正しい残り時間で復元
- [] 全実行時間アプリをバックグラウンドに -> 通知が鳴る
- [] タイマー中にアプリクラッシュ -> 次回起動時に `'interrupted'` でマーク

注意: 間違った構造のまま押し進めるのは sunk cost の誤謬だ。同じ機能で3〜4回ループしても解決しないなら、AI の初期アプローチの方向自体が根本から間違っていないか点検すること。制約条件を整理し直して最初からやり直したほうが早いこともある。

核心的な教訓

状態の同期とは、フロントエンドのタイマーと DB のデータが常に同じ値を指すようにする作業のことだ。この問題は AI 単独では解決しにくい。なぜなら、3つのことを先に人間が決めなければならないからだ。フロントとバックエンドのどちらがどの役割を担うか、アプリが異常終了したらどう復旧するか、そしてデータ一貫性をどう保証するか。これら3点を決めて AI に伝える必要がある。そうでなければ AI は、すべてが正常な場合だけ動くコードを書く。

5回のループを経たあと、**タイマー状態同期チェックリスト**を作った。

- ☐ フロントエンドのタイマーと DB の行は同じ瞬間に開始されるか?
- ☐ アプリクラッシュ時、開いているセッションに何が起こるか?
- ☐ スリープ/バックグラウンドからの復帰時、ユーザーに見える状態は?
- ☐ 終了通知を鳴らすのはフロントエンドかバックエンドか?
- ☐ セッションメタデータ(タスクなど)が実行中に変化したら、無視するか反映するか?

似たような機能を作るときは、プロンプトの先頭にこの5項目を貼って始める。1回の試行錯誤で得た教訓が、以後ずっと使えるプロンプト資産になる。次節では、この「事前明示」の原則を別種の落とし穴 — ドメインルール — に適用してみる。

9.2 Mandal-Art Cell ID — 一般的なパターンがドメインルールを上書きする瞬間

状態の同期の次によく出会った落とし穴が **ドメインモデルの逸脱**だ。AI はプロジェクト固有のルールを知らないまま、学習データで頻繁に見たパターンをそのまま当てはめる。Cell ID の事例が最も直感的だ。

第6章で cell ID の形式を `{center_idx}-{sub_idx}-{cell_idx}` と決めた。この決定は `.cursorrules` にも書き残してあった。それなのに AI は UI コンポーネントを書く際にまったく別の形式を生み出した。

どこでズレたか

Claude が書いた `MandalaGrid.svelte` はセルを `id="cell-${row}-${col}"` の形式でレンダリングしていた。row と col は 0 から 8 までの数値だ。結果として DB のテーブルは 1-3-5 形式を使い、画面の DOM は `cell-3-5` 形式を使う状態になった。クリックハンドラが DOM の id を使って DB を検索した瞬間、二つの形式が噛み合わずエラーになった。

原因を突き止めるのに2時間かかった。コンソールログに `No cell found with id 'cell-3-5'` というメッセージが繰り返し出るのに、DB には `'1-1-1'`、`'1-1-2'` のような形式しか入っていない。ID 体系が二系統に割れていた。

なぜこんなことが起きるのか

私が送ったプロンプトが一般的すぎた。「9×9 のセルグリッドを作ってほしい」としか書かず、このグリッドが実は **3×3 のブロックが重なって構成されている構造**であることを伝えなかった。Claude は自然に一般的なグリッドパターン (row/col) を選んだ。AI の側から見れば妥当な選択だったが、Mandai のドメインモデルとは合わなかった。`.cursorrules` に形式は書いてあったが、他の形式を禁じるとは明記していなかった。だから AI はより馴染みのある一般的なパターンを選んだ。

修正の方法

修正はコードではなくルールファイルから始めた。`.cursorrules` に次の一文を加えた。

Mandal-Art (9×9 目標グリッド) の cell ID は ``{center_idx}-{sub_idx}-{cell_idx}`` 形式の文字列とする。
DOM の `data-cell-id` も同じ形式を使うこと。他の形式は許可しない。

プロンプトもより具体的にした。「9×9 グリッド」とだけ書くのではなく、「3×3 のサブグリッドをまとめた構造で、ID は `{center_idx}-{sub_idx}-{cell_idx}` 形式を

使うこと」と明示した。修正後のコンポーネントは DOM の `data-cell-id` も同じ形式に統一するようにした。

確認すべき出力: セルをクリックしたときコンソールに 1-3-5 のような形式が出力されれば正常。cell-3-5 が出るなら古いコードが残っている。

`.cursorrules` にその一行を加えてから、同じ間違いは繰り返されなくなった。**ルールが明示的に書かれていないと、AI は毎回その都度推測し直す。**

注意: AI はドメインモデルと衝突しても「標準的な」パターンを自信たっぷりに使う。明示的に禁じない限り、一般的なパターンがドメイン固有のルールを上書きしてしまう。

この経験から得られる結論は単純だ。AI の推測は「一般的によく使われているもの」のほうに流れる。ドメインがその一般的なパターンから外れる箇所では、毎回その違いを明示する必要がある。特に次の3つは、プロンプトかルールファイルの先頭に書いておくのが安全だ。

- ・ 一般的な慣用と異なる ID の付け方
- ・ 一般的な慣行と異なるデータフロー
- ・ 標準ライブラリの規約と異なる命名規則

この3つは好みの違いではなく、データモデルの一部だ。次節ではドメインではなくデータ自体の境界値で同じパターンがどう繰り返されるかを見ていく。

9.3 生産性スコアの計算 — エッジケースを明示すると AI が改善する

Mandai は「週間生産性スコア」を表示する。スコアの計算式は自分で決めた。

```
score = (completed_tasks × 10 + pomodoro_minutes × 1) /
effective_time × correction
effective_time = 作業日設定の「集中時間」の合計
```

この式を SQL クエリにしてほしいと AI に依頼した。最初のプロンプトはシンプルだった — 「この式に従って週ごとに計算しろ」。

ループのたびに現れたエッジケース

Claude が生成したクエリは、月曜から日曜まで1週間が丸ごと埋まっている場合にはきれいに動いた。しかし実際にアプリを使う人は完成した一週間だけを見るわけではない。月曜が過ぎたばかりの時点、水曜の午後、祝日が挟まる週など、途中で確認するケースのほうがはるかに多い。

ループを重ねながらエッジケースが一つずつ表に出てきた。

ループ	見つかった問題	修正
1	常に 7 で割る。水曜に見るとスコアが低すぎる	現在週は経過日数で割る
2	週末も働いていないのに分母に含まれる	平日のみ合算
3	休日設定が無視される	休日テーブルを LEFT JOIN
4	実効時間が 0 だと除算エラー	0 のときは 0 を返す

ループのたびにクエリが4~5行ずつ伸びた。最も大きな違いは分母の決め方だった。最初は無条件に 7 で割っていたが、修正後は「現在週なら経過した曜日まで、過去週なら 7 日」と分岐するようにした。

確認すべき出力: 新しい週の月曜にスコアを見たとき、低すぎないこと。分母が 1 で水増しされたスコアになっていないかも合わせて確認する。

この経験から得たもの

パターンは単純だ。**AI は明示されたものだけを処理する。言わなければ抜け落ちる。**エッジケースは開発者が自分で見つけて伝えなければならない。一度伝えれば、AI はそのパターンをその後の作業にも適用する。

この一件以降、別の機能を依頼したとき、Claude が自発的に次のようなケースを先に確認するようになった。

- ・「もし分母が 0 だったら？」
- ・「もし日付範囲が空だったら？」

- ・「もし設定値が null だったら？」

エッジケースを一度教えると、AI は別の機能でもその教訓を適用するようになる。ただし、最初に開発者が考えて投げ込んでやる必要はある。

注意: AI はロジックが合っていれば満足してしまう。極端な入力値でも意味のある結果が出るかは、自分でテストする必要がある。

次節では、こちらのコードではなく **フレームワークそのものの変化** が AI を混乱させるケースを見ていく。

9.4 バージョンの混同 — Tauri 1.x と 2.x の API を混ぜてしまうとき

AI の学習データには Tauri 1.x と 2.x の例が両方混ざっている。進化の速いフレームワークほど、インターネット上には古いチュートリアルがそのまま残っており、AI はそれも学習している。結果として「Tauri のコード」とだけ伝えたと、モデルは統計的により頻繁に見ている v1 のパターンを自信たっぷりに選び、v2 のコードの間に挟み込む。こうしたコードはコンパイルは通るが、実行するとエラーになる。さらに厄介なのは、開発中はうまく動くのに本番ビルドでだけ壊れるパターンだ。

典型例: ウィンドウ管理 API

「最小化してトレイへ」機能を依頼したら、Claude が返してきたコードは v1 と v2 の import が混ざっていた。v1 では `appWindow` というグローバルオブジェクトを使うが、v2 では `getCurrentWindow()` 関数でウィンドウを取得する。AI は両方の方式を同じファイルの中で同時に使っていた。

TypeScript は import を有効と認識したのでコンパイルは通った。しかし実行すると「そのような関数は存在しない」というエラーになった。

失敗したときに見る場所: 「function not found」のようなエラーは、ほぼ常にバージョン不一致のサインだ。Rust 側のコマンドではなく、フロントエンドの import を先に確認すること。これを知らずに Rust 側を4時間眺めてしまった。

バージョン明示プロンプトで解決

解決策はプロンプトでバージョンを釘付けにすることだった。

Tauri 2.x を使用 (1.x ではない)。トレイへの最小化を実装せよ。主な違い:

- '@tauri-apps/api/window' から `getCurrentWindow()` で取得。appWindow グローバルではない
- 基本的なウィンドウ操作で `invoke()` を使わず、window メソッドを直接使う
- トレイ API は `@tauri-apps/api/tray`。旧 `system-tray` プラグインではない
- ウィンドウイベントは `window.listen()`。appWindow.listen() ではない

Tauri 2.x のみを表示すること。API に確証がない場合は推測せず質問せよ。

これ以降、すべての import が v2 のパスに統一された。

確認すべき出力: package.json の @tauri-apps/api のバージョンと import パスが一致していること。v1 パターンの appWindow グローバルが残っていたら赤信号だ。

v1 -> v2 で混ざりやすいパターン

領域	v1	v2
ウィンドウオブジェクト	appWindow グローバル	getCurrentWindow()
ファイルシステム	@tauri-apps/api/fs	@tauri-apps/plugin-fs + プラグイン登録
HTTP	@tauri-apps/api/http	@tauri-apps/plugin-http の fetch()
トレイ	system-tray プラグイン	@tauri-apps/api/tray

特にファイルシステムは罠が多い。v2 では tauri.conf.json に権限を明示的に宣言しなければならない、Rust 側でもプラグインを登録する必要がある。AI が v1 パターンをそのまま提案すると、コンパイルは通るのに、ランタイムでモジュールが見つからず静かに失敗する。

すべての Tauri プロンプトは次の一行で始めている。「**TAURI 2.x を使用 - すべての API を確認すること**」。そして上の表の核心項目をいっしょに貼り付けておく。このガード一行だけで、バージョン混同のほとんどが消えた。

注意: AI が学習したデータには旧バージョンと新バージョンが入り混じっている。進化の速いフレームワークではこの混同が特にひどい。必ず正確なバージョンを明示し、`import` が `package.json` と一致しているかを確認すること。

ここまで4つの落とし穴を別々に見てきた。次節では、これらの落とし穴を **防ぐための道具を3つまとめて整理する**。

9.5 防ぐ方法: プロンプト・レビュー・テストの3点セット

4つの失敗は発生のしかたは異なるが、防ぐ方法はすべて同じ段階を踏む。**着手前に明示**(チェックリスト)、**AI が書いたコードを再点検させ**(レビュー用プロンプト)、**人間が自分で動作を確認する**(テストプロトコル)。3つは同じレイヤーの道具だ。どれか1つが欠けると、残りの2つでは捕まえられない失敗が漏れていく。

この節を本格的に読む前に、短い背景がひとつ役に立つ。

偽の生産性

速いから生産性が高いとは限らない。

Claude は30秒で200行を生成する。前にぐんぐん進んでいる感覚がある。だが、AI が **エッジケースを処理していない** コードを書いているため、あとで状態同期のバグ一つをデバッグするのに7時間かかる。AI は技術的負債が積もる間も、進んでいる感覚を与えてくる。

- ・ **従来の開発:** ゆっくり書いて、速くデバッグする。
- ・ **AI 補助の開発:** 速く書いて、ゆっくりデバッグする。

Mandai の6週間のうち、毎週1日以上を AI の失敗の巻き戻しに費やした。ガードレールなしで AI を使うと純効果がマイナスになるという事実を、次の表が示している。表中の時間値は厳密な合計ではなく、パターンを示す参考だ。

失敗の種類	頻度	主な原因
状態同期の問題	3機能	ライフサイクル仕様の欠落
バージョン混同	8回	API 学習データの混在
エッジケースの盲点	多数の機能	正常系のみテスト
ドメインモデルの逸脱	4機能	一般パターンがドメインルールを上書き

この表が3点セットの出発点だ。3つの道具を順に見ていく。

道具1 — プロンプトチェックリスト

CRUD(作成・読み取り・更新・削除)より複雑な機能には例外なくこのチェックリストで着手する。

状態管理:

- [] フロントエンドとバックエンドの状態同期計画が定義済み
- [] クラッシュ/スリープ時の復旧動作が明示されている
- [] エッジケースを列挙: データなし、処理中断、タイムゾーン

API バージョン管理:

- [] フレームワークのバージョンが明示されている (Tauri 2.x, SvelteKit 2 + Svelte 5 など)
- [] `import` 文が現行バージョンのドキュメントと一致している
- [] 学習データの `deprecated` パターンが含まれていない

ドメイン固有性:

- [] `.cursorrules` にカスタム ID 体系が文書化されている
- [] 非標準のルールが明示的に記録されている
- [] 「一般パターン」を上書きするビジネスルールが列挙されている

テストシナリオ:

- [] 正常系のテストケース
- [] エッジケースシナリオ (空、部分、オーバーフロー)
- [] 失敗復旧シナリオ (ネットワーク、クラッシュ、データ破損)

このチェックリストは 9.1～9.4 で扱った4つの落とし穴にそれぞれ対応している。各項目が空欄になっているということは、その落とし穴の口がそのまま開いているという意味だ。

道具2 — レビュー用プロンプト

チェックリストで着手しても、AI はやはり間違える。だからコードを受け取った後、AI 自身にもう一度レビューさせる。**いつ実行するか?** AI が作った機能を2～3回修正した直後、ただし手動テストの前が最も効率的だ。早すぎるとまだ書いていない部分について評を並べ立てるし、遅すぎるとすでに見つけたバグ以外に拾えるものがない。

この機能の AI 落とし穴をレビューせよ:

状態同期

- アプリの再起動/クラッシュ状況でも状態を安全に復旧できるか?
- ライフサイクル全体でフロントエンドとバックエンドの状態が一貫しているか?
- ユーザーが処理中にアプリをバックグラウンド/スリープにしたらどうなるか?

API バージョンの正確性

- すべての `import` が `[FRAMEWORK_VERSION]` の API を使っているか?
- 旧バージョンに見えるパターンはないか?
- `deprecated` メソッドや古い `import` パスはありますか?

エッジケースのカバレッジ

- データがない/空のときどうなるか?
- 境界条件（深夜0時、月末など）をどう扱うか?
- タイムゾーン、祝日、勤務スケジュールの変動がカバーされているか?

ドメインモデルとの整合

- コードがカスタム ID 体系とルールに従っているか?
- 「一般的なベストプラクティス」がドメイン固有のルールを上書きしていないか?
- `.cursorrules` への準拠を確認

問題が見つかったら、具体的な修正内容と更新したプロンプト案を提案せよ。

レビューテンプレートが Claude の見落としを拾った実例を1つだけ示す。AI が作ったタイマー復元ロジックは、DB に保存された `remaining_minutes` の値をそのまま信用してタイマーを再開していた。問題は、その値がスリープ中の経過時間を反映していないことだ。30分スリープしていたら、実際の残り時間より大きい値が表示される。

レビュープロンプトをかけたら、すぐにこの問題を指摘してきた。「保存値を信用せず、`session.start_at` から経過時間を再計算せよ」という応答だった。9.1 で5回さまよった末にたどり着いた結論とまったく同じだ。

レビューツールが効果を発揮するには、ひとつ前提が必要だ。**AI が書いたコードを人間がある程度読めることだ**。行ごとに精査するのではなく、危険なパターンを認識できればよい。次のようなパターンが見えたら立ち止まって疑う。

- ・ **欠落した `await`:** `invoke(...)` のような非同期関数の前に `await` が抜けていると、結果ではなく Promise オブジェクトが入る。
- ・ **重要な経路にエラー処理がない:** `save_data` が失敗してもすぐに「成功」メッセージを表示するコード。
- ・ **リアクティブな更新を伴わない状態の変更:** オブジェクトを直接書き換えると画面が再描画されないフレームワーク (Svelte など) で、配列の要素を直接書き換えるコード。

この3つはドメイン知識がなくても見抜け、最も頻繁に登場する。AI コードを行単位で全部読まなくても、このパターンに気づける目があるからこそ、レビュー用プロンプトが指摘してくる問題を判断できる。

注意: AI コードのデバッグは、人間が書いたコードのデバッグより難しい。AI はそれらしく見えるが微妙に間違ったロジックを作る。本番に上げる前に捕まえるには、レビュー習慣を必ず作ること。

道具3 — テストプロトコル

チェックリストとレビュー用プロンプトを通過したコードでも、人間が自分で動かしてみる必要がある。検証パターンは3段階で構成する。機能1つあたり13分で十分だ。

段階1: 動作テスト (3分)。ビルドして実行し、基本フローが動くか確認する。動かなければ修正ループに戻る。

段階2: エッジケースの探索 (5分)。チェックリストに沿ってテストする。曜日を変え、タイムゾーンを変え、データを削除し、ボタンを素早く3連打する。

段階3: 状態復旧テスト (5分)。アプリを強制終了して再起動する。データは復元されるか? 中途半端な状態で止まっていないか? もしそうなら、アプリはそれを正しく扱えるか?

Mandai で実際に使ったシナリオはこんな具合だ。段階1ではポモドーロ開始後の30秒カウントダウン確認、タスク完了後の一覧更新確認、マンダラのサブ目標保存の確認。段階2ではポモドーロ中にアプリを最小化して2分後に復元、空タイトルでタスク完了を試みる、OS のタイムゾーンを変更して再起動。段階3ではポモドーロ中の強制終了からの再起動、マンダラ編集集中の強制終了、DB ファイルが破損した状態での再起動。3段階を一貫して適用することで、AI が作ったバグの大半をユーザーより先に拾える。

注意: AI が書いたコードは単独テストでは無事に見えても、実際に使い込むと壊れることが多い。依頼した関数1つだけをテストするのではなく、ユーザーフロー全体をテストすること。

3つの道具の関係

3つの道具は同じ落とし穴を異なるタイミングで防ぐ。

- ・ **チェックリスト** — コードを書く前に、罠の口を狭める。
- ・ **レビュー用プロンプト** — コードが出来上がった直後に、AI 自身に再点検させる。
- ・ **テストプロトコル** — 人間が最終段階で実際の動作を見る。

3つのうちどれか1つを使うだけでも効果はあるが、組み合わせて使うとき落とし穴は最も少なくなる。

9.6 まとめと次章

Mandai を開発する中で、デバッグに相当の時間を費やした。本章で必ず覚えておくべきことは4つだ。

1つ目、AI は4つの失敗を繰り返す。フロントエンドとバックエンドの状態がずれる状態同期問題、古い API と新しい API を混ぜるバージョン混同、正常系だけ気にしてエッジケースを抜かすエッジケース盲点、そしてドメイン固有のルールを無視して一般パターンを当てはめるドメインモデル逸脱。パターンを知っていれば先回りして防げる。

2つ目、防ぐ方法は事前に明示することだ。コードを依頼する前に、ライフサイクル(いつ始まり、止まり、復元されるか)、正確なバージョン、エッジケース一覧、ドメインルールをプロンプトに書き込んでおく。一行の明示が数時間のデバッグを減らす。

3つ目、検証は3点セットで作る。着手前のチェックリスト、書いた直後のレビュー用プロンプト、最後に3段階テスト。1つだけでも効くが、3つを同時に回したとき落とし穴は最も少なくなる。

4つ目、AI で節約した時間はデバッグで戻ってくる。ガードレールなしに AI を使うと、コードは速く積み上がるが、その分だけ後から時間が漏れていく。流れはこう変わるべきだ。

- ・ **従来の開発:** 考える -> コード -> デバッグ
- ・ **AI 補助の開発:** 考える -> 制約 -> プロンプト -> 検証 -> デバッグは少なめ

「制約」の段階を1つ追加することが、デバッグ時間を縮める核心だ。

次章

Mandai の章はこれで終わる。次は **readbooks.ai** だ。2025年に Next.js で作った PDF 翻訳アプリを Tauri 2 で作り直す。今度のループは性格が違う。コードのバグを捕まえるループではなく、**出力の品質**を検証するループだ。別の AI で AI の翻訳結果を検証する方式になる。本章で作った「事前明示+事後検証」の思考枠が、次章では翻訳品質という別形態の出力に対して適用される。

第10章. readbooks.ai —AI出力品質を設計する プロンプトエンジニアリング

Part III —readbooks.ai(第10～11章) Mandaiが「コードの正しさ」を扱ったとすれば、Part IIIは**出力品質**を扱う。AIが36言語に翻訳した結果が正しいかどうかをどう判断するか—この問いがパート全体を導く。第10章はプロンプト設計、第11章は検証パイプラインだ。初読では36言語という規模よりも「AI出力もプロダクトであるなら基準と検証が必要だ」という点を先につかむとよい。

AIの**出力**そのものがプロダクトとなるアプリは、フィードバックの仕方が違う。コードであれば型エラーや実行時例外はすぐに表面化する。しかし翻訳結果は文法的に自然に見えても意味が誤っている場合がある。コンパイラもテストもその違いを捕まえてくれない。だからこの章の核心の問いは機能実装ではない。**翻訳品質をプロンプト (AIに渡す指示文) でどう設計するか**である。

readbooks.aiはPDFを36言語に翻訳する。ところが決定的な制約が一つある。**36言語のうち私が直接読める言語は4つだけだ**。残りの32言語は翻訳結果が正しいかを目で確認できない。この一点が、この章のあらゆる決定を引っ張っていく。レビューア役のAI、自動フォーマット検査、次章の逆翻訳まで—すべて「読めない言語をプロダクト品質で保つには何が必要か」への答えになっている。

プロンプトが間違っていれば一つの言語だけが壊れるのではない。同じプロンプトを使う36言語すべてが同じ形で壊れる。プロンプト品質がそのままプロダクト品質になる。

背景: readbooks.aiは2025年にNext.jsで作ったPDF翻訳アプリだ。現在はサービスを運営していない。本書ではそのソースをTauri 2 + SvelteKit 2 + Svelte 5で再実装した。マイグレーション自体はこの章のテーマではない。この章の核心は、再実装の過程で設計した翻訳プロンプトシステムだ。

この章の流れ

この章は、翻訳プロンプトがどう失敗したかから始まり、品質基準と検証可能な制約へと磨いていく流れだ。

- ・ **10.1 背景** —DeepL一本に依存していた構造を、なぜLLMの組み合わせに変えたのか。
- ・ **10.2 最初のプロンプト** —「とにかく翻訳して」がなぜ失敗するのか。
- ・ **10.3 言語別の分岐** —一つのプロンプトで36言語を扱えない理由。
- ・ **10.4 メタループ** —AIでプロンプト自体を磨く方法。
- ・ **10.5 品質基準** —「良い翻訳」を検査可能なチェックリストに変える。
- ・ **10.6 ループの整理とまとめ**。

学習目標

- ・ AI出力がプロダクトであるとき、開発ループがどう変わるかを理解する。
 - ・ 翻訳品質を決定づける制約条件をどう設計するかを身につける。
 - ・ 言語別プロファイルを作る過程を追体験する。
 - ・ AIを翻訳者・レビュアー・言語専門家の三つの役割に分けて使う方法を学ぶ。
-

10.1 背景

readbooks.aiは当初、DeepL API一つで36言語を翻訳していた。結果は概ね「読める」レベルだった。しかし技術書において「読める」と「意味が正確だ」はまったく別の話だ。

文章が自然でも問題は起きる。コードブロックの説明が間違っていたり、API名をローカライズしてしまったり、コマンドの一部を訳してしまったりすると、読者は原文より混乱する。本書ではその構造をTauri 2 + SvelteKit 2 + Svelte 5で作り直した。

この章の核心の問いはこれだ。**36言語にわたって翻訳品質をどう一貫して保つか。**この問いに答えるには「良いモデルを選ぶ」だけでは足りない。言語別プロファイル、保持すべき技術要素、レビュアー役のプロンプトまで一緒に設計する必要がある。

クラウドとオンデバイスを同じprovider境界へ置く

providerは外部サービスでも利用者の機器内で動く小型モデルでもよい。readbooks.aiはClaude、OpenAI、Gemini、DeepL、デスクトップruntimeを同じtranslate_with_provider Tauri命令の後ろに置き、RustがHTTPか内部推論へ分岐する。UIを作り直さず交換できるが、用語の一貫性には用語集とモデルを識別するcacheが必要だ。

on-device・on-demand・offlineを区別する

- ・ **on-device**：利用者の機器内で推論し、文を翻訳serverへ送らない。

- **on-demand** : 常駐させず必要時に取得またはロードする。
- **offline** : モデル準備後はnetworkなしで推論できる。

On-deviceとGGUFを選ぶとRustがapp process内でllama.cppを実行する。初回翻訳でロードし、次から同じinstanceを使う。localhost serverやAPI keyは不要だ。

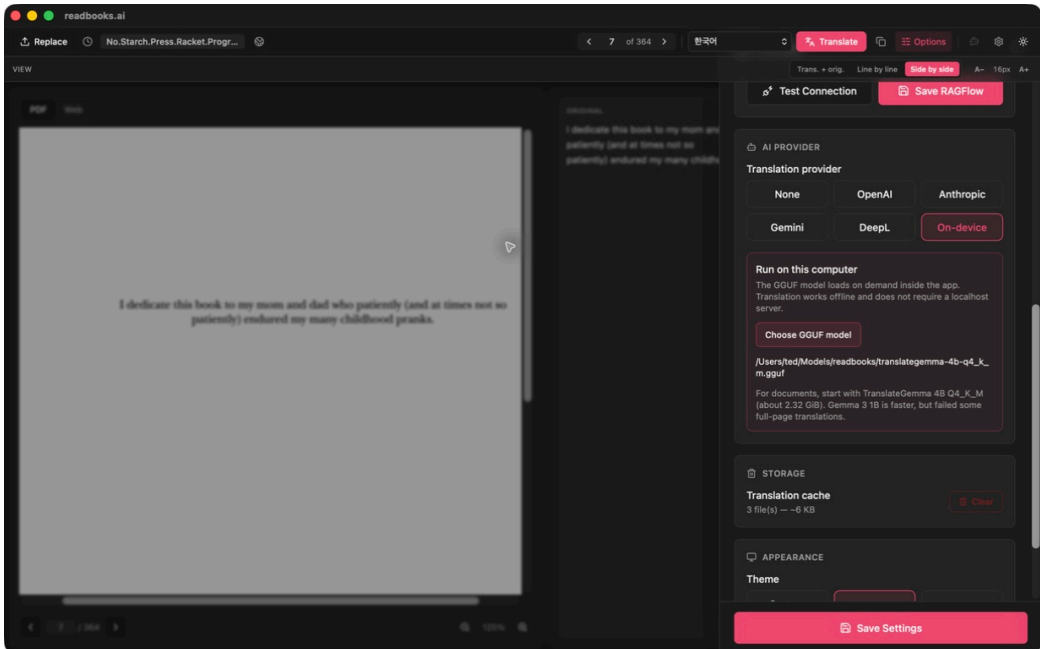


図10-2 TranslateGemma 4B Q4_K_M GGUFの選択。必要時にアプリ内でロードする。

Svelte -> invoke -> Tauri Rust: GGUFをロード・再利用
-> llama.cpp -> 結果 + DuckDB cache

分岐の概略は次のとおりである。

```
match provider.as_str() {
    "openai" => do_translate_openai(/* ... */).await,
    "deepl" => do_translate_deepl(/* ... */).await,
    "on_device" => {
        let path = model.ok_or("Settings에서 GGUF 모델을 선택하세요");
    }
}
```

```

        on_device::translate(path, system_prompt, text,
                             target_lang).await
    }
    _ => Err("지원하지 않는 Provider".into()),
}

```

on_device::translateは重い推論をspawn_blockingへ送り、非同期executorを止めない。GGUFのchat templateを使い、ないGemmaにはuser/model turnを構成する。contextは4096 token、生成は最大512 tokenで、長すぎるpageは黙って切らず分割を求める。

小型モデルは用途別に検証する

文書翻訳の記載した出発点は**TranslateGemma 4B Q4_K_M**である。text-only GGUFは2.32GiB、検証SHA-256はe7afd56008be424d2ba36ed3b371d40039ae15fa467e8cf71442570d842b1119。

Gitやappへ同梱せず、利用者がGemma条件とhashを確認して取り込む。

著者の検査ではGemma 3 270Mが英語をそのまま返す場合があり、1Bは短い会話を訳せてもPDFでは混合文字を出した。TranslateGemma 4Bはコーヒーの質問と一文の献辞を韓国語へ訳した。これはpipeline確認、短い会話、文書翻訳を区別すべき観測であり、多言語全体のbenchmarkではない。

Rust testはハングルの有無だけでなく「コーヒー」の意味も確認する。appは韓国語要求なのにハングルがない結果を拒否するが、これで自然さや意味の正確さは保証できない。長文は第11章の評価と適切な大型・クラウドモデルへの切替が必要だ。

Ollamaは探索や取得に便利だが実行依存ではなく、llama-cpp-2が直接開く。検査したOllama vision blobにはllama.cppが必要とするgemma3.attention.layer_norm_rms_epsilonがなく、text-only変換版を別途取得してhash確認した。拡張子だけで互換とせず、使用runtimeでロードと生成を試す。

小型モデルは長いpage、低資源言語対、code・表・脚注、用語集と長い文脈の両立で苦戦し得る。privacy・費用・速度・実測品質に応じて選べるようcloudも残す。

製品で取得するなら容量とlicenseを先に表示し、一時fileでSHA-256を検証してからApplication Supportへ移す。cacheにはmodel ID・版・hashを含める。現例の

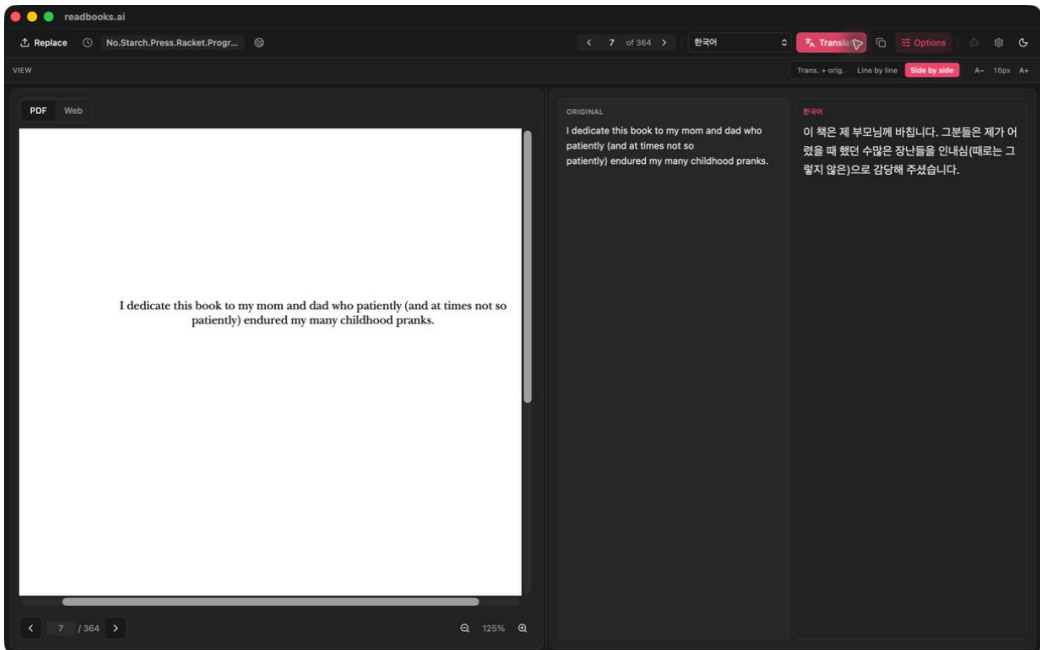


図10-3 アプリ内TranslateGemmaでPDFの献辞を韓国語へ翻訳する。

DuckDB keyはPDF ID、page、目標言語、原文hash、provider、model pathである。別pathは区別できるが同じpathの中身交換は検出できないので、製品では検証済みcontent hashも追加する。旧localStorage結果は翻訳として読み出さない。

難しいのは品質判断である。翻訳者に任せる範囲と変更禁止の技術要素をpromptで分ける。

10.2 最初のプロンプト—すべてを取りこぼした初稿

最初のプロンプトは短かった。翻訳作業なのだから、入力テキストと目標言語を渡せばモデルが適切に処理してくれるはずだと思っていた。私が最初にしたプロンプトはこの一行だった。

「次の英語テキストを{言語}に翻訳してください。翻訳だけを出力してください。説明なしで。」

36言語で実行したとき、結果は表面的にはまともに見えた。段落は自然で、ほとんどの文は目標言語に置き換わっていた。ところが実際の出力を開いてみると、技術文書で最も重要な部分が繰り返し壊れていた。

- ・コードブロック内の変数名が翻訳された (`const userName -> const ユーザー名`)
- ・引用符が変わった (`"foo" -> 「foo」`)
- ・数値フォーマットがローカライズされた (`1,000 -> 1.000`)

なぜAIはこう動くのか

原因は一つだ。**指示がなければAIは一般的な翻訳の慣例に従う**。一般的な翻訳の慣例では、すべてのテキストを自然に訳すのが正解だ。文学翻訳ならそれで正しい。技術文書では誤りになる。ファイル名・コード・コマンド・API名は、読者がそのままコピーして実行しなければならない情報だからだ。

この原理はこの章全体を貫いている。プロンプトとは「AIに何をしろ」と命じる道具ではなく、**AIがデフォルトで従おうとする慣例のうち、どこで止まるかを伝える道具**なのだ。技術文書の翻訳プロンプトは「自然に翻訳しなさい」から始めてはいけない。まず保持すべき構造を定義し、その範囲の中でだけ文を変えるよう求める必要がある。

実際の失敗例—ブロックに5つの誤り

次の英語の抜粋を、最初のプロンプトで日本語訳にかけてみた。

原文: ~markdown ## Chapter 3: Database Setup Set up the database.config.js file.

```
const config = { host: "localhost", port: 5432, database:
"readbooks" };
```

Connection timeout is 30,000ms. ~

出てきた訳: ~markdown ## 第3章: データベース設定 データベース.設定.js ファイルを構成してください:

```
const 設定 = { ホスト: "localhost", ポート: 5432, データベース:
  "readbooks" };
```

接続タイムアウトは30.000msに設定されます。~

小さなコードブロック一つで5つの問題が同時に発生した。ファイル名・変数名・JSONキーがすべて翻訳され、数値30,000がヨーロッパ式の30.000に変わり、結果としてコードのシンタックスハイライトまで壊れた。

注意点: シンプルな「これを翻訳して」プロンプトは**すべてのテキストを翻訳対象として扱う**。技術文書には必ずそのまま残すべき区間(コード、URL、コマンド)がある。制約条件を明示しなければ、AIはすべてをローカライズする。

1回目の修正—保持すべき範囲を先に固定する

最初の修正は「もっと良い品質にしろ」という抽象的な要求ではなかった。コードブロック、インラインコード、数値、URLのように保持すべき対象をまず固定し、それ以外の散文だけを翻訳するよう範囲を分けた。要は5行の**システムプロンプト**(すべての要求の前に常に付与される、AIの役割とルールを定める指示文)だ。

あなたは技術文書を翻訳します。ルール:

- ```` のfencedコードブロック内の内容は翻訳しないでください。そのままコピーしてください。
- `インラインコード` は翻訳しないでください —原文のまま維持してください。
- 引用符、数値フォーマット、日付はソースのとおり保存してください。
- URL、メール、DOI: 原文のまま維持してください。
- 翻訳だけを出力してください。説明や前置きなしで。

実行してみると、コードブロックの保持は機能した。しかし新しい問題が生じた。**引用符の中の英語の文がソース言語のまま残った**。AIが「引用符を保持する」を「引用符の中のすべてを保持する」と解釈したからだ。

2回目の修正—「記号」と「内容」を分離する

私が望んだのはむしろその逆だ。引用記号は残すべきだが、引用された内容は通常の文と同じように翻訳されるべきだ。そこで次の2行を追加した。

- 句読記号 (" , ' «» „" , など) はそのまま保存します。

ただし引用符の中のテキストは翻訳します。

– 数値フォーマット（カンマ、小数点）はソースのとおりに保存します。

同じ文をもう一度回してみる。

原文: The system returns {"status": "success", "message": "File uploaded"} **日本語:** システムは{"status": "success", "message": "ファイルがアップロードされました"}を返します。

JSONの構造とキーはそのまま保たれ、人間が読むメッセージだけが訳された。まさに望んでいた挙動だ。

ここで得た教訓はシンプルだ。「**Xを変更するな**」という制約は、**Xの正確な範囲も一緒に明示しなければならない**。範囲が曖昧だとAIは独自の解釈を作り、その解釈はたいてい一般的な翻訳の慣例に従う。

10.3 言語別の分岐—「36言語のための一つのプロンプト」という幻想

一つの言語(韓国語)で制約を整えるのにも修正が2回必要だった。では36言語を同時に扱うとしたらどうなるか。最初はプロンプト一つですべてを処理できと思っていた。実際にやってみると違った。同じ英語の文を翻訳しても、ドイツ語・ロシア語・アラビア語・中国語はそれぞれ別の壊れ方をする。言語群ごとに制約条件を別々に決める必要があった。

まず、どんな種類の問題が言語別に起きるかを整理しておく。

- ・ 韓国語/日本語: 英語の固有名詞は慣例として英語のまま維持(例: “React”は”React”のまま)
- ・ ドイツ語: 名詞の先頭文字を大文字にする規則をLLMが一貫して守れない。
- ・ ロシア語: 定冠詞のない言語なのに、LLMが原文にない指示語(「この」「その」)を挿入する。
- ・ アラビア語/ヘブライ語: 右から左に読む言語(RTL)なのでmarkdownレンダリングに影響する。
- ・ 中国語: 簡体字と繁体字で、同じ英単語が違う用語に訳されるべき。

たとえば英語の“The react component handles state management through hooks.”をドイツ語に翻訳すると、AIが react, komponente, hooks をすべて小文字で出力することがある。ドイツ語の正書法ではなく、英語の大文字規則をそのまま引きずったのだ。ロシア語の場合は原文にない指示語(「ЭТОТ」)が紛れ込み、「特定のフック」という意味が新たに生まれてしまう。自然さを高めようとして原文の意味を変えてしまうのだ。

注意点: 言語群でまとめて考えると、個別言語のレベルでルールが崩れる。ポルトガル語とスペイン語のように近い言語でも、技術用語の好み異なる。

解決—基本プロンプト + 言語別プロファイル

解決策は、システムプロンプト一つに**言語別の追加ブロック**を貼り付ける方式だった。共通ルールは基本プロンプトに置き、言語ごとに変わる慣例だけを短いプロファイルとして分離する。

```
const base = `あなたは技術文書を翻訳します。ルール: ...`;

const languageProfiles: Record<string, string> = {
  ko: `ー 英語の固有名詞(React, npm, API)は英語のまま維持`,
  de: `ー ドイツ語の名詞は大文字化必須\nー 複合名詞は一語のまま維持`,
  ar: `ー RTLテキスト方向を適用。コードブロックはLTRを維持`,
};
```

各プロファイルは10行以下に抑えた。ルールが長くなるとモデルが重要な制約を取りこぼしたり、矛盾する指示を同時に守ろうとしてしまう。

実際のプロファイルはどんな形か

上の一行ずつの例だけではイメージしにくい。実際に使ったドイツ語プロファイルをすべて示すと次のようになる。

ドイツ語 (de) プロファイル:

- ー 名詞の先頭文字は常に大文字で表記する。英語の外来語も名詞として使う場合は大文字(Hook -> Hook, component -> Komponente)。
- ー 英語の固有名詞(React, npm, API)はそのまま維持するが、名詞として使うときは大文字規則を適用。

- 複合名詞は一語にまとめて書く (Datenbankverbindung、ハイフン不可)。
- 敬体 (Sie/Ihnen) を使用。du形は禁止。
- "you" は文脈に応じてSieまたは受動態に。

言語ごとに文体、禁止する誤訳、数字・単位、固有名詞、出力形式を記載し、誤例と正例を一文ずつ添える。五行の例はこの形である。その言語の検討者が確認した規則だけを使い、36言語すべての精度を保証したと考える。

言語プロファイルの構築: AIをリサーチアシスタントとして使う

36言語を一人で調べる代わりに、Claudeをドメイン専門家のように活用した。要点は「プロファイルを作ってくれ」と丸投げしなかったことだ。まず自分が必要な基準を決めた(タイポグラフィ、技術用語の慣例、フォーマットの違い)。次に言語ごとによく崩れる箇所を尋ね、返ってきた回答から実際のプロンプトに入れる項目だけを選んだ。

この方法で2時間以内に36言語のプロファイルを整理できた。ただし、結果をそのまま信じはしなかった。言語ごとに短いサンプルを直接回し、コードブロック・数値・用語が正しく保たれるかを確認した。**AIはコード作成だけでなく、ドメイン知識の調査にも使える。だが最終的な基準を決め、結果を検証する仕事は、依然としてプロダクトを作る人がやらなければならない。**

注意点: 複雑な言語プロファイルはルール同士の衝突を起こす。「敬体を使う」と「自然に翻訳する」を一緒に書くと、AIは流れより敬体を選ぶ。ルールには優先順位をつける。

10.4 AIと一緒にプロンプトを設計する—メタループ

この章で最も重要な内容がここにある。翻訳にだけAIを使うのではなく、**AIでプロンプト自体を設計する**。一度作ったプロンプトをそのまま使い続けるのではなく、失敗事例を集めて磨き続ける。

メタプロンプトとは

まず用語を整理する。**メタプロンプト**とは「ほかのプロンプトを作る、もしくは直すためのプロンプト」だ。普通のプロンプトはAIに仕事をさせる(「この文を翻訳して」)。メタプロンプトはAIにプロンプト自体について仕事をさせる(「この翻訳プロンプトの弱点を見つけて」)。同じモデルでも、入力が変わると出力の性格はまったく変わる。使い方は2レベルに分かれる。

1. **レベル1**: 良い翻訳を生むプロンプトを書く。
2. **レベル2**: より良い翻訳プロンプトを作るプロンプト(メタプロンプト)を書く。

レベル2の形はこうだ。

「あなたはプロンプトエンジニアリングの専門家です。技術文書向けのこの翻訳プロンプトを改善する必要があります。現在のバージョン: [現在のプロンプトのテキスト] 私が直面している問題: [具体的な失敗例] あなたのタスク: プロンプトへの具体的な改善案を3つ提案してください。合計200語以内で。」

36言語を一つひとつ手でテストするより、このメタプロンプトのパターンでプロンプト自体を反復改善するほうがずっと速い。

同じAIをレビュアーとして再利用する

同じモデルに役割を変えて尋ねることもできる。

「あなたはこれから次のプロンプトを受け取る翻訳AIになります。読んで、**指示が曖昧な箇所3つ**を見つけてください。修正はしないでください。」

Claudeが見つけた曖昧な3点は、すべて実際の翻訳出力に影響を与えていた。たとえば「数値フォーマットの保存」が日付も含むのか、「インラインコード」がバッククォート1個か2個かが不明確だった。これらを反映してプロンプトを磨いたら品質が上がった。**同じAIモデルでも、役割を変えれば違う視点で見えてくれる。**

デュアルエージェント—翻訳者とレビュアーの分離

ここで**エージェント**という言葉を使うが、本書では「特定の役割を与えられて一つの仕事だけをするAIインスタンス」という狭い意味で使う。同じモデル(たとえばClaude)でも、どのプロンプトで呼び出すかによって違うエージェントになる。

核となるパターンはシンプルだ。一つのエージェントの出力を、別のエージェントにレビューさせる。

- ・ **Agent 1(翻訳者)**: 翻訳プロンプトに従って出力を生成する。
- ・ **Agent 2(レビュアー)**: 同じ出力を受け取り、コード保持・用語の一貫性・フォーマット保持・意味保持を検査する。問題があればリストとして出力し、問題がなければ‘No issues’を返す。

たとえばドイツ語訳でファイル名 `app.config.js` が `anwendungs.konfig.js` に翻訳されてしまったら、レビュアーが「ファイル名が翻訳された—コード保持ルール違反」と指摘してくれる。単独の翻訳では見逃される誤りを捕まえられる。32の読めない言葉に対して、このパターンはとくに有効だった。

失敗したメタプロンプトたち

すべてのメタプロンプトのアプローチが成功したわけではない。漠然とした要求は漠然とした答えしか呼ばない。

- ・ 「もっと良くして」 -> 「もっと具体的にしてください」のような一般論しか返ってこない。**具体的な失敗事例を一緒に渡す**必要がある。
- ・ 「このプロンプトにスコアをつけて」 -> 「8/10」のような数字は出るが、実際の性能と無関係だ。**スコアの代わりにテスト事例から失敗を特定するよう求めなければならない。**

注意: メタプロンプトも普通のプロンプトと同じく具体的な基準を必要とする。

プロンプトのバージョン管理

プロンプトを何度か直すうちに、変更履歴の管理が必要になった。gitのように管理した。

```
prompts/
  v1.0-base-naive.md          # 「このテキストを翻訳して」
  v1.1-add-code-preservation.md
  v1.2-fix-quote-handling.md
  v2.0-language-profiles.md
  v2.1-ambiguity-fixes.md
  current.md -> v2.1-ambiguity-fixes.md
```

各バージョンには、何が違って、なぜ変わったかの記録を残した。おかげで**プロンプトのバージョンを比較**でき、改善のつもりが退化だったときにすぐ巻き戻せた。実際、Claudeが自然さを高めるために「翻訳でクリエイティブになれ」を追加しようと提案したことがあるが、適用したら技術用語がばらつき出したのですぐに巻き戻した事例がある。

10.5 翻訳品質基準—AIに伝える方法

「翻訳品質を高くする」というような宣言は通用しない。AIは具体的な指示を受けて動く。**検査可能な項目に細かく分解**しなければならない。

最終プロンプトから抜き出した検査項目はこれだ。

品質基準（出力するな —内部検査用）:

1. 意味保存: 文が省略されたり改ざんされたりしていない。
2. 用語の一貫性: 同じ原文用語 -> 同じ訳語を文書全体で。
3. トーンの一致: 技術文書はフォーマルに、会話調ではない。
4. AIメタトーク禁止: 「翻訳は次のとおりです」などは禁止。
5. Markdown構造を同一に: 同じ見出しレベル、リスト、コードフェンス。

5項目を「内部検査」としてまとめると出力が変わる。モデルが実際に内部でどんな手順を踏むかは分からないが、プロンプトにチェックリストを入れると結果がより一貫して出る傾向がある。

各基準がどの失敗から生まれたか

5つの基準はすべて、実際の失敗事例から作られた。代表的な例を2つだけ見る。

- ・ **意味保存**: “The function returns null if no user is found.”が「ユーザーが見つからない場合、関数はnullを探します。」と訳された。「返す」が「探す」に変わってしまった。「正確に翻訳しろ」だけでは足りず、**意味保存を明示的に要求する**必要がある。
- ・ **トーン的一致**: 英語の“Execute the following command”がフランス語であまりにカジュアルな tu 形(“Va exécuter…”)になったことがある。敬体の vous を明示しないと、AIは会話調をデフォルトで使う。

用語の一貫性、AIメタトークの遮断、Markdown構造の保持も、似たような事例が積み重なって生まれたルールだ。

ドメイン別の追加基準

基本の5つでカバーできないエッジケースがあったため、短い付録を追加した。APIエンドポイントのパス、エラーコード、設定キー、コマンドラインフラグ、ファイル拡張子—こうした識別子は決して翻訳しないように明示した。たとえばロシア語の翻訳が `.config.json` を `.конфиг.json` にローカライズしようとした事例があった。ロシア語としては論理的な訳だが、コード例がすべて壊れる。

注意: 品質基準どうしが衝突することがある。代表例が「意味保存」vs「自然な表現」だ。優先順位をあらかじめ決めておく。**技術的正確性が第一、自然さが第二だ。**

検証パイプライン—自動検査 + 人間レビュー10%

品質基準だけでは足りない。出力が本当にその基準を守っているかを確認するステップが必要だ。次の4ステップを順に通すようにした。

ステップ1 — AI翻訳

品質基準5項目を含むシステムプロンプトで翻訳を生成

↓
v

ステップ2 — 自動フォーマット検査

原文 vs 翻訳の見出し数、コードブロック数、リンク数を比較

↓
v

ステップ3 — 用語の一貫性検査

同じ原文用語が文書全体で同じ訳語に置き換わっているか



ステップ4 —人間レビュー（全体の10%）

文化的ニュアンス、非標準の技術用語など、自動では拾えない項目

ステップ4がとくに重要だった。AIが作った品質基準で問題の80%は拾えたが、文化的ニュアンスやその言語で標準ではない技術用語は人間がレビューしないと捕まらない。この4ステップの流れが次の第11章の検証パイプラインへとつながっていく。

10.6 この章のループの整理

これまでの反復過程を一表にまとめるとこうなる。

ラウンド	内容	結果
1	初期の制約条件 (コードブロック、URL)	2パス目が必要 (引用符)
2	引用符に対する「記号 vs 内容」の分離	完了
3	言語プロファイルの設計	自分主導、AIが補助
4	言語専門家としてのClaude	36言語のプロファイルを生成
5	Claudeをプロンプトレビューアーとして再利用	3つの曖昧さを発見
6	自己リライト実験	コスト対効果が低く廃棄
7	Viteの?rawで.mdを文字列として読む	一度で解決

サイドノート—ラウンド6のコスト分析: プロンプトに「5項目がすべて満たされるまで書き直せ」を追加する実験もやってみた。トークン使用量がほぼ倍増したのに、品質改善の幅はごくわずかだった。内部チェックリスト方式はトークンを半分しか使わずに、品質改善効果の大部分を出してくれる。

補足—Vite文字列インポート: `import prompt from "./prompt.md?raw";` は内容を文字列で読む。秘密を入れるとバンドルへ含まれ得るため記載しない。TypeScriptが認識しない場合はVite型宣言を確認する。Markdown文字列の取得と安全なHTML描画は別処理である。

繰り返し現れた3つのパターン

すべてのループで3つのパターンが繰り返し現れた。

1. **制約の具体性**: 「うまく翻訳しろ」のような漠然とした指示は品質がばらついた。「コードを正確に保持しろ」のような具体的な制約を与えると結果がずっと安定した。
2. **AIの役割切り替え**: 同じAIを翻訳者・レビュアー・言語専門家など別の役割で使うと、それぞれ違う視点の結果が得られた。
3. **反復改善**: どんなプロンプトも最初から完璧ではなかった。体系的にテストして直す過程を経てこそ品質が上がる。

この3つは翻訳プロンプトに限らない。精緻な結果が必要なAI作業ならどこにでも当てはまる。

核となるメッセージ: AIはコードを書いてくれる道具だけではない。**レビュアー・専門家・実験パートナー**としても使える。状況に応じて役割を切り替えて使うのが、この章の核心だ。

10.7 次の章

プロンプトが堅牢でも一つの問題が残る。**出力が本当に正確かどうか、実際に検証できるのか?** 36言語のうち私は3〜4言語しか読めない。残りの32言語は目では確認できない。次の章はその問題を扱う。**別のAIで逆翻訳し、自動でスコアをつけるパイプライン**だ。

まとめ

この章は**プロンプトエンジニアリングを人間とAIの協業プロセス**として扱う。要点は一つだ。プロンプトを一度書いて捨てる文ではなく、**継続的に管理する品質システム**として扱うべきだ。

この章で必ず覚えておくこと:

- ・ **制約の具体性:** AIは言葉を字面どおりに受け取る。「引用符を保持」と「引用符の記号は保持するが内容は翻訳する」はまったく違う結果を生む。
- ・ **言語別プロファイル:** 一つのプロンプトで36言語すべてを処理することはできない。言語ごとに別のルールが必要だ。
- ・ **メタプロンプティング:** AIでAI用のプロンプトをより良く作る。同じモデルを翻訳者・レビュアー・言語専門家の役割に分けて活用できる。
- ・ **コードとしての品質基準:** 「高品質」のような漠然とした目標ではなく、検証可能な5つの条件に分ける。AIは主観的な目標より、明確なチェックリストのほうがずっとよく従う。

これをコードで実装するには2つが必要だ。**Providerの抽象化**(複数の翻訳サービスを一つのインターフェースにまとめる)、そして**プロンプト管理**(バージョン管理、部品としての組み立て、自動テスト)。

実用的には次の5つを覚えておけばよい。

1. **シンプルに始めて制約を追加:** 基本の翻訳から始め、問題が出るたびに具体的なルールを一つずつ足す。
2. **AIを複数の専門家として活用:** 同じモデルでも役割(翻訳者/レビュアー/専門家)によって違う視点を与えてくれる。
3. **対象言語ごとのテスト:** 出力言語を自分で読めないなら、自動検証システムを用意しておく。
4. **プロンプトのバージョン管理:** コードと同じく変更履歴を追跡し、以前のバージョンに巻き戻せるようにする。
5. **計測とモニタリング:** 実運用中にプロンプトのパフォーマンスを追跡し、その後の改善に活かす。

次の章のプレビュー: 出力言語を読めないとき、翻訳品質をどう検証するか。逆翻訳パイプラインで自動的に翻訳品質スコアをつける方法を扱う。

第11章 翻訳の検証— AIの出力をどう信頼するか

readbooks.ai は36言語をサポートしている。そのうち32言語は、ほとんどの開発者が自分では読めない言語だ。翻訳が自然か、用語は合っているか、段落が抜けていないか、目で確認できないとなれば、品質管理は別のやり方で設計する必要がある。

プロンプトはよく書けているし、出力もそれらしく見える。それでも、どこが間違っているかを知る手段がない。この問題は翻訳に限らない。AIが人の追いつけない速度で専門コンテンツを量産する分野では、どこでも同じ状況が起きる。

解決策は **二つめのAIを使う検証パイプライン** だ。まず、実際によく起きる失敗パターンをリストアップする。次に、それぞれの失敗が実際に起きているかをAIに自動で確認させる。

完璧な方法ではない。あらかじめ想定した失敗パターンしか捕まえられず、想定外のエラーは依然として人が直接見る必要がある。だからこそ検証は、生成のステップに紛れ込ませず別ステップとして切り出す。コードレビューアを作成者から分けるのと同じ理由だ。同じ人が書いてレビューすれば、自分が見落とした箇所は同じように見落としてしまう。生成AIが作った結果をそのまま信用するのではなく、独立した検証プロセスを通過したものだけを製品として出す。

この章の流れ

大きなステップだけ見れば四つだ。

1. **問題を定義する** — 「それらしいのに間違っている」翻訳の四つの形。
2. **逆翻訳を一度やってみる** — 検証のいちばん単純な形。
3. **三つの軸へ広げる** — 意味・長さ・用語に分けて見る。
4. **ユーザーのフィードバックを再び集める** — 自動検証が取り逃したものをプロンプトに戻す。

運用フェーズの細部(合成テストケースの調整、語族ごとの回帰テスト、カナリアリリース)は章の後ろで短くまとめるにとどめ、最初の通読では上の四つのステップを追うことに集中する。

学習目標

- ・ AI翻訳の結果を自分で読めないとき、検証がなぜ難しいのか、どう取り組みればよいかを理解する。
- ・ 逆翻訳(訳文をもう一度元の言語に戻して比較する方法)で検証する流れと、スコアの基準の置き方を学ぶ。
- ・ 36言語を自動でテストする全体像をつかむ。
- ・ ユーザーフィードバックをプロンプト改善に戻すパターンを把握する。

この章で作る最小の成果物: `verify_translation` という Tauri command を一つと、三つの軸のスコア(意味・長さ・用語)を表示する画面を一つ。

11.1 問題—「それらしいのに間違っている」翻訳

LLMが翻訳でやってしまういちばん厄介なミスは、文章は自然なのに意味が間違っているケースだ。見た目はきれいなので、ユーザーもなかなか気づかない。

技術文書では特に危ない。文章が滑らかに読めても、コードの説明やAPI名が間違っていれば、読者はそのまま誤った手順を踏むことになる。

よく見かける失敗パターンは四つある。

- ・ **欠落**: 段落から文がまるごと抜ける。たとえば、データベース接続プールを説明する二文のうち、二つめの文がそっくり消えてしまう。
- ・ **でっち上げ (hallucination)**: 原文になかった例が追加される。英語の原文が `fetch API` しか触れていないのに、スペイン語の訳に `XMLHttpRequest` が紛れ込む。
- ・ **用語のぶれ**: 同じ単語を文書の前半では「コンポーネント」、後半では「構成要素」と訳し分けてしまう。読者は二つが同じ概念かどうか確信が持てなくなる。
- ・ **文体の崩壊**: 技術文書の途中で突然カジュアルな話し言葉が混ざる。きちんとした文体で書かれていたガイドに、急に親しげな口語が飛び出すような感じだ。

このうち **欠落**と**でっち上げ** は、原文を並べて見ないと利用者にはまず気づけない。

懸念: この種のエラーはコンパイルエラーのようにすぐには表に出ない。誰かが気づくまで、数千人にそのまま届いてしまう。

11.2 逆翻訳—手作業で一度やってみる

最初の戦略は古典的なものだ。訳した結果をもう一度元の言語に戻し、原文と比べる。対象言語が読めなくても、逆翻訳の結果は英語に戻ってくるので直接見比べられる。

まずコードなしで、手で一度なぞってみよう。

1. 英語の原文を一段落書く。例: “Use the fetch API to make HTTP requests.”
2. これをスペイン語に訳す(生成AIに任せる)。結果: “Use the fetch API y el método XMLHttpRequest para hacer solicitudes HTTP.”
3. そのスペイン語をもう一度英語に訳す(検証AIに任せる)。結果: “Use the fetch API and XMLHttpRequest method to make HTTP requests.”

4. 1番と3番を人の目で比べる。XMLHttpRequest がどこから出てきたのかすぐに見える。

要は、**ステップ2**でどの言語に行ったかわからなくても、**ステップ4**では英語どうして**比較する**という点だ。36言語を全部読めなくても検証が成り立つ理由はここにある。

ただし、これも完全な方法ではない。逆翻訳の過程で誤りが隠れてしまうこともある。だからこの方法を自動化しつつ、スコアの基準も同時に整える必要がある。

11.3 検証 command を作る—いちばん単純な骨格

ここから上の流れを Tauri command に移していく。最初はできあがった品質評価器ではなく、いちばん小さな骨格をまず手に入れたかった。原文と訳文を入れたら英語に戻して、スコアを一つ返す構造だ。

プロンプト:

「Translation 検証 command。入力: source(英語)、translation、target language。内部で英語に back-translate。source との類似度スコア。translate_with_provider command を再利用。骨格だけ、10行未満で。」

Claude が返してきた最初の骨格は、verify_translation という Tauri command を作り、その中で第10章で作った translate_with_provider を再び呼び出して訳文を英語に戻す。そして原文と比較したスコアを一つ返す。

第10章の translate_with_provider は、「どの provider(OpenAI/Anthropic/…)とモデルで、どの言語からどの言語へ訳すか」を引数で受け取る汎用関数だった。第11章ではこの関数を **二回呼び出す**: 一回は原文 → 対象言語、もう一回は対象言語 → 英語。同じ関数を方向だけ変えて二度使うことになる。

11.4 単スコアの限界—BLEU と cosine 類似度

骨格が動いたところで、次の課題はスコアの部分になった。「似ている」をどんな数字で表すか。

最初は BLEU(翻訳品質を単語列の重なりで測る古典的な指標)を試したが、**逆翻訳の検証には合わなかった**。BLEU は同じ単語の並びがどれだけ重なるかを数えるので、意味が同じでも違う単語で言い換えられているとスコアが低くなる。¹

そこで方向を変えて、**意味的類似度スコアを返す API** を使うことにした。原文と逆翻訳をそれぞれ埋め込みAPIに入れると、二つの文が意味的にどれだけ近いかを0~1の数字で返してくれる。意味が近いほど1に寄った値になる。

これで決まりなら楽だが、この方法にも限界がある。意味類似度は **長さの違いに鈍感**だ。訳文から段落が一つそっくり抜けていても、残った内容が原文に近ければ0.95以上が出てしまう。

実際に、React コンポーネントのライフサイクルを説明する英語段落から、“mounting”の説明をまるごと落としたドイツ語訳を作ってテストしてみた。欠けたままで逆翻訳しても、類似度は0.95と出た。**単スコアでは足りない**ことがここでもわかる。

11.5 三つの軸に分ける

翻訳品質を一つの数字で表すのは難しい。意味は近くても段落が抜けていることはあるし、長さは似ていても核となる用語が違うこともある。だから検証の基準を **三つの軸**に分けた。

- ・ **意味的類似度**: 埋め込みAPIが返す0~1のスコア。
- ・ **長さ比**: 訳文の単語数 ÷ 原文の単語数。普通は0.85~1.2の範囲に収まる。
- ・ **用語の一貫性**: 主要な用語(固有名詞、API名)の頻度がどれだけ一致しているか。

それぞれの指標を別関数として実装し、どれか一つでも基準を外れたら、その訳文にフラグ(エラー表示)を立てる。

¹BLEU は本来、翻訳モデルどうしを同じ正解文にどれだけ近く再現できるかで比べるための指標であり、「意味は同じで単語は違う」逆翻訳には向かない。

結果型をフロントエンドと揃える

フロントエンドが検証結果を受け取って画面に出すには、結果の形を両側で同じに定義しておく必要がある。

- ・ TypeScript 側には `VerifyReport` 型を作る。意味スコア、長さ比、用語一貫性、フラグ配列を持たせる。
- ・ Rust 側には同じ形の構造体を作る。そして `serde::Serialize` を付ける。

`serde::Serialize` を付ける理由は単純だ。Rust の構造体はそのまま JavaScript に渡せない。Tauri は構造体をいったん JSON に変換してからフロントエンドへ送る。流れは次の三行で整理できる。

1. Rust が `VerifyReport` 構造体を作る。
2. `serde` がそれを JSON 文字列にシリアル化する。
3. `invoke('verify_translation', ...)` の戻り値として、JavaScript 側に `VerifyReport` オブジェクトが届く。

確認すべき出力: フロントエンドが受け取ったオブジェクトの `flags` 配列に、どのキーワードが入っているかが鍵となるシグナルだ。空なら `GOOD`、一つか二つあれば `REVIEW`、複数あれば `FAIL` として扱う。

三つの指標で、欠落エラーの80%は捕まえられた。ただ、でっち上げのエラーはすり抜けてしまった。`flags` 配列に枠は用意したものの、検知ロジックはまだなかった。**検証パイプラインは最初から完成しない**。失敗事例に出会うたびに指標を一つずつ足しながら強くなっていく。

懸念: 長さ比は言語によって誤検知を出す。ドイツ語は複合語のせいで単語数が減るし、インドネシア語は助詞が増える。言語ごとに許容範囲を別に調整する必要がある。

でっち上げ検知を追加する

でっち上げとは、原文になかった内容が訳文に紛れ込むエラーだ。意味類似度と長さ比だけでは捕まえにくい。

検知方法は単純だ。LLM に、原文と訳文それぞれから固有名詞・製品名・API識別子を一行一つで抜き出してもらう。その上で両者を比較し、**訳文にだけあって原文にない単語**をでっち上げ候補として扱う。完璧ではないが、十分に実用的だ。

11.1 で挙げた XMLHttpRequest のケースは、この方法でそのまま捕まる。

三段階バッジに整理

しきい値を決めるのも試行錯誤だった。最初は厳しすぎて、すべてのページに「この翻訳は怪しいです」のバッジが出てしまった。緩めになると今度はほとんど発動しなくなった。最終的に **三段階バッジ** に落ち着いた。

- ・ **GOOD**: すべての指標が範囲内。
- ・ **REVIEW**: 一つの指標が警告範囲。
- ・ **FAIL**: 二つ以上の指標が警告範囲。

三段階に分けた後、誤検知への不満は減った。**自動検証ですべてのエラーを捕まえることはできない**。現実的な目標は、人が見るべき訳文とそうでない訳文を分けることに置くのがよい。

11.6 運用しながら磨いていく話

ここまでが、最初の通読で押さえておきたい内容だ。この節は、実際のサービスへ持っていくときに追加で踏むステップを軽く触れる。深入りはせず、「こういうステップもある」くらいの感覚で読めばよい。

- ・ **合成テストでしきい値を補正**: AI に「一つの軸だけ問題のあるケース」のようなシナリオを作らせ、その結果で GOOD/REVIEW/FAIL の境界線を微調整する。
- ・ **言語別テストマトリクス**: 36 言語を語族(ロマンス語・ゲルマン語・膠着語・表意文字)ごとにまとめ、語族ごとによく出るエラータイプを選んで自動生成する。ゲルマン語は複合語が、膠着語は長さ比が厄介、といった特徴が見えてくる。
- ・ **言語別の回帰テスト**: ドイツ語のプロンプトを直したらフランス語が壊れた、といった言語間の干渉はよくある。言語ごとの基準値を JSON で分けて持ち、変更したときは影響を受ける言語だけ再検証する形で防ぐ。

検証表には言語、入力種類、期待条件、実結果、判定、検討者・ツール版を置く。「12.5%を保持」「3段落を保持」「訳だけ出力」「固有名詞が一貫」を別々に判定する。合成文の通過率と実文書の人による評価を分け、未試験言語の精度を根拠なく書かない。

11.7 ユーザーフィードバックのループ

自動検証がどれだけ精巧になっても、実際の利用者が指摘する問題を完全には代替できない。Tauri 2 で作り直したバージョンには、フィードバックボタンを追加した。

機能

- ・各段落の末尾に小さな「この翻訳が怪しい」ボタン。
- ・クリックすると、段落の内容、原文、対象言語、provider 情報がログDBへ送られる。
- ・2週間ごとにログを見直す。

実データでプロンプトを更新

3週間分のログを分析すると、次のような問題が出てきた。

- ・ドイツ語の「複合名詞の分割」 12件
- ・フランス語の「性一致のミス」 8件
- ・スペイン語の「vos/tu の混用」 5件

このデータを Claude に渡して、言語別のプロファイルを更新した。たとえばドイツ語のプロファイルには、「ドイツ語の複合名詞は単一語のまま保つこと(Datenbankverbindung、Datenbank-Verbindung ではなく)。原文がハイフンを明示しない限り分割しない」といったルールを足した。

確認すべき出力: 同じ問題が次の2週間で再び報告されるかどうかを検証シグナルになる。この一行で、同じタイプの問題が12件も再発することはなくなった。**フィードバック -> プロンプト更新** は、費用対効果がはっきりした改善方法だ。

肝になるのは、**モデル A と B を別の provider にすると検証が確かになる** という点だ。生成と検証に同じモデルを使うと、同じバイアスを共有してしまう。Claude で訳して GPT で検証するか、その逆でもよい。

11.9 自分のアプリに応用するコツ

このパターンは翻訳だけのものではない。AIの出力なら、どこでも使える。

コツ1. 単一スコアは禁止。軸を分けよ。

品質を一つのスコアで表すと、何が問題か見えなくなる。意味・長さ・形式・一貫性のように、評価の基準を項目別に分けて、ようやくどの部分が悪いかすぐにわかる。

コツ2. 生成モデルと検証モデルを別にする。

同じモデルは同じやり方で失敗する。Claude が生成した結果は GPT が、OpenAI が生成した結果は Gemini が検証する、といった構成にする。モデルが違えば、同じエラーを揃って見落とす状況を減らせる。

コツ3. しきい値は三段階で。

合格・不合格の二つだけにすると、基準が必要以上に厳しくなる。良い・要確認・失敗の三段階に分ければ、人が直接見るべき量を現実的に調整できる。

コツ4. ユーザーフィードバックをプロンプトに反映する。

自動検証で取り逃した問題は、ユーザーが見つけてくれる。2週間ごとにフィードバックログを集めて、言語別のプロンプトプロファイルを更新する。少ない手間で出力品質を着実に上げていける方法だ。

11.10 この章のまとめ

平易な言葉で並べ直すと次の四つだ。

1. **品質は一つのスコアでは測れない。** 意味・長さ・用語、最低でも三つの軸に分けて見るしかない。
2. **作るAIと検査するAIは別にする。** 同じモデルは同じミスをする。
3. **合格/不合格の二段階では粗すぎる。** GOOD / REVIEW / FAIL の三段階が、人が実際に運用しやすい。
4. **自動検証で終わりではない。** ユーザーフィードバックを集めてプロンプトを定期的
に直し続けて、はじめて品質が保たれる。

この章の運用原則は次のとおりだ。

パターン: AIの出力バイアスを捕まえたいなら、別のAIモデルで検証せよ

アーキテクチャ: 複雑な出力には、単一品質指標より多軸スコアのほうが向く

テスト: AIでテストケースを生成して全範囲を自動検証し、しきい値を変える前には必ず回帰テストを先に走らせる

運用: 言語ごとの基準値で言語間の干渉を防ぎ、実ユーザーのフィードバックで継続的に補正せよ

11.11 Part II と Part III を振り返って

Part II(Mandai) と Part III(readbooks.ai)、二つのアプリが共通して示してくれたものがある。**AIが実装のスピードを上げてくれても、方向を自分で決めなければ、速い速度で間違った方向へ進む。** 要求 -> 実装 -> 検証のループは、スピードと方向を同時に押さえるための仕組みだ。

- ・ Part II では、ループは主に **コードの正しさ** を扱った。コンパイルが通るか、動くか、エッジケースを取りこぼしていないか。
- ・ Part III では、ループは **出力の品質** を扱った。翻訳に正解はない。評価基準を自分で設計するしかなかった。

11.12 次章予告

次の章では、翻訳品質の検証を越えて、ダッシュボード規模のプロジェクトへ移る。検証の焦点も、言語的な正確さから、データの正確さとリアルタイムレンダリングへと変わっていく。